



US012400618B2

(12) **United States Patent**  
**Konduru**

(10) **Patent No.: US 12,400,618 B2**

(45) **Date of Patent: Aug. 26, 2025**

(54) **AVOIDING FLICKER IN ASSOCIATION  
WITH HOTPLUG OF A DISPLAY**

3/00; G06F 3/02; G06F 3/038; G06F  
3/0489; G06F 3/14; G06F 3/34; G06F  
3/42; G06F 3/10; G06F 3/147;

(71) Applicant: **Intel Corporation**, Santa Clara, CA  
(US)

(Continued)

(72) Inventor: **Chandra Mohan Konduru**, Folsom,  
CA (US)

(56)

**References Cited**

U.S. PATENT DOCUMENTS

(73) Assignee: **Intel Corporation**, Santa Clara, CA  
(US)

2011/0037773 A1\* 2/2011 Ishioka ..... G09G 5/393  
345/545  
2013/0120222 A1\* 5/2013 Graham ..... G09G 5/399  
345/1.1

(\*) Notice: Subject to any disclaimer, the term of this  
patent is extended or adjusted under 35  
U.S.C. 154(b) by 699 days.

(Continued)

(21) Appl. No.: **17/557,975**

*Primary Examiner* — Chanh D Nguyen

*Assistant Examiner* — Nguyen H Truong

(22) Filed: **Dec. 21, 2021**

(74) *Attorney, Agent, or Firm* — Jaffery Watson Hamilton  
& DeSanctis LLP

(65) **Prior Publication Data**

US 2023/0197030 A1 Jun. 22, 2023

(57)

**ABSTRACT**

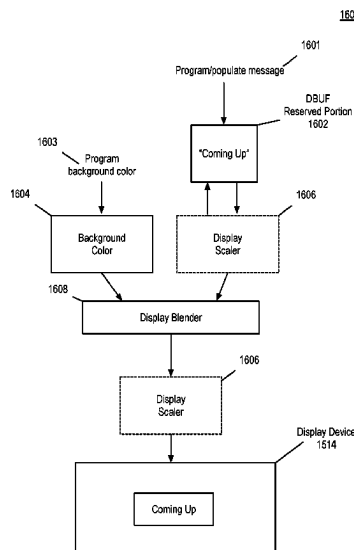
(51) **Int. Cl.**  
**G06T 1/60** (2006.01)  
**G09G 5/00** (2006.01)  
(Continued)

One embodiment provides a graphics processor device that includes circuitry configured to detect a connection of a second display device to a display subsystem of the graphics processor while a first display device of the graphics subsystem is active, write pre-determined pixel data to a reserved portion of memory associated with the display subsystem, configure timings for the second display device while resources allocated to the first display device remain available to the first display device, display the pre-determined pixel data from the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem to enable output the framebuffer data to the second display device, and transition the second display device from the display of the pixel data in the reserved portion of the memory to the display of the framebuffer data after resources of the display subsystem are reallocated.

(52) **U.S. Cl.**  
CPC ..... **G09G 5/006** (2013.01); **G09G 5/026**  
(2013.01); **G09G 2310/08** (2013.01);  
(Continued)

(58) **Field of Classification Search**  
CPC ..... G09G 5/00; G09G 5/02; G09G 5/006;  
G09G 5/026; G09G 5/10; G09G 5/36;  
G09G 5/39; G09G 5/393; G09G 5/395;  
G09G 3/20; G09G 3/36; G09G 3/3266;  
G09G 2310/08; G09G 2320/0247; G09G  
2360/18; G09G 2360/121; G09G  
2370/22; G06T 1/20; G06T 1/60; G06T  
3/18; G06T 3/40; G06T 5/80; G06T  
11/00; G06T 15/00; G06F 1/16; G06F

**20 Claims, 30 Drawing Sheets**



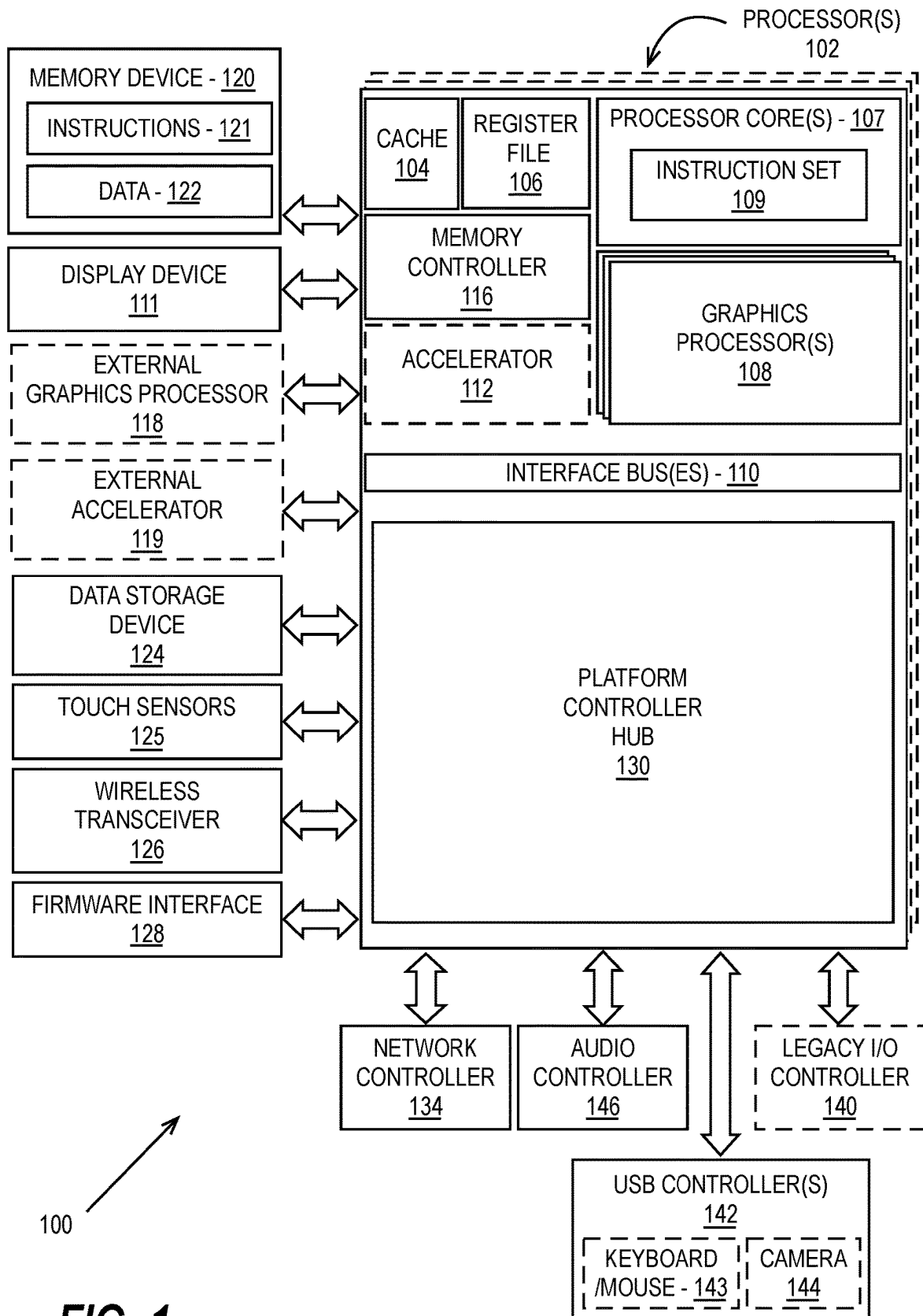
- (51) **Int. Cl.**  
**G09G 5/02** (2006.01)  
**G09G 5/393** (2006.01)
- (52) **U.S. Cl.**  
CPC ..... *G09G 2320/0247* (2013.01); *G09G 2360/121* (2013.01); *G09G 2360/18* (2013.01);  
*G09G 2370/22* (2013.01)
- (58) **Field of Classification Search**  
CPC ..... G06F 9/448; G06F 9/455; G06F 15/80;  
G06F 21/00; G02F 1/19; G02F 1/315;  
G02F 1/1681  
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

|              |      |         |         |       |                        |
|--------------|------|---------|---------|-------|------------------------|
| 2014/0253564 | A1 * | 9/2014  | Redman  | ..... | G09G 5/363<br>345/502  |
| 2016/0189335 | A1 * | 6/2016  | Roy     | ..... | G06F 3/1423<br>345/545 |
| 2019/0303083 | A1 * | 10/2019 | Shekhar | ..... | G09G 5/393             |

\* cited by examiner

**FIG. 1**

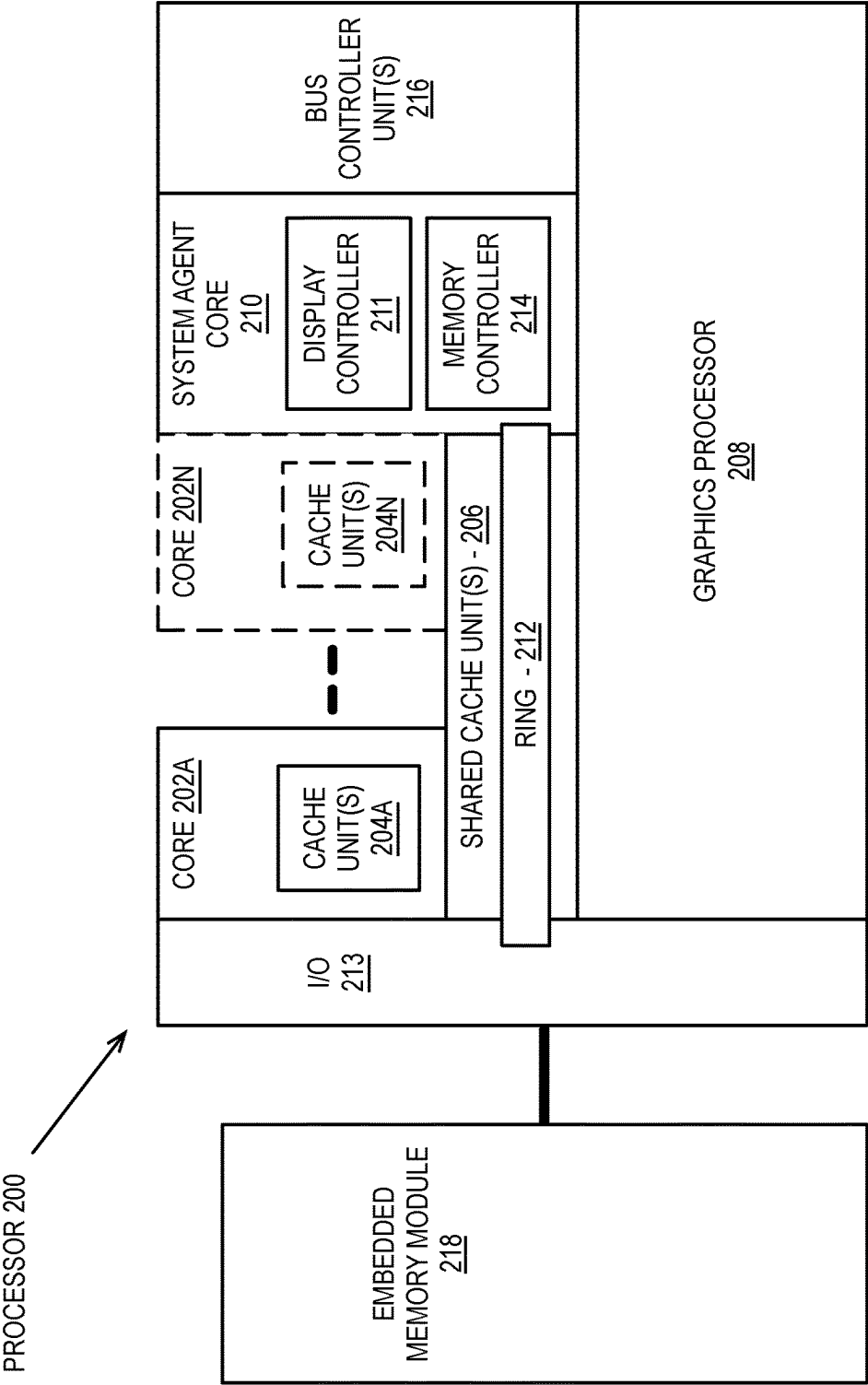
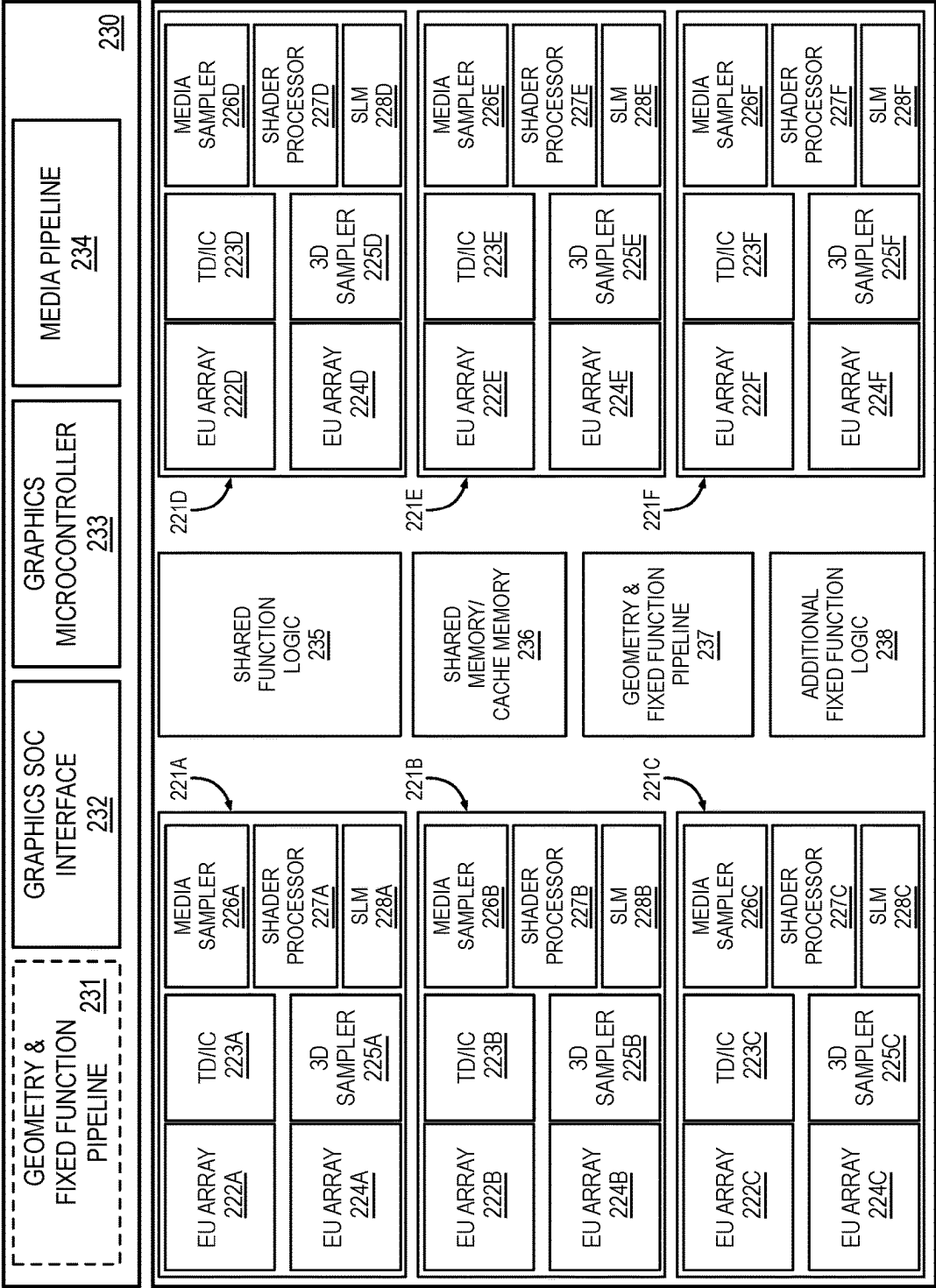


FIG. 2A

219



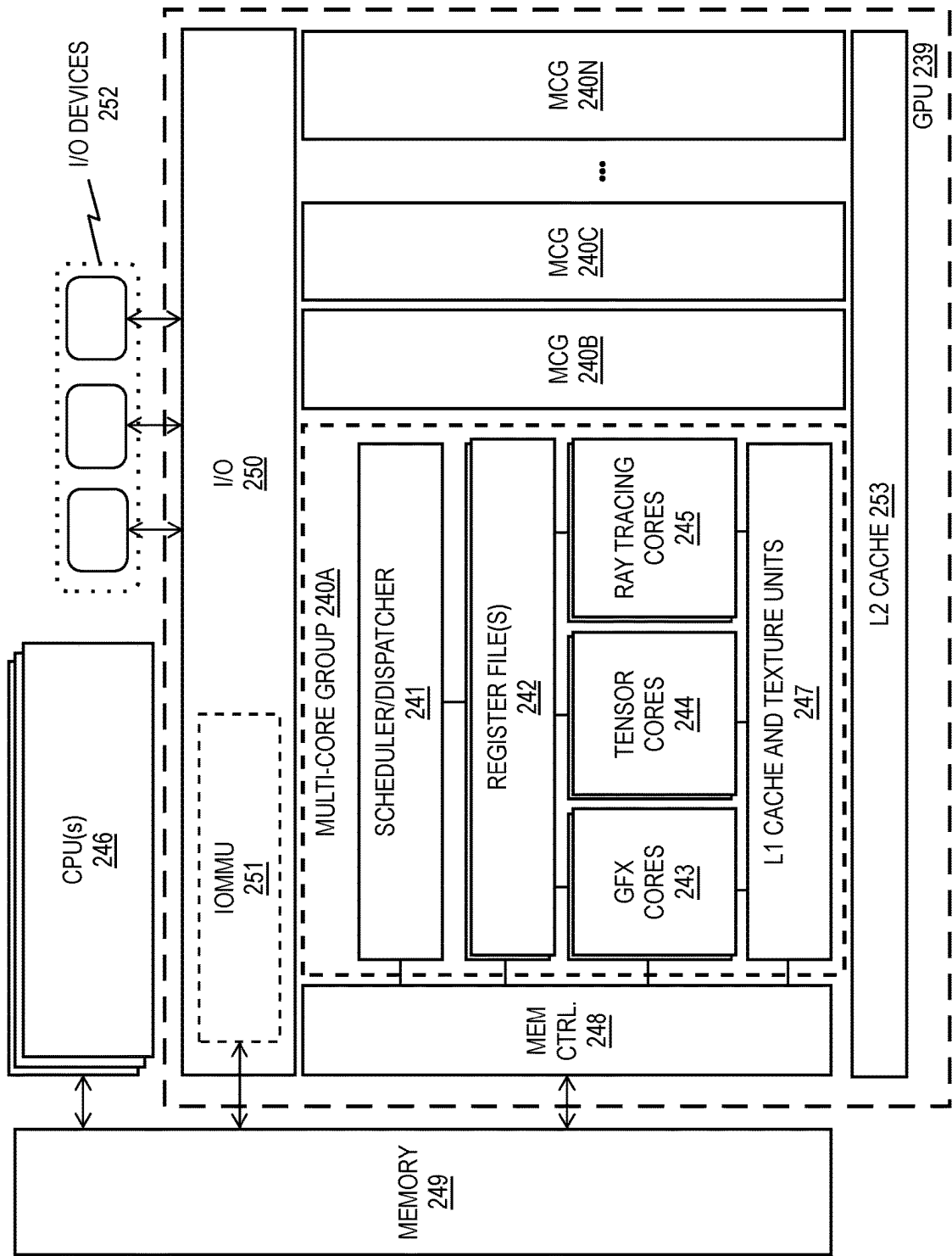


FIG. 2C

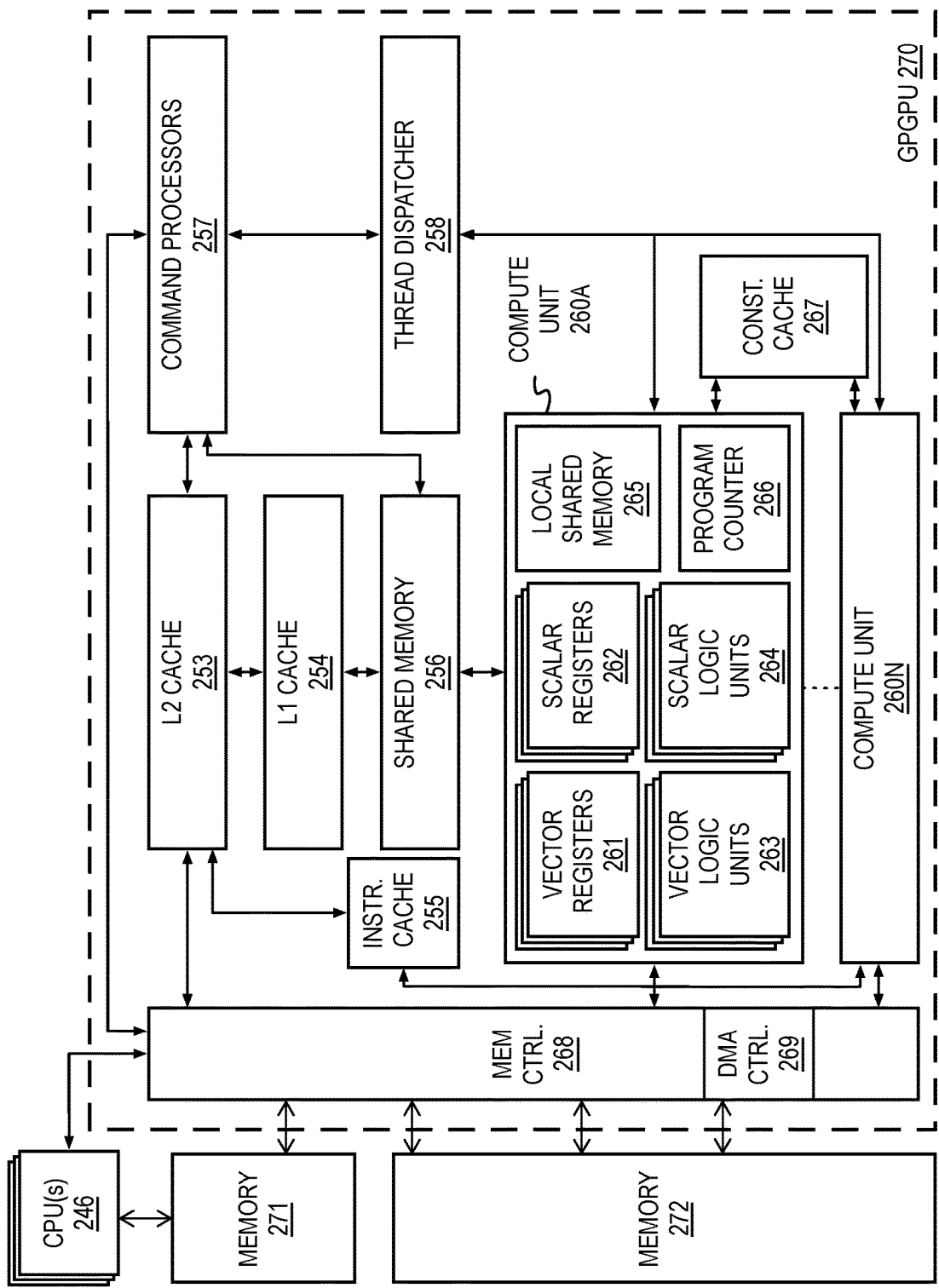


FIG. 2D

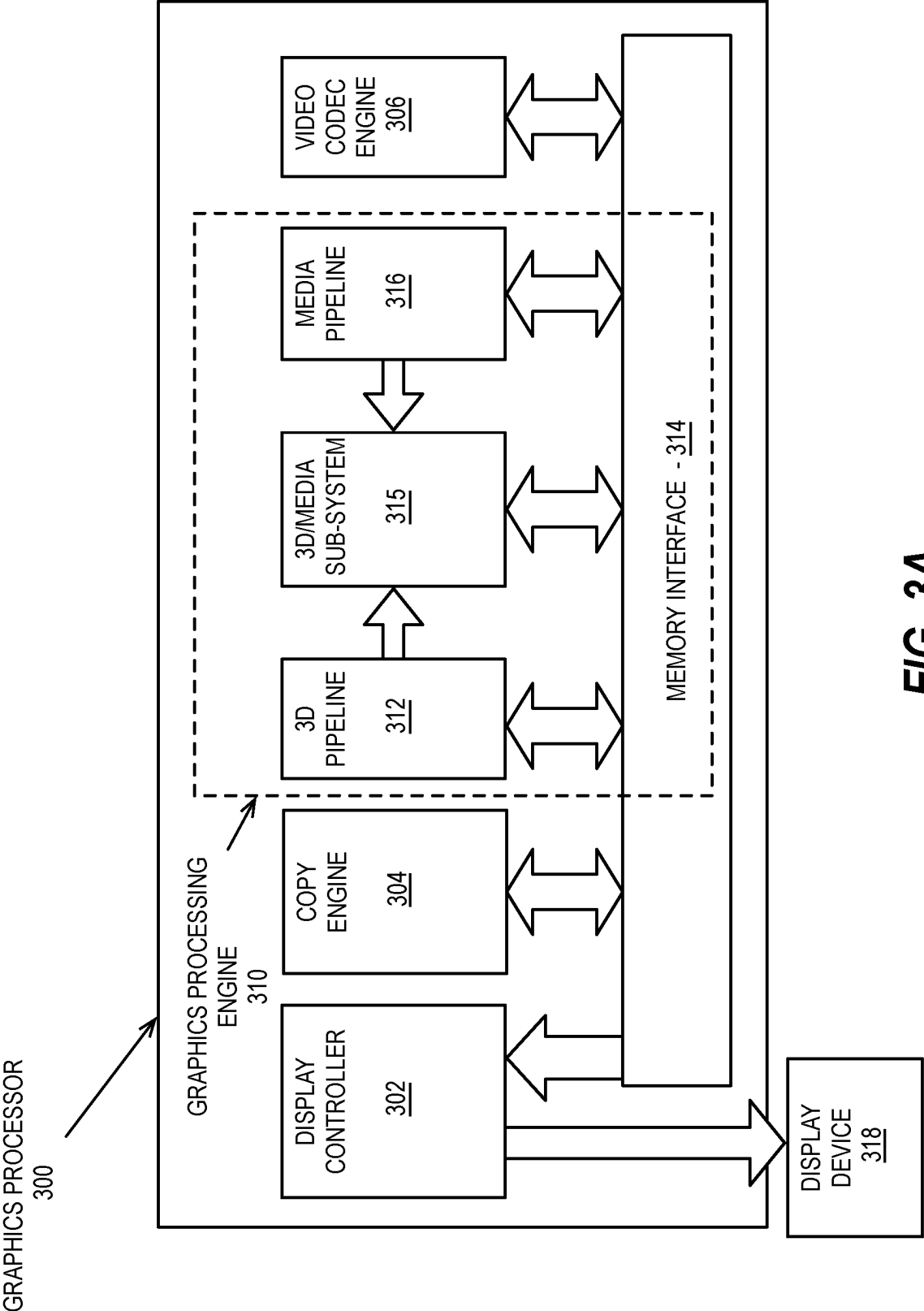


FIG. 3A



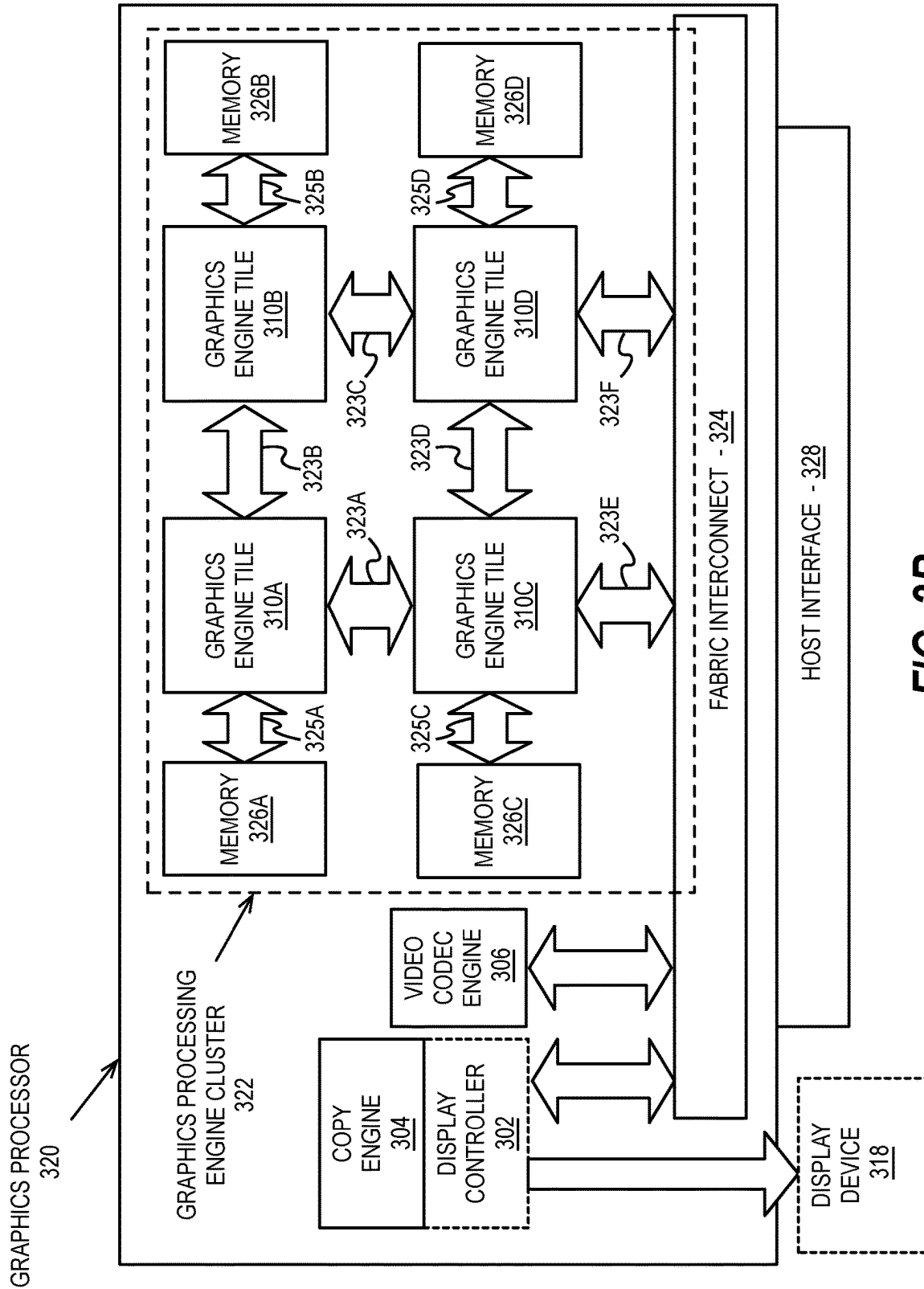
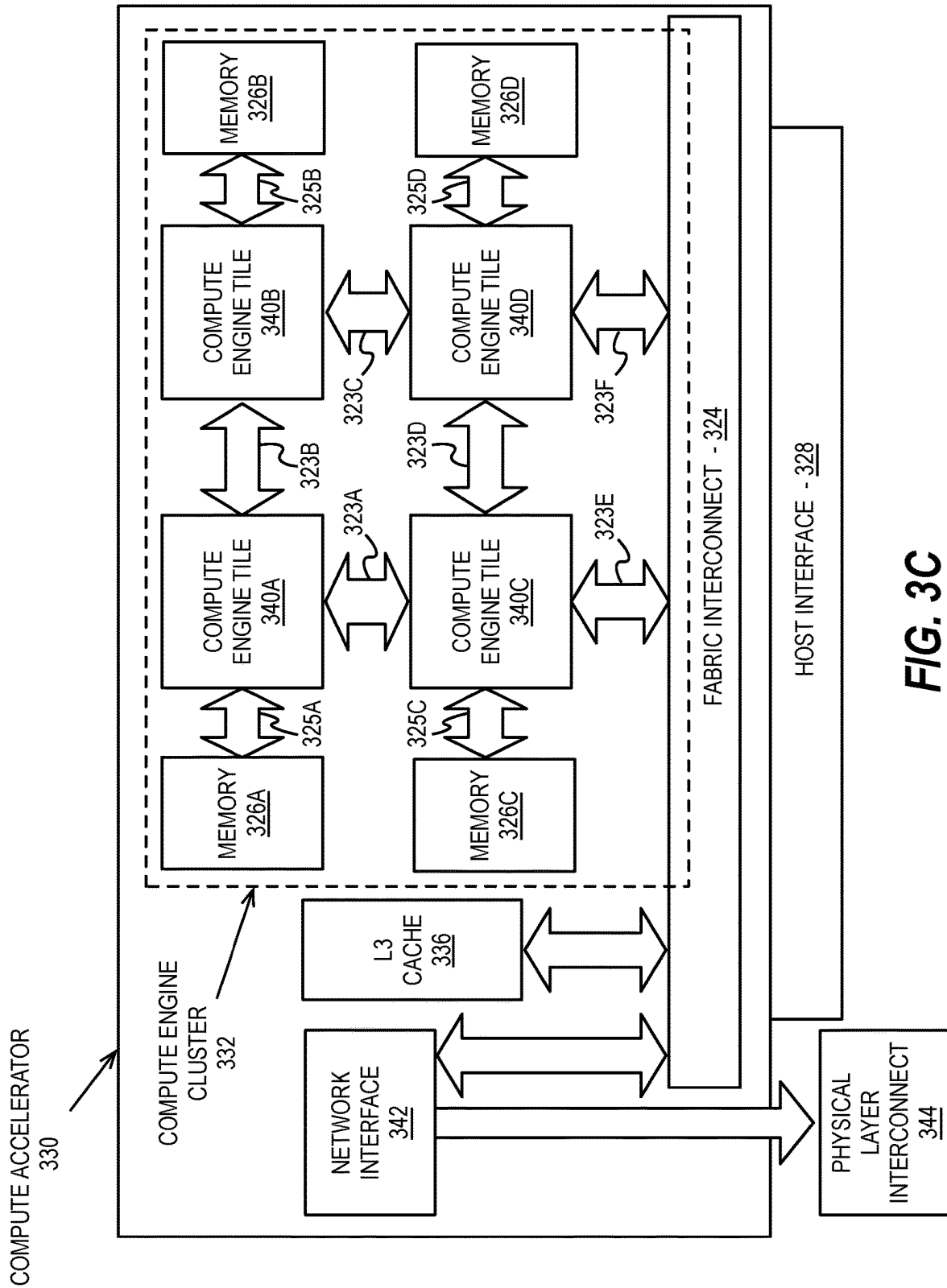


FIG. 3B



**FIG. 3C**

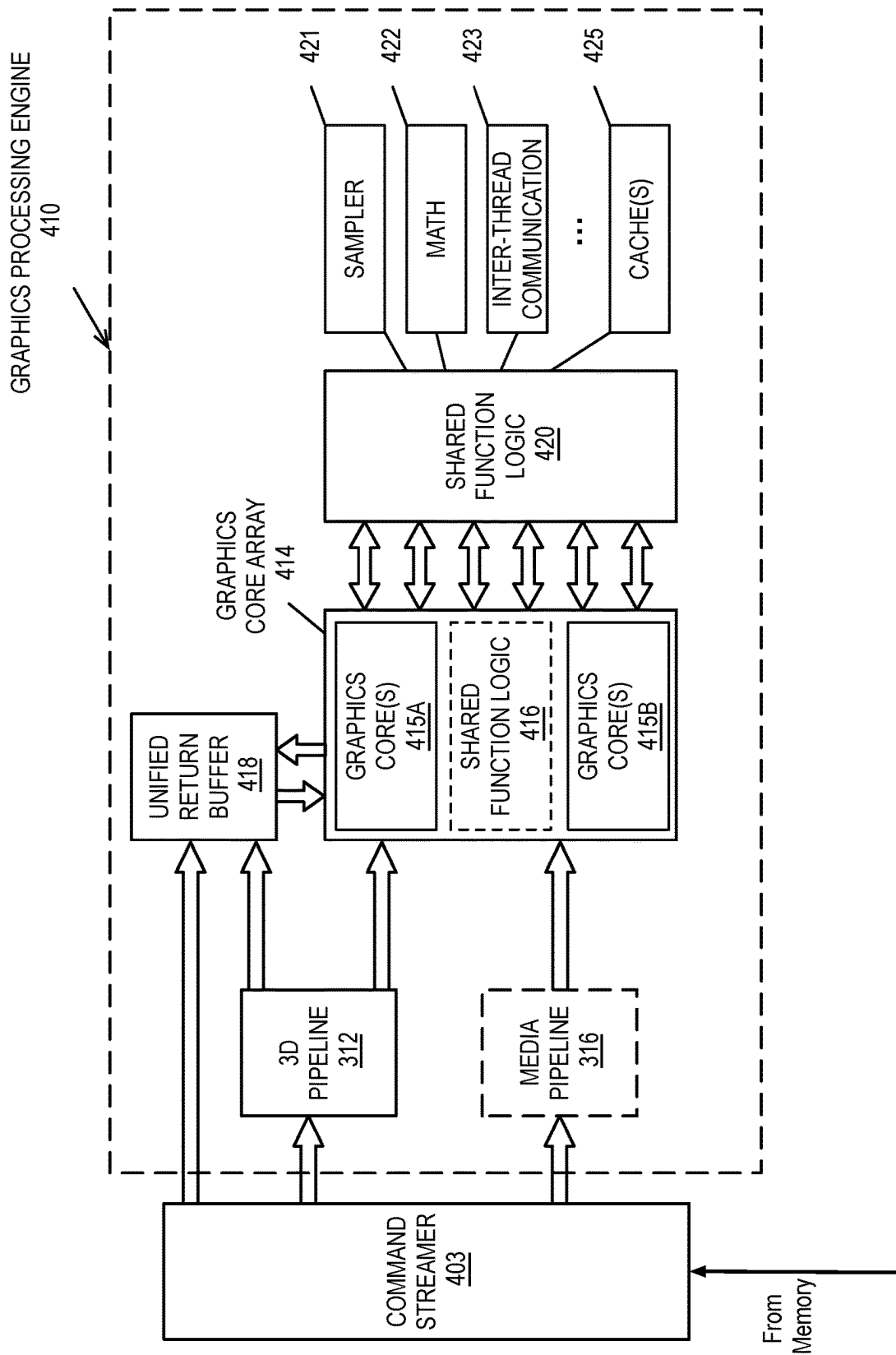


FIG. 4

EXECUTION LOGIC  
500

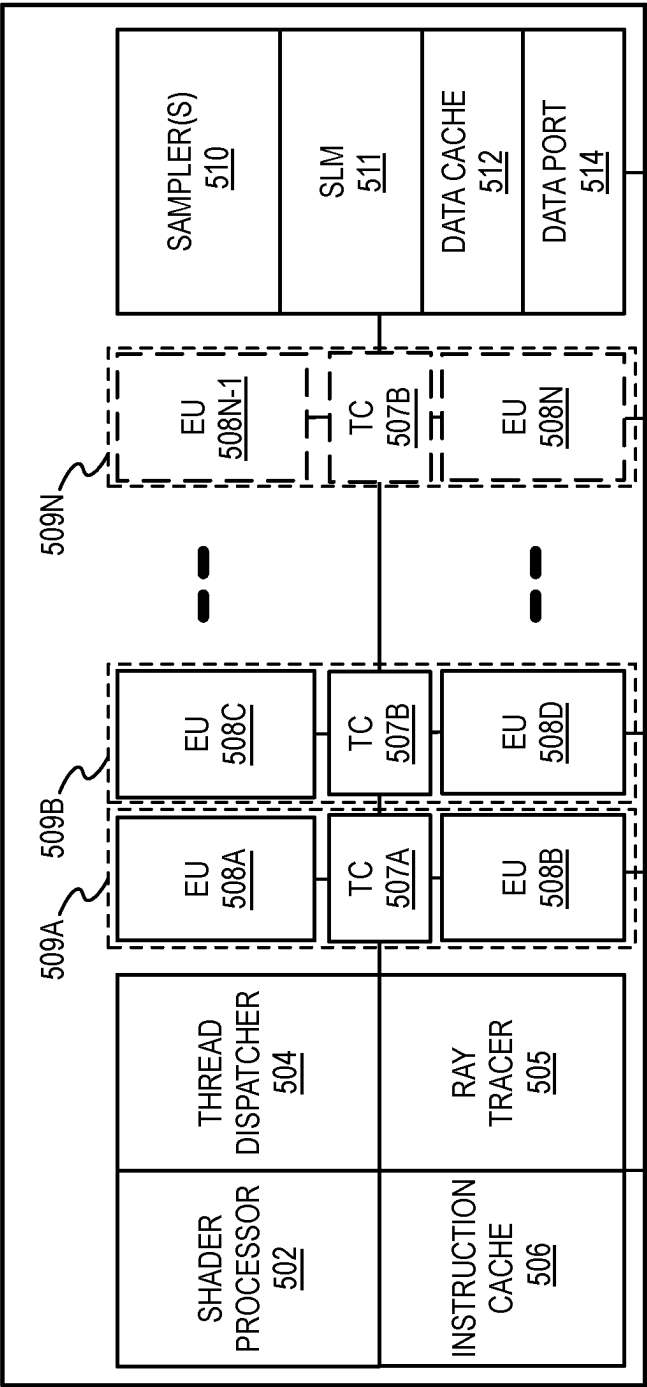


FIG. 5A

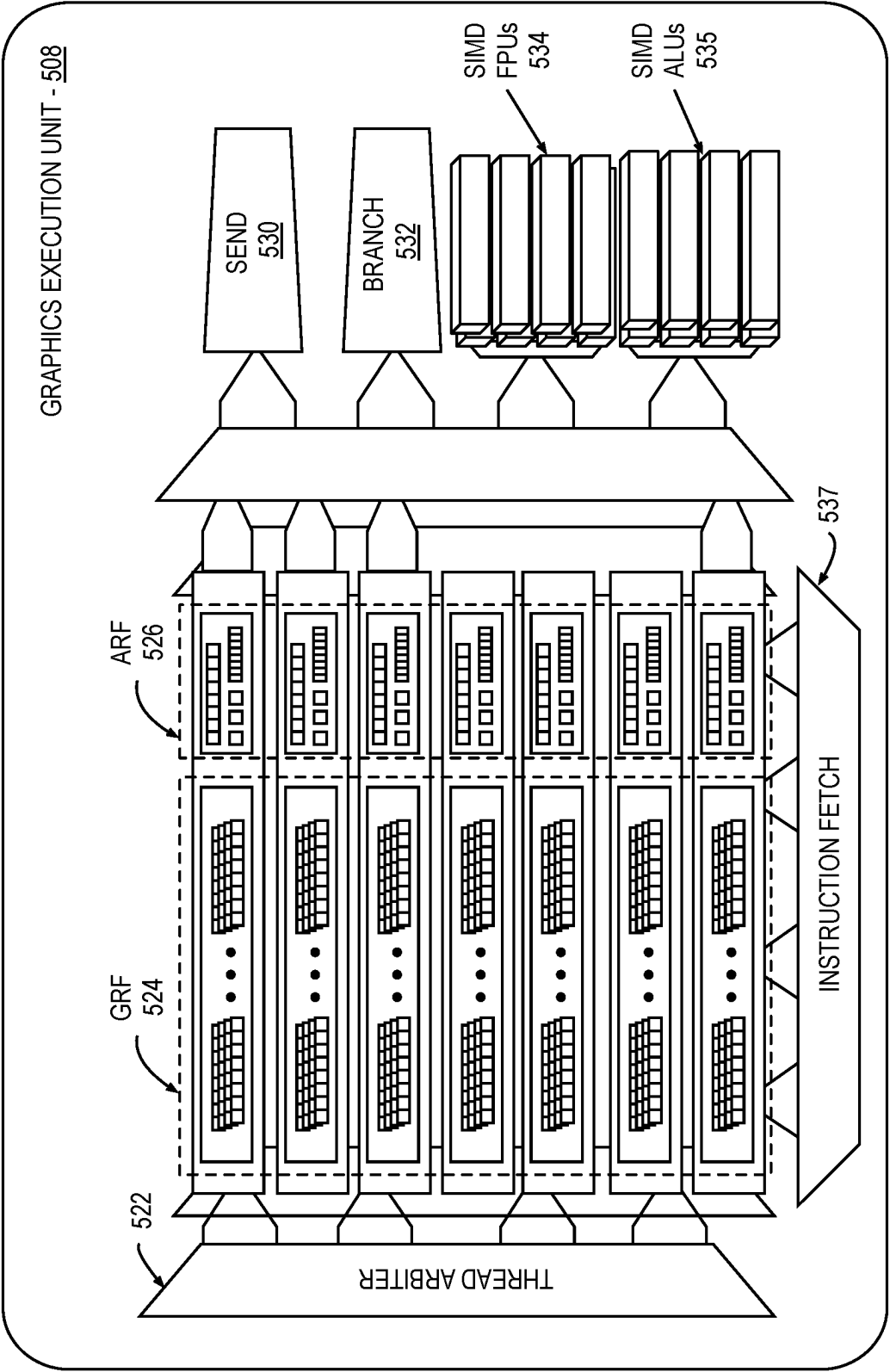


FIG. 5B

EXECUTION UNIT  
600

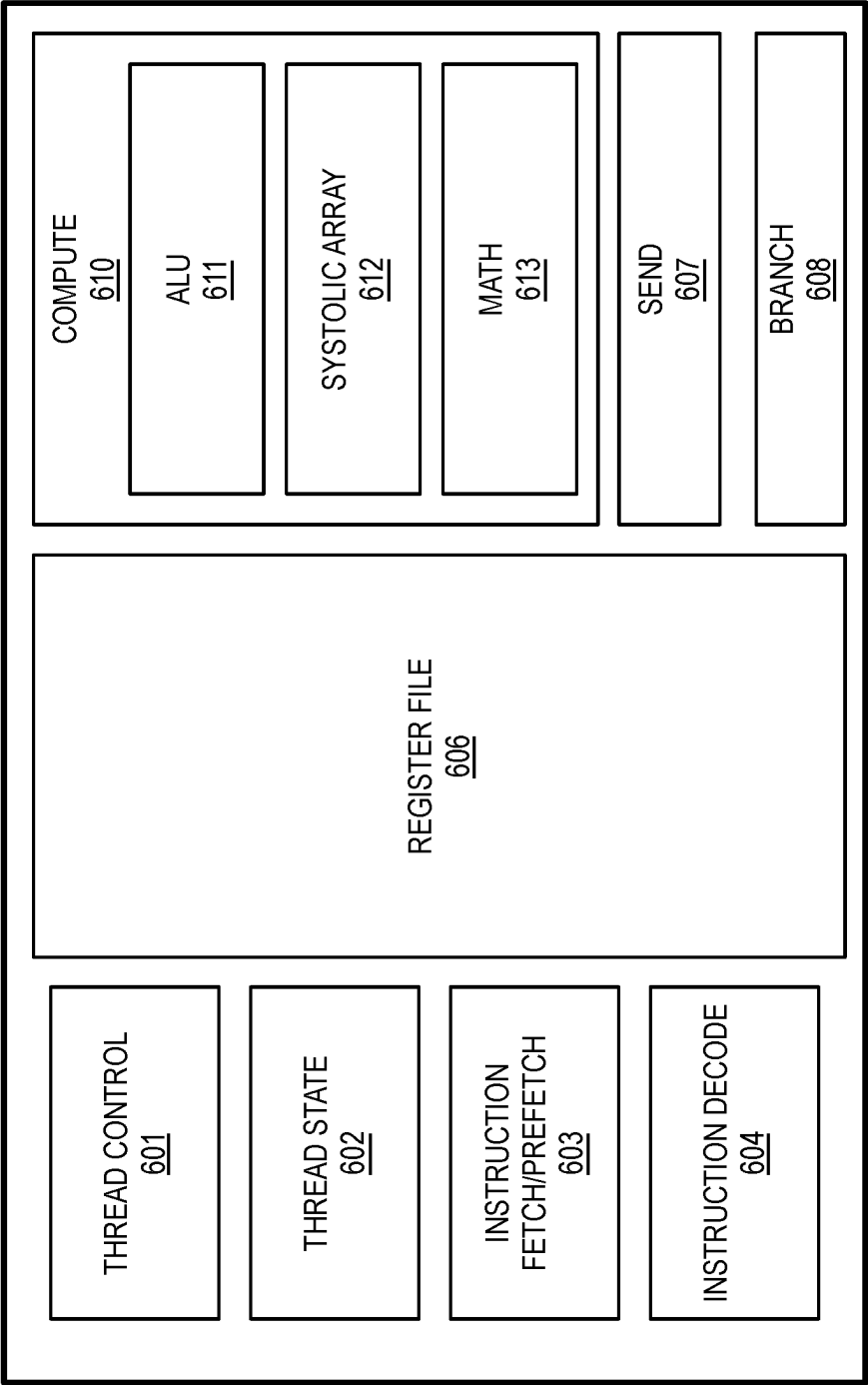


FIG. 6

GRAPHICS PROCESSOR INSTRUCTION FORMATS

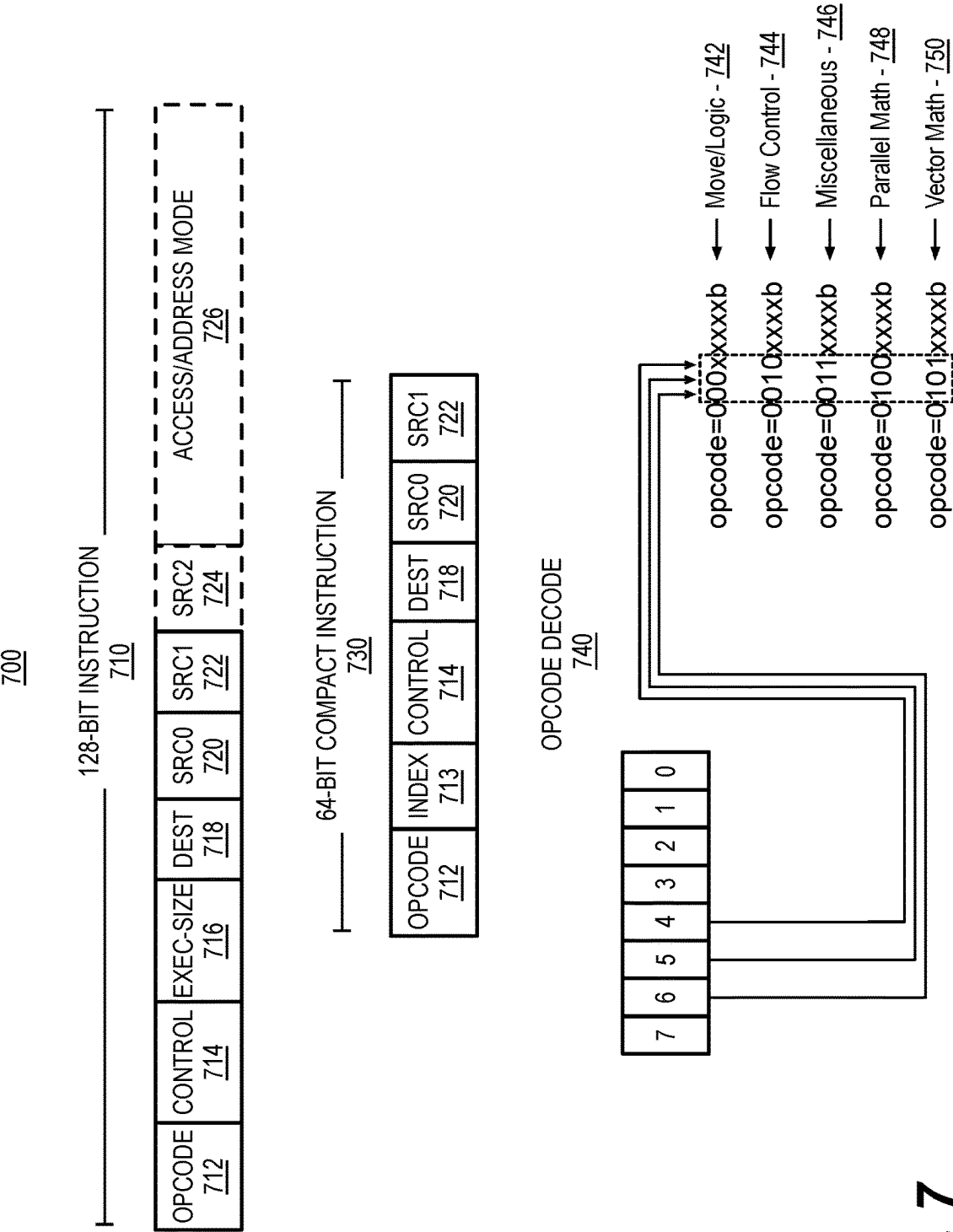


FIG. 7

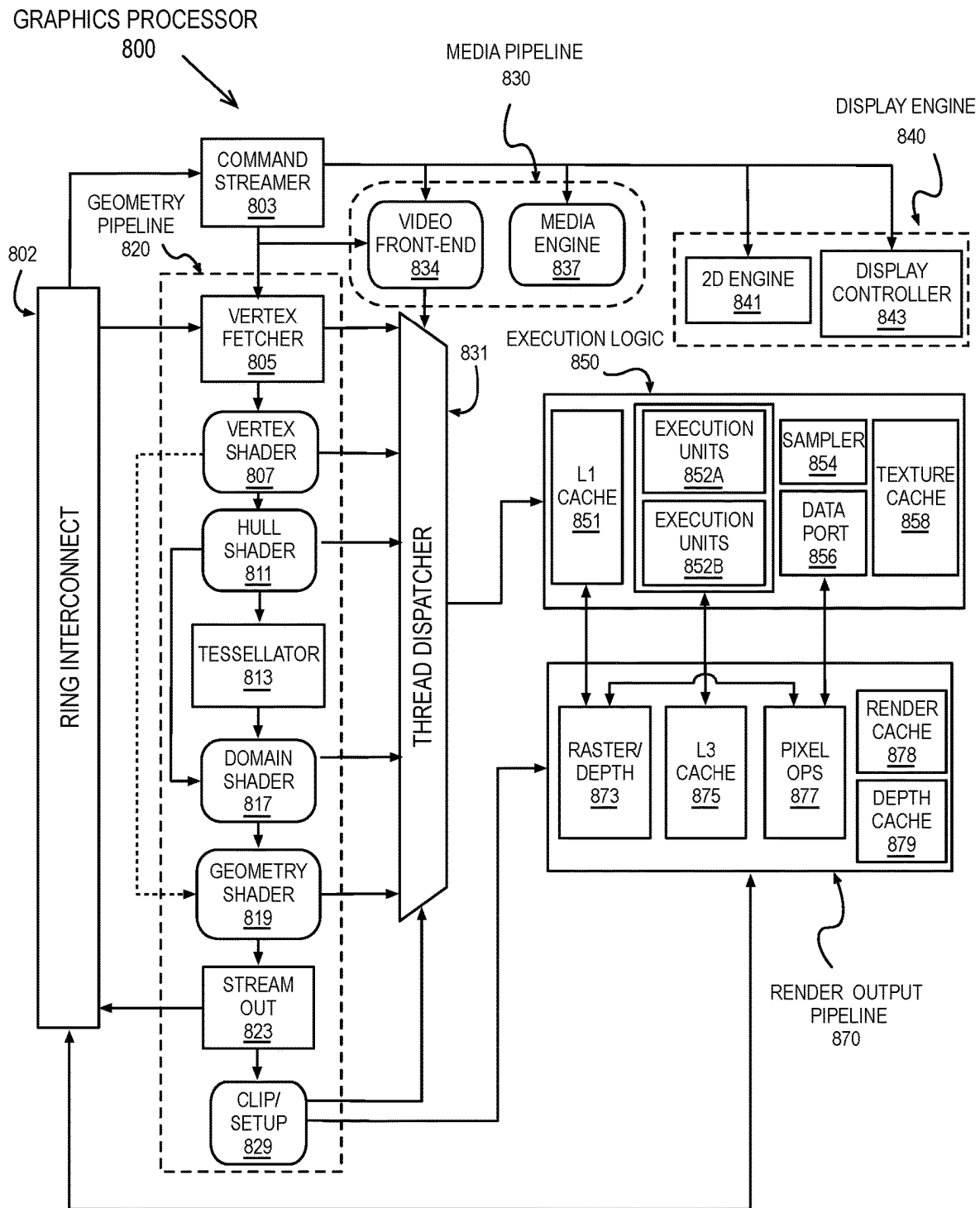
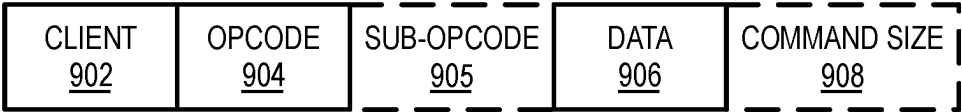


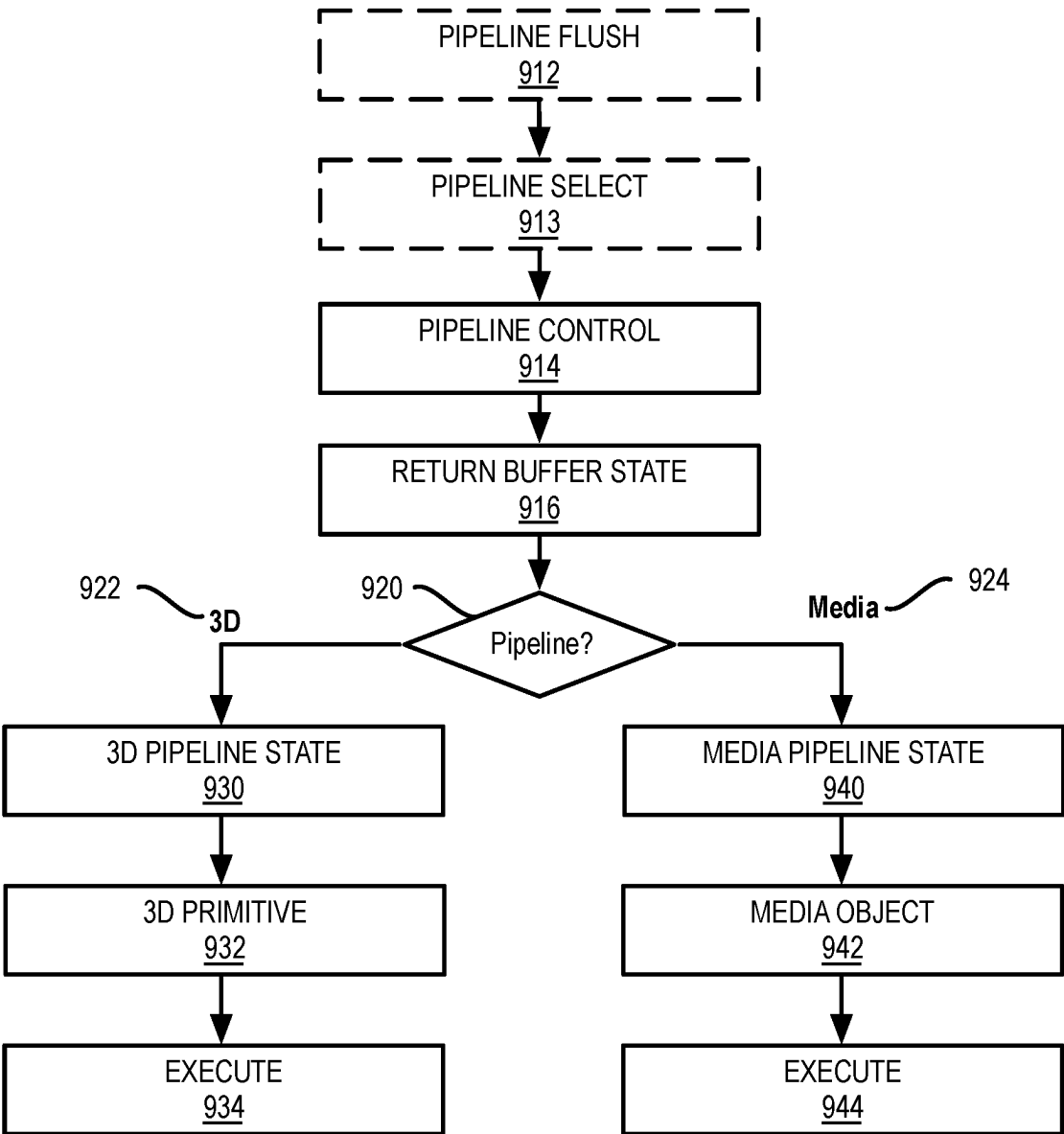
FIG. 8

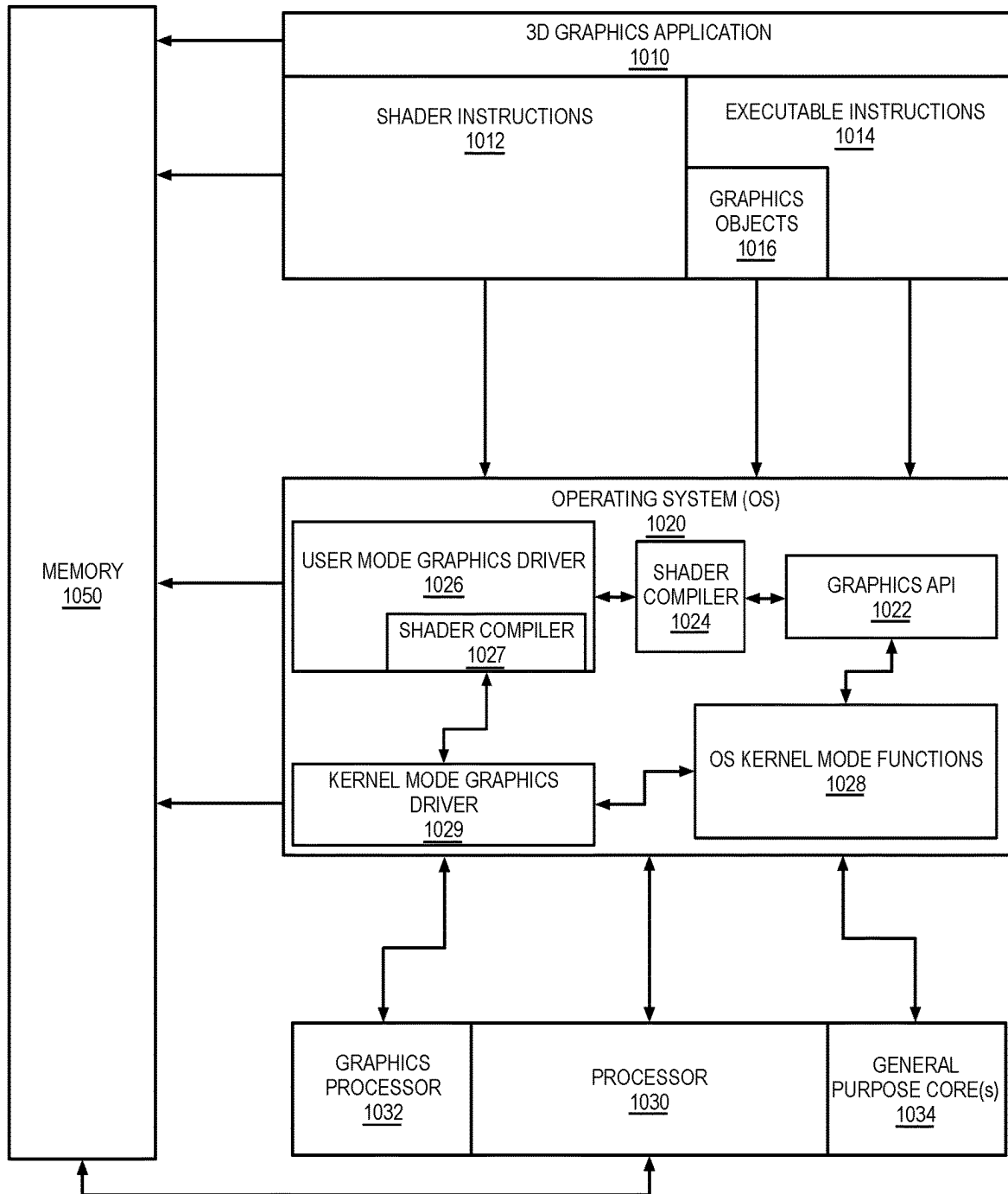


**FIG. 9A**      GRAPHICS PROCESSOR COMMAND FORMAT  
900

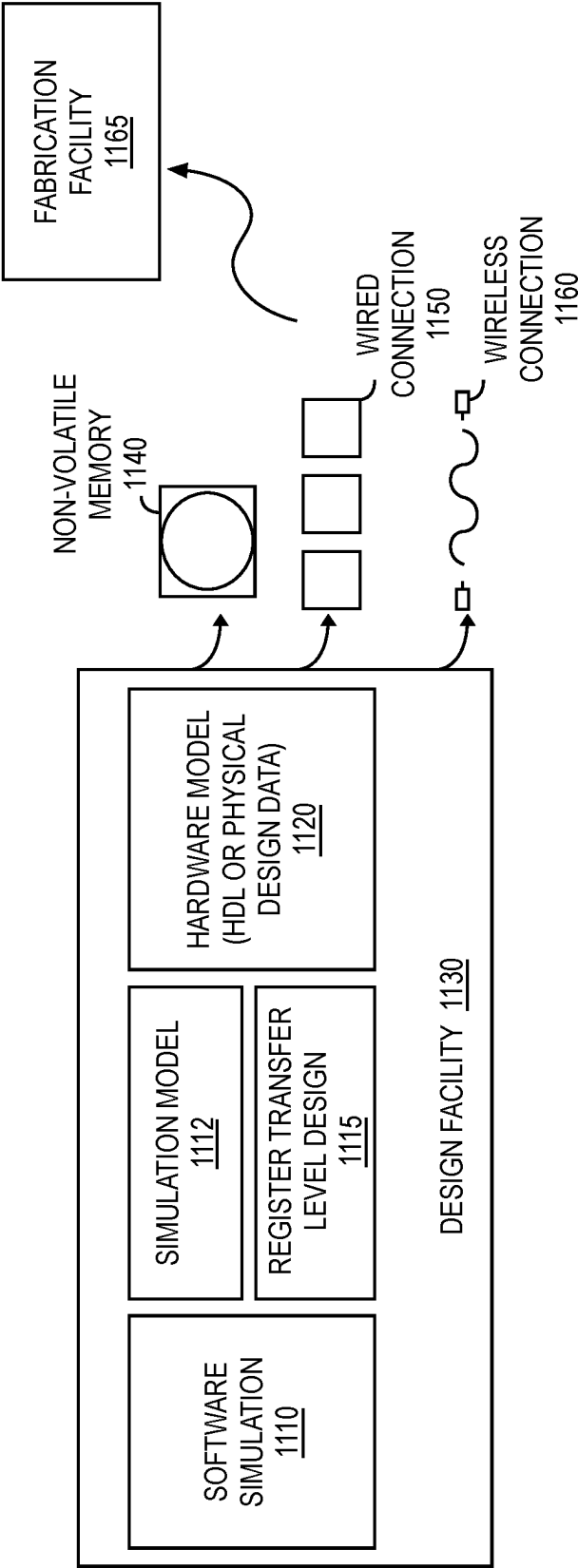


**FIG. 9B**      GRAPHICS PROCESSOR COMMAND SEQUENCE  
910

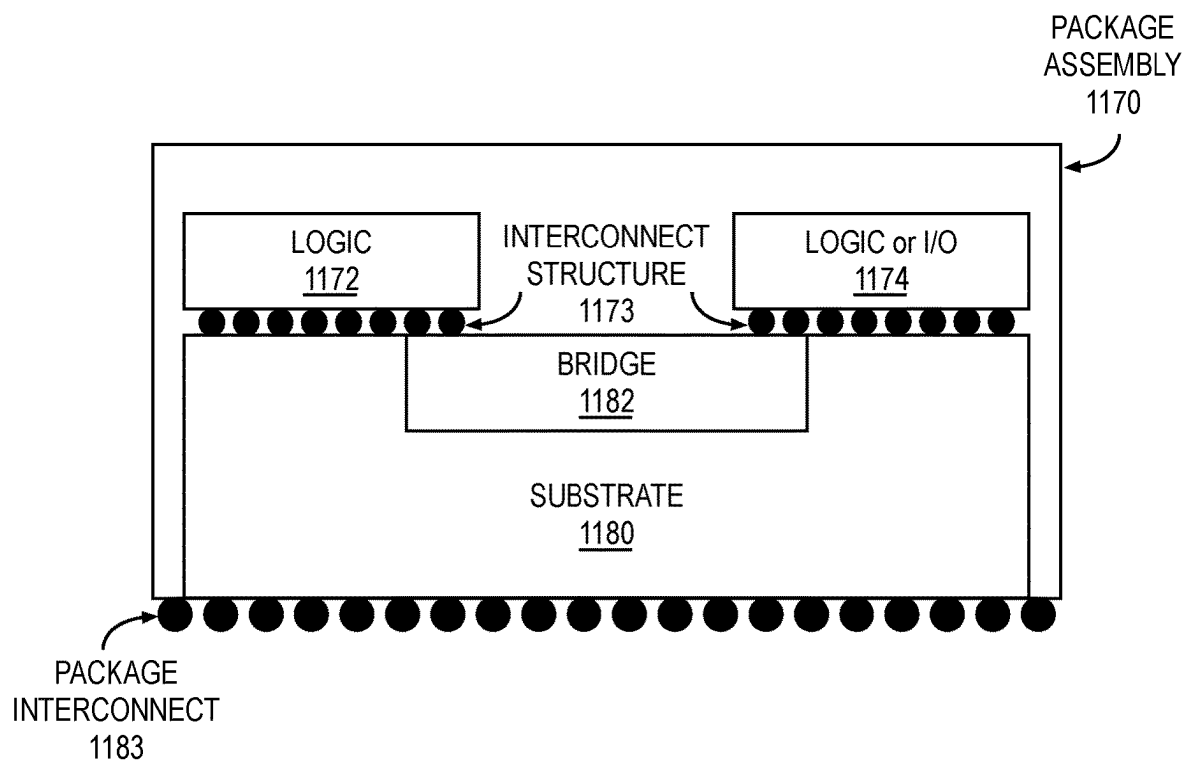


DATA PROCESSING SYSTEM  
1000**FIG. 10**

IP CORE DEVELOPMENT - 1100



**FIG. 11A**

**FIG. 11B**

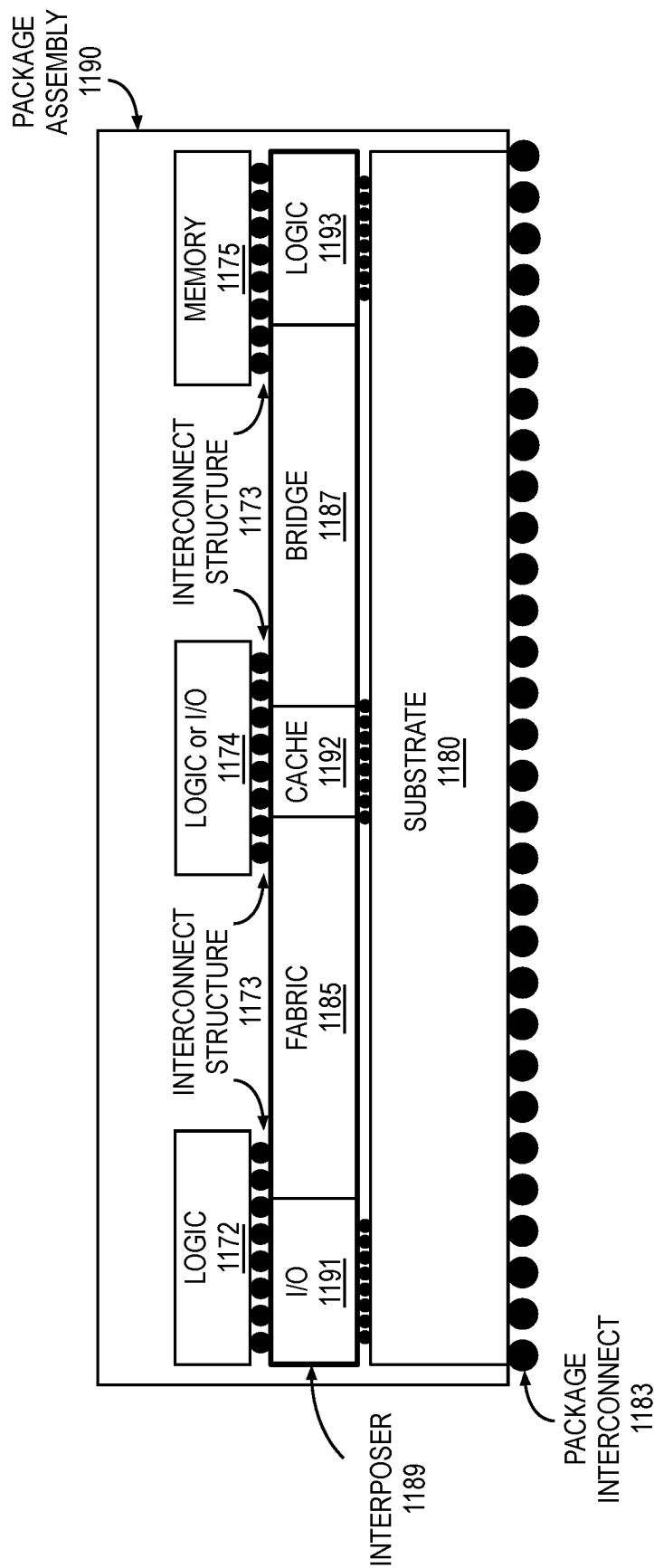
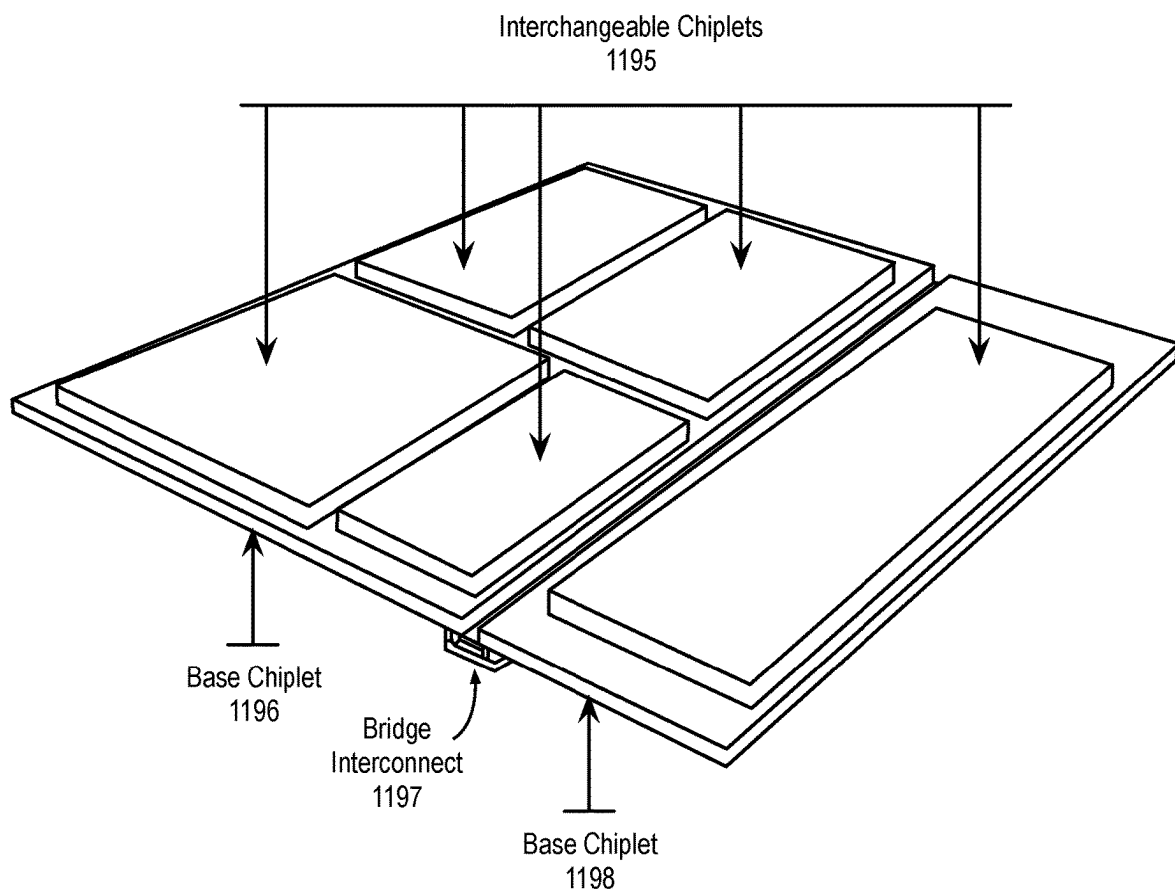
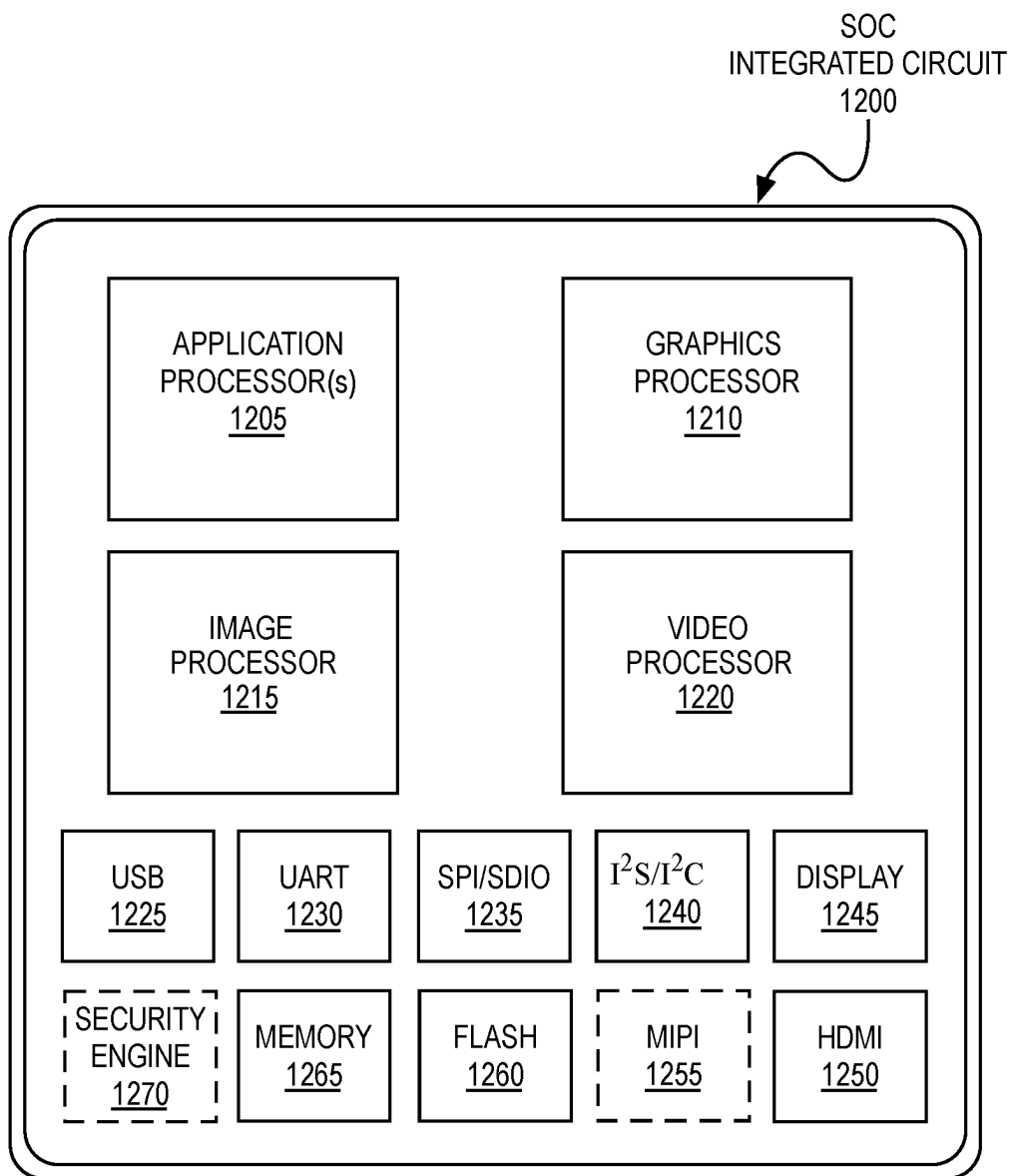


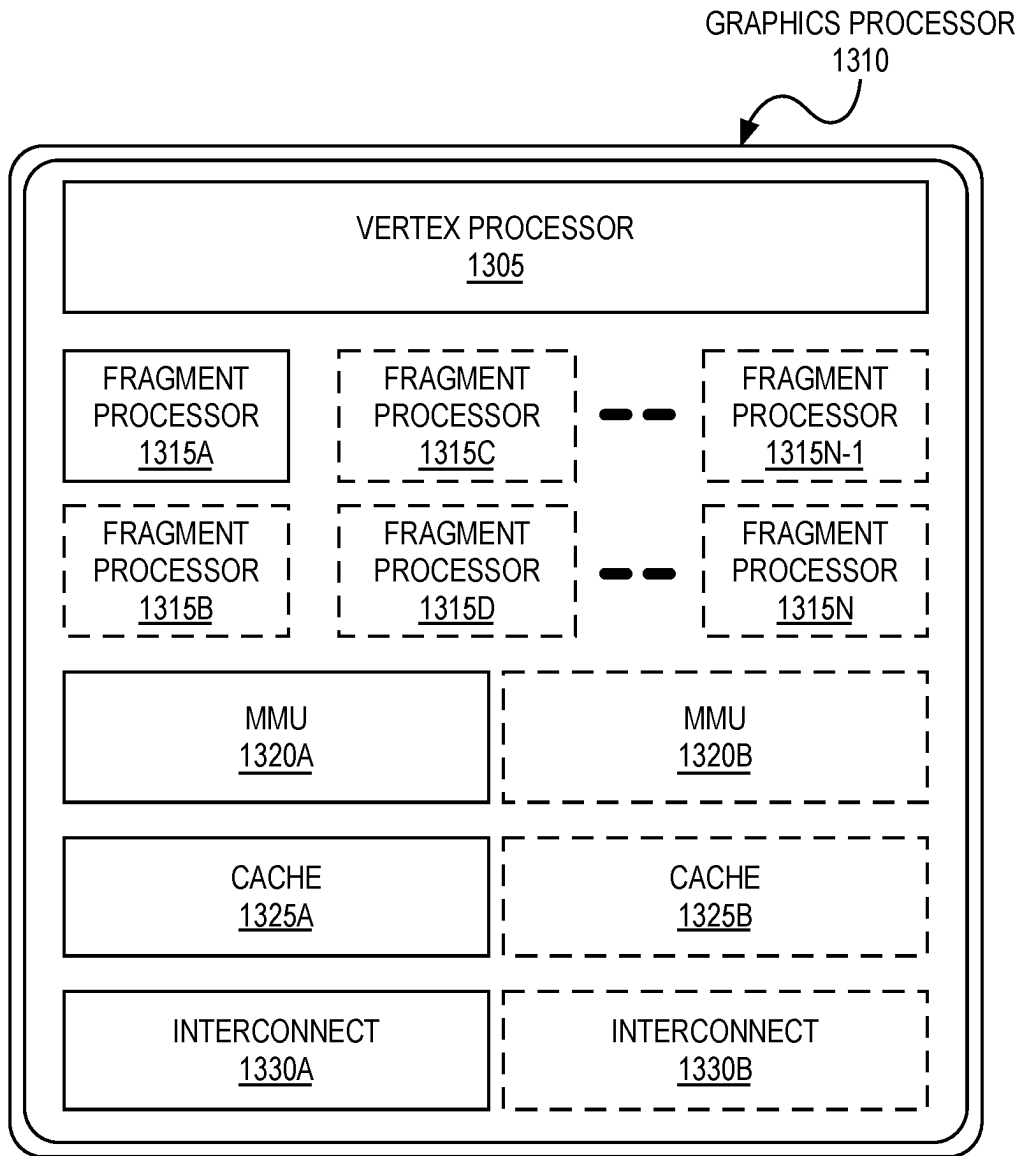
FIG. 11C

1194



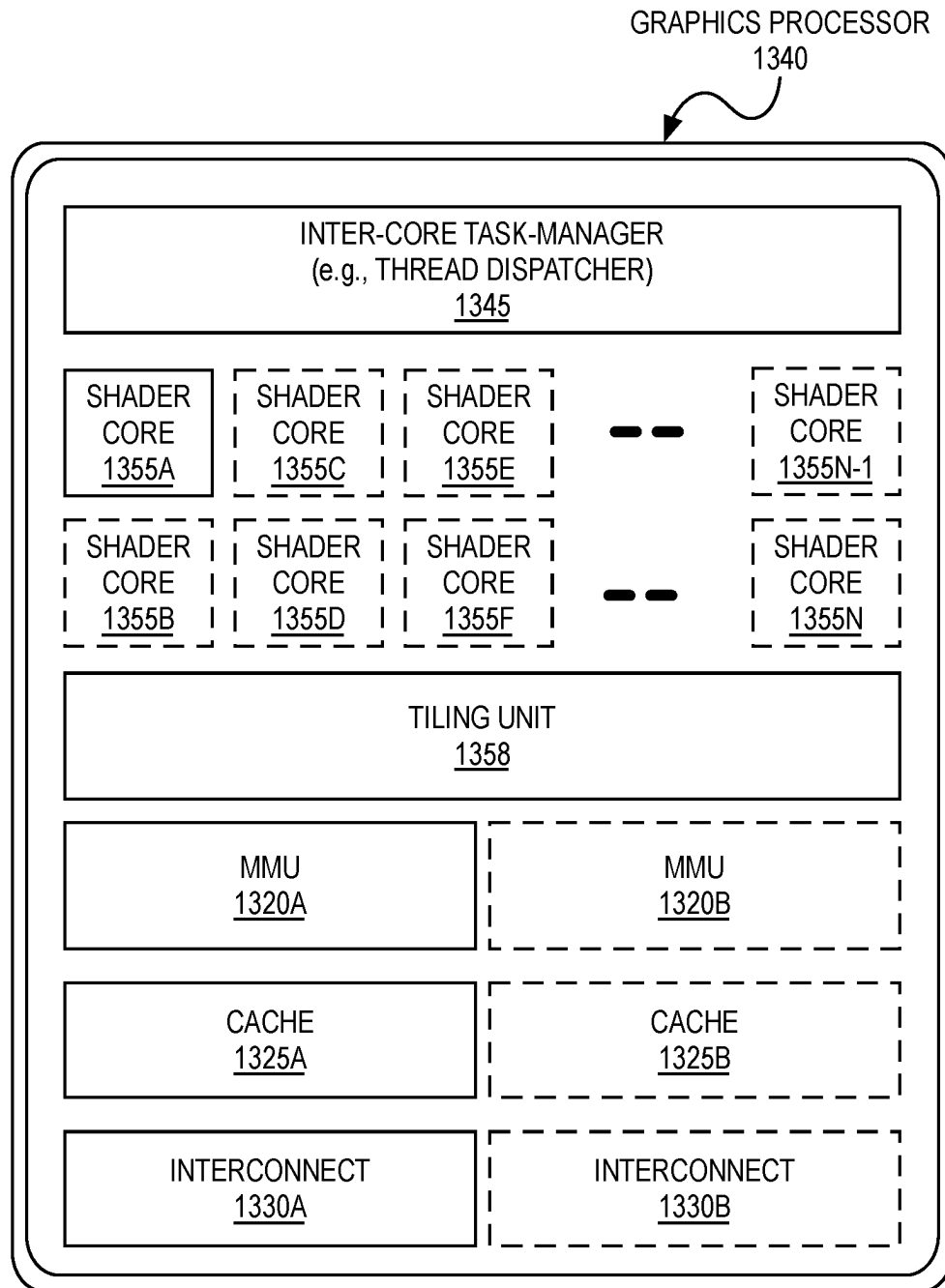
**FIG. 11D**

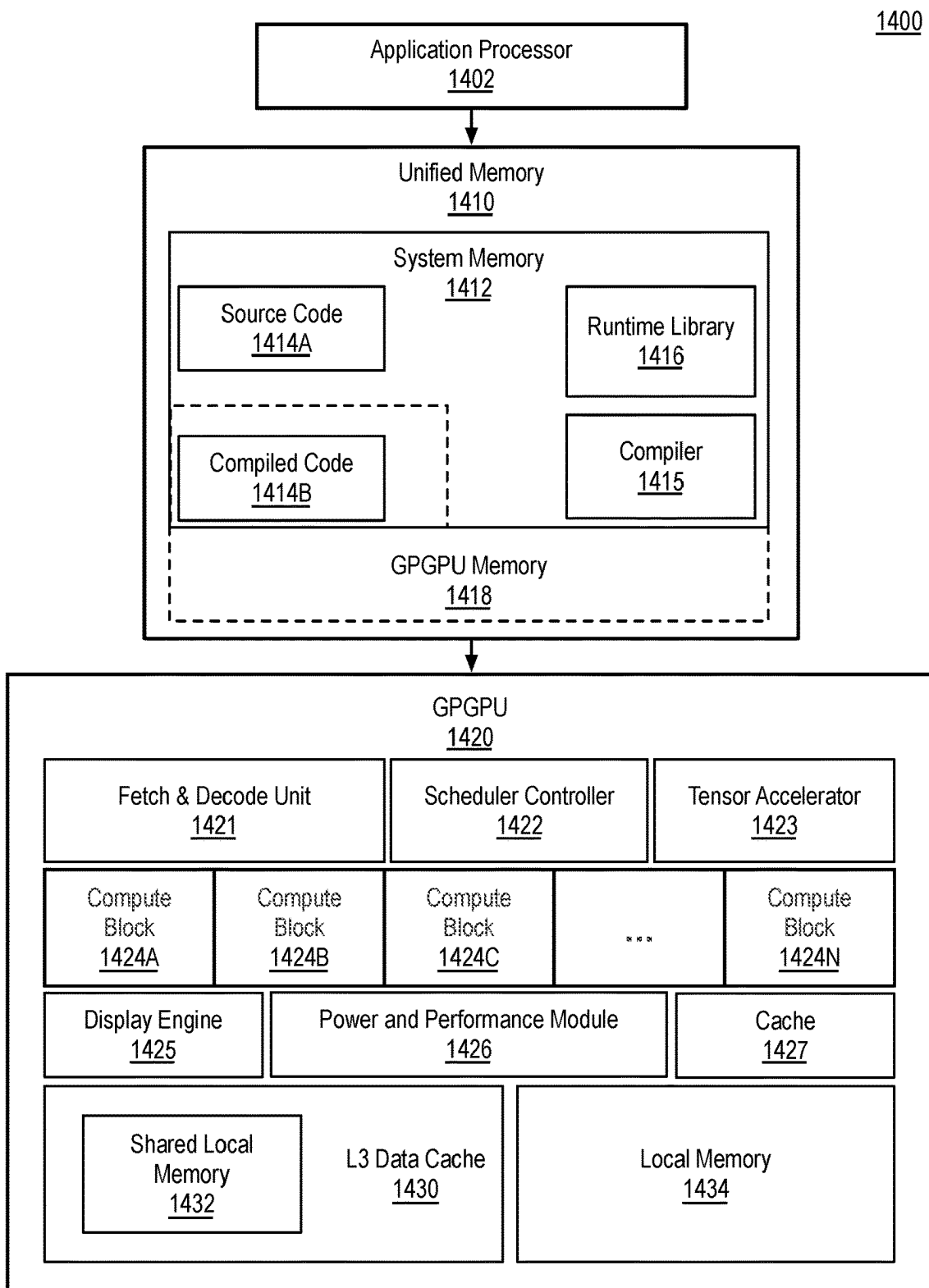
**FIG. 12**

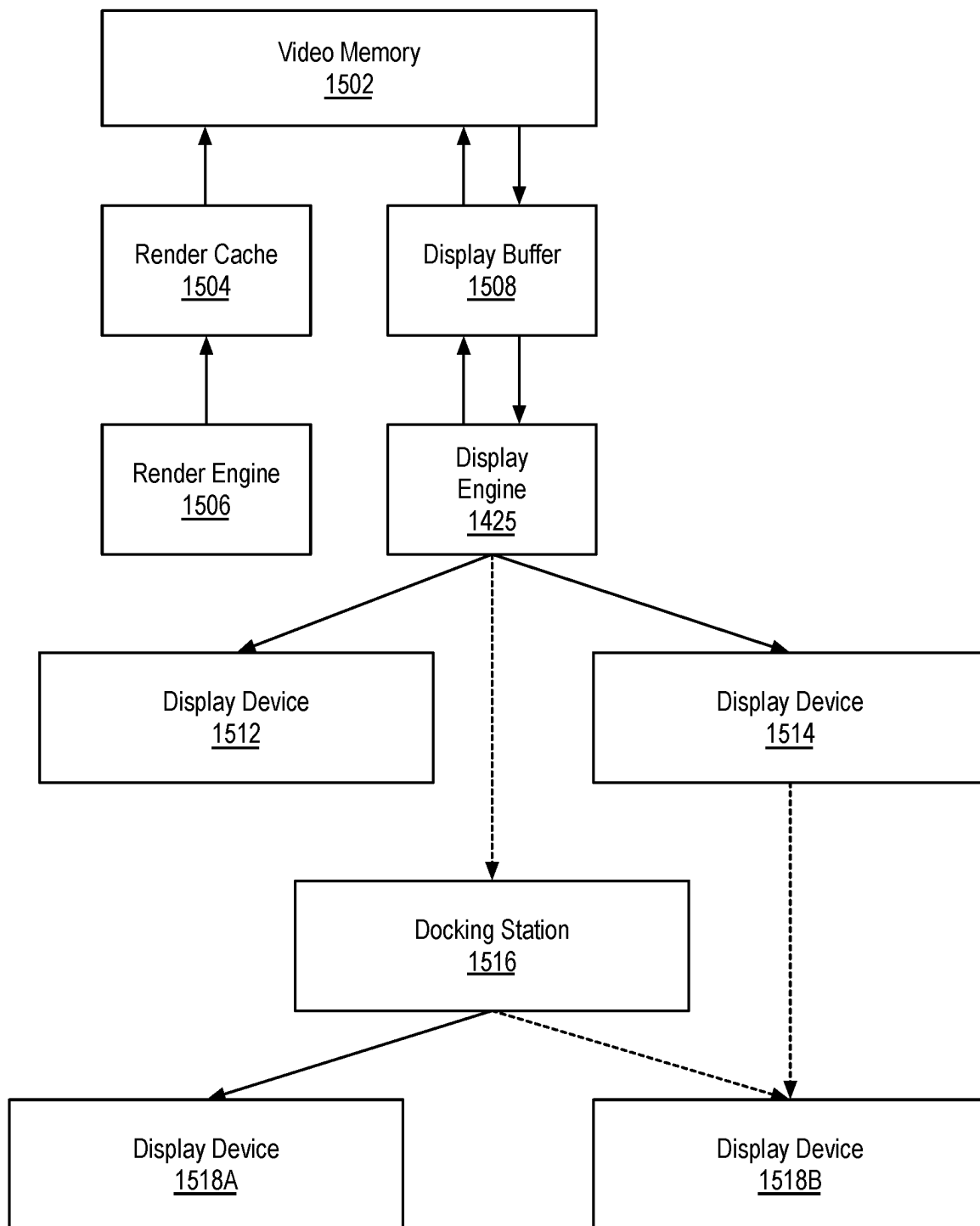


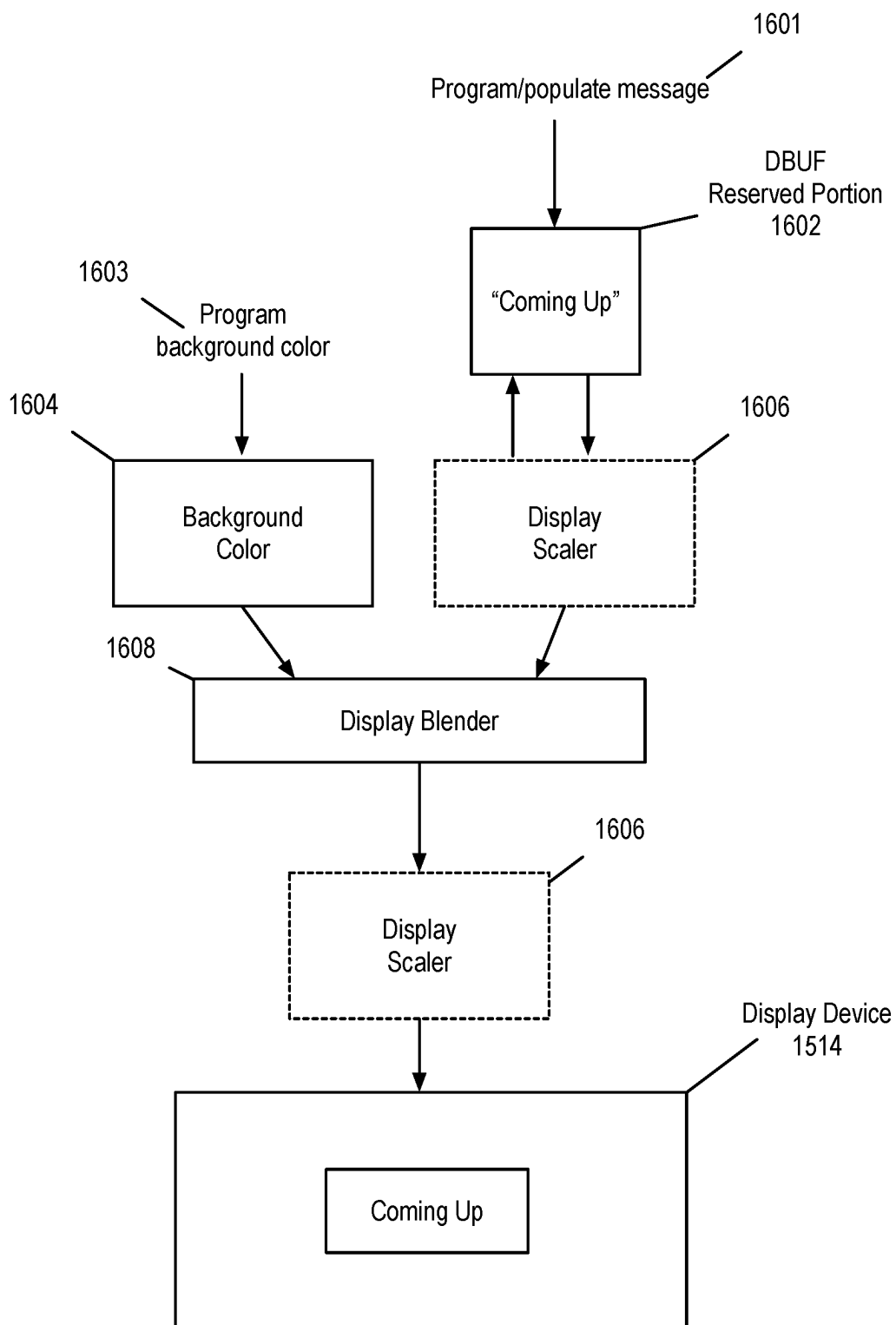
**FIG. 13A**

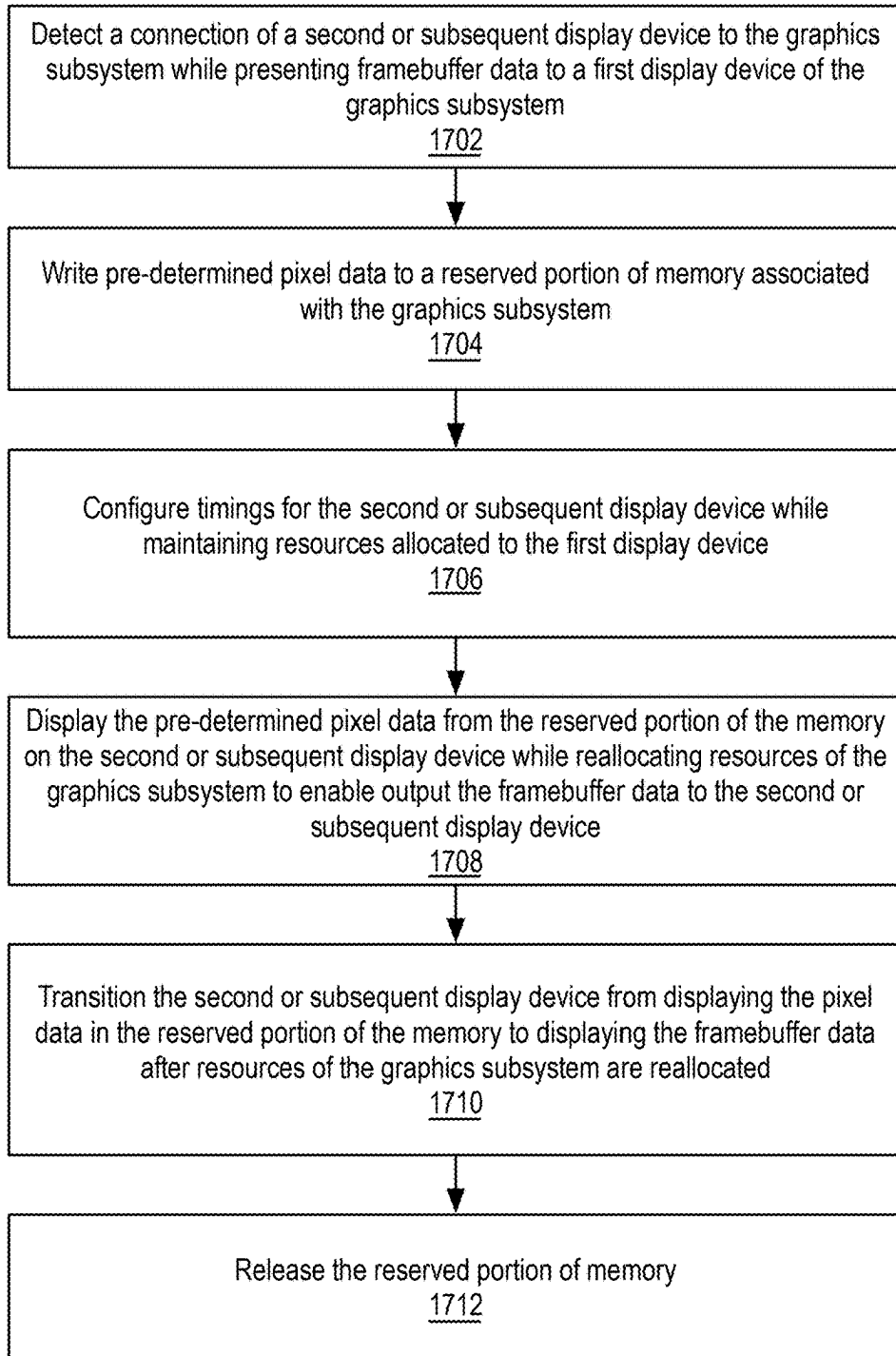


**FIG. 13B**

**FIG. 14**

1500**FIG. 15**

1600**FIG. 16**

1700**FIG. 17**

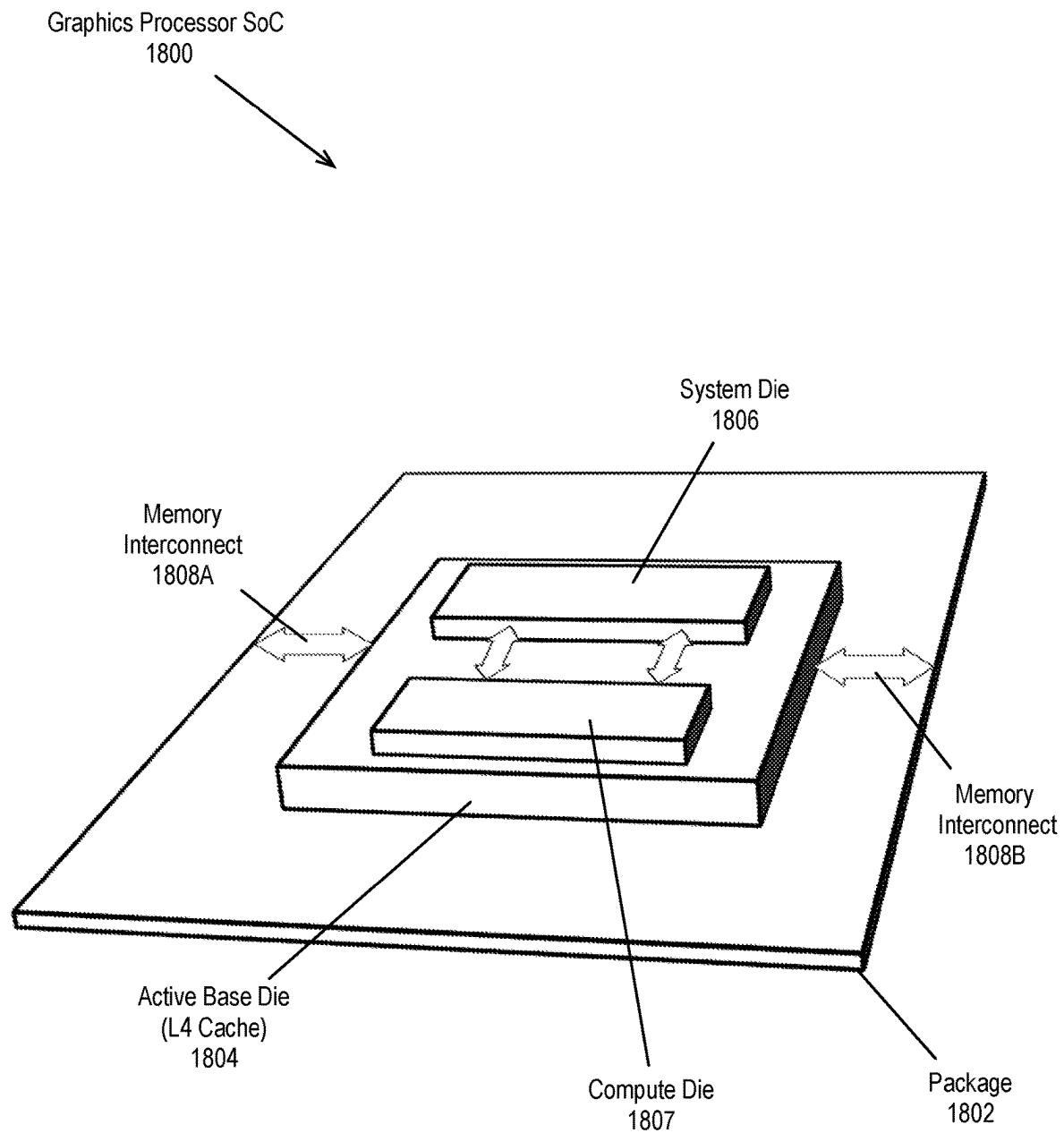


FIG. 18A

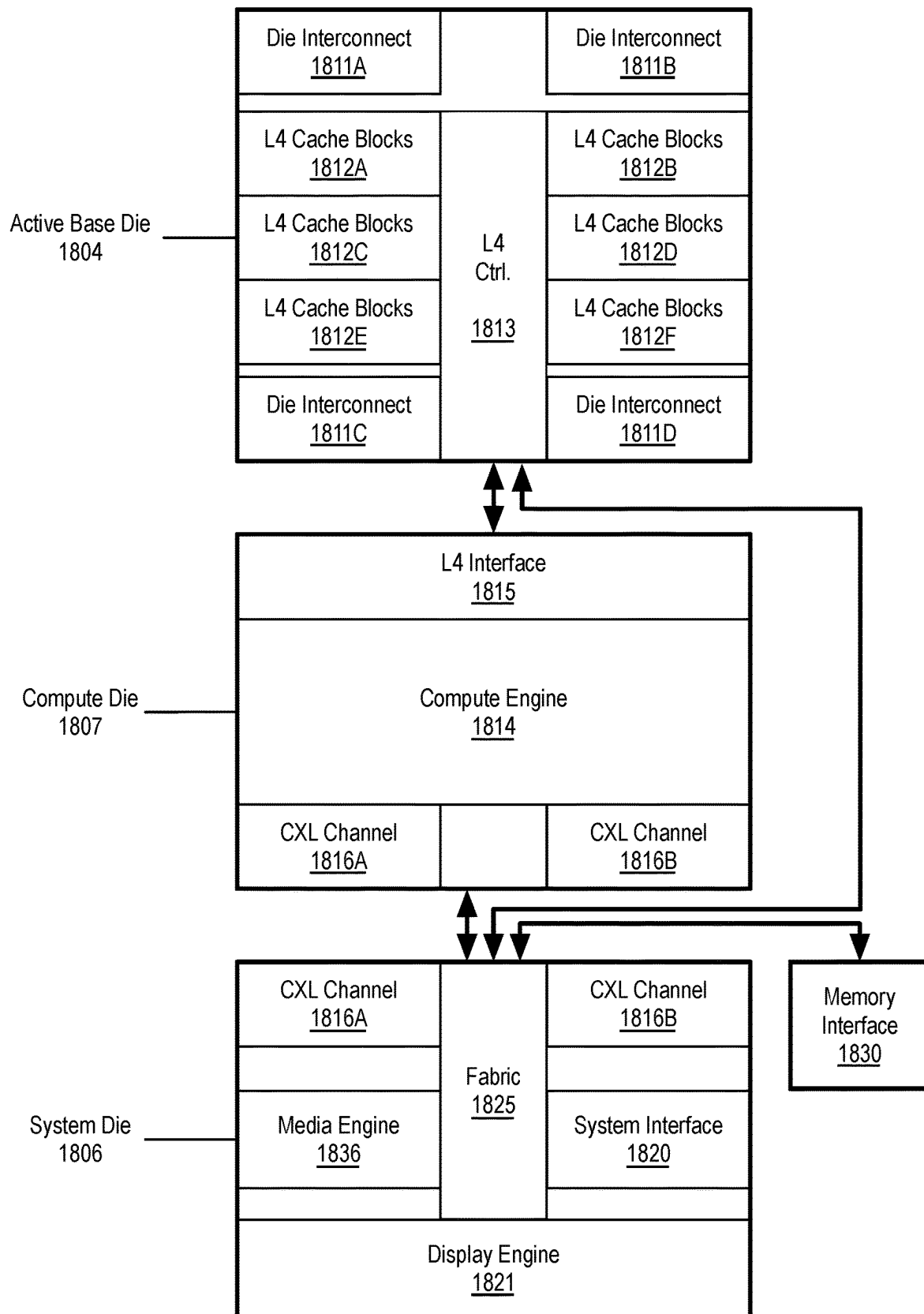
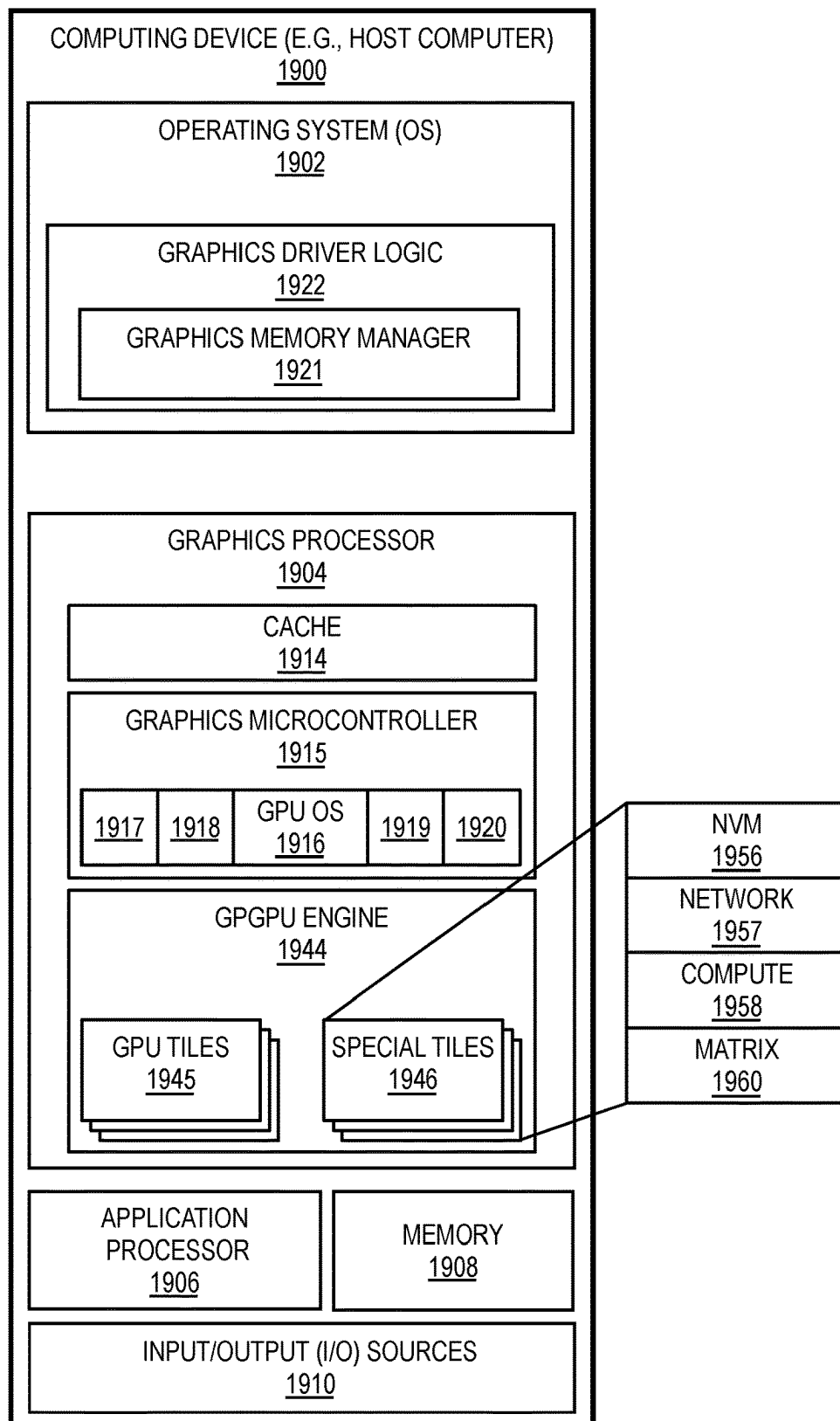


FIG. 18B



**FIG. 19**



1

## AVOIDING FLICKER IN ASSOCIATION WITH HOTPLUG OF A DISPLAY

### FIELD

Embodiments described herein generally relate to graphics devices. More particularly, embodiments are described to avoid display flicker in association with hotplug of a display device to a graphics device.

### BACKGROUND OF THE DISCLOSURE

When a new display device is hotplugged to a computer system, there may not be enough system resources at the time of connection to drive the new display. In such scenarios, graphics subsystem immediately takes away resources from existing connected displays and re-assigns a portion of those resources to the newly attached display. When this occurs, the graphics subsystem may temporarily lack the resources to drive the currently attached displays, which can lead to flashing or flickering on existing monitors.

### BRIEF DESCRIPTION OF THE DRAWINGS

The present invention is illustrated by way of example and not limitation in the figures of the accompanying drawings in which like references indicate similar elements, and in which:

FIG. 1 is a block diagram of a processing system, according to an embodiment;

FIG. 2A-2D illustrate computing systems and graphics processors provided by embodiments described herein;

FIG. 3A-3C illustrate block diagrams of additional graphics processor and compute accelerator architectures provided by embodiments;

FIG. 4 is a block diagram of a graphics processing engine of a graphics processor in accordance with some embodiments;

FIG. 5A-5B illustrate thread execution logic including an array of processing elements employed in a graphics processor core according to embodiments;

FIG. 6 illustrates an additional execution unit, according to an embodiment;

FIG. 7 is a block diagram illustrating a graphics processor instruction formats according to some embodiments;

FIG. 8 is a block diagram of a graphics processor according to another embodiment;

FIG. 9A-9B illustrate a graphics processor command format and command sequence, according to some embodiments;

FIG. 10 illustrates exemplary graphics software architecture for a data processing system according to some embodiments;

FIG. 11A-11D illustrate an integrated circuit package assembly, according to an embodiment;

FIG. 12 is a block diagram illustrating an exemplary system on a chip integrated circuit, according to an embodiment;

FIG. 13A-13B is a block diagram illustrating an additional exemplary graphics processor;

FIG. 14 illustrates a data processing system, according to an embodiment;

FIG. 15 illustrates display engine operation in a graphics subsystem;

FIG. 16 illustrates a display subsystem with updated hotplug handling for display devices;

2

FIG. 17 illustrates a method of enabling and providing a notification to a user for hotplugged displays without disrupting output of currently attached displays;

FIG. 18A-18B illustrate a disaggregated 3D-structured SoC architecture of a graphics processor SoC, according to various embodiments; and

FIG. 19 is a block diagram of a computing device including a graphics processor, according to an embodiment.

### DETAILED DESCRIPTION

A graphics processing unit (GPU) is communicatively coupled to host/processor cores to accelerate, for example, graphics operations, machine-learning operations, pattern analysis operations, and/or various general-purpose GPU (GPGPU) functions. The GPU may be communicatively coupled to the host processor/cores over a bus or another interconnect (e.g., a high-speed interconnect such as Peripheral Component Interconnect Express (PCIe) or NVLink). Alternatively, the GPU may be integrated on the same package or chip as the cores and communicatively coupled to the cores over an internal processor bus/interconnect (i.e., internal to the package or chip). Regardless of the manner in which the GPU is connected, the processor cores may allocate work to the GPU in the form of sequences of commands/instructions contained in a work descriptor. The GPU then uses dedicated circuitry/logic for efficiently processing these commands/instructions.

Current parallel graphics data processing includes systems and methods developed to perform specific operations on graphics data such as, for example, linear interpolation, tessellation, rasterization, texture mapping, depth testing, etc. Traditionally, graphics processors used fixed function computational units to process graphics data. However, more recently, portions of graphics processors have been made programmable, enabling such processors to support a wider variety of operations for processing vertex and fragment data.

As display devices increase in refresh rate and resolution, the graphics subsystem requires an increasingly large amount of resources to drive them. The time required to re-balance resources may increase in relation to the complexity of the resource rebalancing that are performed when a new display device is attached. Existing solutions may be configured to enable newly attached displays as soon as possible, often before the display subsystem has time to rebalance resources between existing and newly attached displays, which may lead to flickering on the current and new displays. Alternatively, if a significant delay occurs before content is displayed on the newly attached display, the user may attempt to unplug and reconnect the display, which exacerbates the problem by placing the graphics subsystem into a nested loop of resource balancing, which further increases display flashing and further deteriorates user experience.

The techniques described herein provide techniques that provide the graphics subsystem time to adjust the resource balance between displays while minimizing the disruption to currently attached displays.

In the following description, numerous specific details are set forth. However, embodiments, as described herein, may be practiced without these specific details. In other instances, well-known circuits, structures and techniques have not been shown in detail in order not to obscure the understanding of this description.

It is contemplated that terms like “request”, “query”, “job”, “work”, “work item”, and “workload” may be refer-

enced interchangeably throughout this document. Similarly, an “application” or “agent” may refer to or include a computer program, a software application, a game, a workstation application, etc., offered through an application programming interface (API), such as a free rendering API, such as Open Graphics Library (OpenGL®), Open Computing Language (OpenCL®), CUDA®, DirectX® 11, DirectX® 12, etc., where “dispatch” may be interchangeably referred to as “work unit” or “draw” and similarly, “application” may be interchangeably referred to as “work-flow” or simply “agent”. For example, a workload, such as that of a three-dimensional (3D) game, may include and issue any number and type of “frames” where each frame may represent an image (e.g., sailboat, human face). Further, each frame may include and offer any number and type of work units, where each work unit may represent a part (e.g., mast of sailboat, forehead of human face) of the image (e.g., sailboat, human face) represented by its corresponding frame. However, for the sake of consistency, each item may be referenced by a single term (e.g., “dispatch”, “agent”, etc.) throughout this document.

In some embodiments, terms like “display screen” and “display surface” may be used interchangeably referring to the visible portion of a display device while the rest of the display device may be embedded into a computing device, such as a smartphone, a wearable device, etc. It is contemplated and to be noted that embodiments are not limited to any particular computing device, software application, hardware component, display device, display screen or surface, protocol, standard, etc. For example, embodiments may be applied to and used with any number and type of real-time applications on any number and type of computers, such as desktops, laptops, tablet computers, smartphones, head-mounted displays, and other wearable devices, and/or the like. Further, for example, rendering scenarios for efficient performance using this novel technique may range from simple scenarios, such as desktop compositing, to complex scenarios, such as 3D games, augmented reality applications, etc.

#### System Overview

FIG. 1 is a block diagram of a processing system 100, according to an embodiment. Processing system 100 may be used in a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processors 102 or processor cores 107. In one embodiment, the processing system 100 is a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices such as within Internet-of-things (IoT) devices with wired or wireless connectivity to a local or wide area network.

In one embodiment, processing system 100 can include, couple with, or be integrated within: a server-based gaming platform; a game console, including a game and media console; a mobile gaming console, a handheld game console, or an online game console. In some embodiments the processing system 100 is part of a mobile phone, smart phone, tablet computing device or mobile Internet-connected device such as a laptop with low internal storage capacity. Processing system 100 can also include, couple with, or be integrated within: a wearable device, such as a smart watch wearable device; smart eyewear or clothing enhanced with augmented reality (AR) or virtual reality (VR) features to provide visual, audio or tactile outputs to supplement real world visual, audio or tactile experiences or otherwise provide text, audio, graphics, video, holographic images or video, or tactile feedback; other augmented reality

(AR) device; or other virtual reality (VR) device. In some embodiments, the processing system 100 includes or is part of a television or set top box device. In one embodiment, processing system 100 can include, couple with, or be integrated within a self-driving vehicle such as a bus, tractor trailer, car, motor or electric power cycle, plane or glider (or any combination thereof). The self-driving vehicle may use processing system 100 to process the environment sensed around the vehicle.

In some embodiments, the one or more processors 102 each include one or more processor cores 107 to process instructions which, when executed, perform operations for system or user software. In some embodiments, at least one of the one or more processor cores 107 is configured to process a specific instruction set 109. In some embodiments, instruction set 109 may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). One or more processor cores 107 may process a different instruction set 109, which may include instructions to facilitate the emulation of other instruction sets. Processor core 107 may also include other processing devices, such as a Digital Signal Processor (DSP).

In some embodiments, the processor 102 includes cache memory 104. Depending on the architecture, the processor 102 can have a single internal cache or multiple levels of internal cache. In some embodiments, the cache memory is shared among various components of the processor 102. In some embodiments, the processor 102 also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor cores 107 using known cache coherency techniques. A register file 106 can be additionally included in processor 102 and may include different types of registers for storing different types of data (e.g., integer registers, floating point registers, status registers, and an instruction pointer register). Some registers may be general-purpose registers, while other registers may be specific to the design of the processor 102.

In some embodiments, one or more processor(s) 102 are coupled with one or more interface bus(es) 110 to transmit communication signals such as address, data, or control signals between processor 102 and other components in the processing system 100. The interface bus 110, in one embodiment, can be a processor bus, such as a version of the Direct Media Interface (DMI) bus. However, processor busses are not limited to the DMI bus, and may include one or more Peripheral Component Interconnect buses (e.g., PCI, PCI express), memory busses, or other types of interface busses. In one embodiment the processor(s) 102 include an integrated memory controller 116 and a platform controller hub 130. The memory controller 116 facilitates communication between a memory device and other components of the processing system 100, while the platform controller hub (PCH) 130 provides connections to I/O devices via a local I/O bus.

The memory device 120 can be a dynamic random-access memory (DRAM) device, a static random-access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. In one embodiment the memory device 120 can operate as system memory for the processing system 100, to store data 122 and instructions 121 for use when the one or more processors 102 executes an application or process. Memory controller 116 also couples with an optional external graphics processor 118, which may communicate with the one or more

graphics processors **108** in processors **102** to perform graphics and media operations. In some embodiments, graphics, media, and/or compute operations may be assisted by an accelerator **112** which is a coprocessor that can be configured to perform a specialized set of graphics, media, or compute operations. For example, in one embodiment the accelerator **112** is a matrix multiplication accelerator used to optimize machine learning or compute operations. In one embodiment the accelerator **112** is a ray-tracing accelerator that can be used to perform ray-tracing operations in concert with the graphics processor **108**. In one embodiment, an external accelerator **119** may be used in place of or in concert with the accelerator **112**.

In some embodiments a display device **111** can connect to the processor(s) **102**. The display device **111** can be one or more of an internal display device, as in a mobile electronic device or a laptop device or an external display device attached via a display interface (e.g., DisplayPort, etc.). In one embodiment the display device **111** can be a head mounted display (HMD) such as a stereoscopic display device for use in virtual reality (VR) applications or augmented reality (AR) applications.

In some embodiments the platform controller hub **130** enables peripherals to connect to memory device **120** and processor **102** via a high-speed I/O bus. The I/O peripherals include, but are not limited to, an audio controller **146**, a network controller **134**, a firmware interface **128**, a wireless transceiver **126**, touch sensors **125**, a data storage device **124** (e.g., non-volatile memory, volatile memory, hard disk drive, flash memory, NAND, 3D NAND, 3D XPoint, etc.). The data storage device **124** can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI express). The touch sensors **125** can include touch screen sensors, pressure sensors, or fingerprint sensors. The wireless transceiver **126** can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, 5G, or Long-Term Evolution (LTE) transceiver. The firmware interface **128** enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). The network controller **134** can enable a network connection to a wired network. In some embodiments, a high-performance network controller (not shown) couples with the interface bus **110**. The audio controller **146**, in one embodiment, is a multi-channel high definition audio controller. In one embodiment the processing system **100** includes an optional legacy I/O controller **140** for coupling legacy (e.g., Personal System 2 (PS/2)) devices to the system. The platform controller hub **130** can also connect to one or more Universal Serial Bus (USB) controllers **142** connect input devices, such as keyboard and mouse **143** combinations, a camera **144**, or other USB input devices.

It will be appreciated that the processing system **100** shown is exemplary and not limiting, as other types of data processing systems that are differently configured may also be used. For example, an instance of the memory controller **116** and platform controller hub **130** may be integrated into a discreet external graphics processor, such as the external graphics processor **118**. In one embodiment the platform controller hub **130** and/or memory controller **116** may be external to the one or more processor(s) **102**. For example, the processing system **100** can include an external memory controller **116** and platform controller hub **130**, which may be configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with the processor(s) **102**.

For example, circuit boards (“sleds”) can be used on which components such as CPUs, memory, and other components are placed are designed for increased thermal performance. In some examples, processing components such as the processors are located on a top side of a sled while near memory, such as DIMMs, are located on a bottom side of the sled. As a result of the enhanced airflow provided by this design, the components may operate at higher frequencies and power levels than in typical systems, thereby increasing performance. Furthermore, the sleds are configured to blindly mate with power and data communication cables in a rack, thereby enhancing their ability to be quickly removed, upgraded, reinstalled, and/or replaced. Similarly, individual components located on the sleds, such as processors, accelerators, memory, and data storage drives, are configured to be easily upgraded due to their increased spacing from each other. In the illustrative embodiment, the components additionally include hardware attestation features to prove their authenticity.

A data center can utilize a single network architecture (“fabric”) that supports multiple other network architectures including Ethernet and Omni-Path. The sleds can be coupled to switches via optical fibers, which provide higher bandwidth and lower latency than typical twisted pair cabling (e.g., Category 5, Category 5e, Category 6, etc.). Due to the high bandwidth, low latency interconnections and network architecture, the data center may, in use, pool resources, such as memory, accelerators (e.g., GPUs, graphics accelerators, FPGAs, ASICs, neural network and/or artificial intelligence accelerators, etc.), and data storage drives that are physically disaggregated, and provide them to compute resources (e.g., processors) on an as needed basis, enabling the compute resources to access the pooled resources as if they were local.

A power supply or source can provide voltage and/or current to processing system **100** or any component or system described herein. In one example, the power supply includes an AC to DC (alternating current to direct current) adapter to plug into a wall outlet. Such AC power can be renewable energy (e.g., solar power) power source. In one example, power source includes a DC power source, such as an external AC to DC converter. In one example, power source or power supply includes wireless charging hardware to charge via proximity to a charging field. In one example, power source can include an internal battery, alternating current supply, motion-based power supply, solar power supply, or fuel cell source.

FIG. 2A-2D illustrate computing systems and graphics processors provided by embodiments described herein. The elements of FIGS. 2A-2D having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such.

FIG. 2A is a block diagram of an embodiment of a processor **200** having one or more processor cores **202A-202N**, an integrated memory controller **214**, and an integrated graphics processor **208**. Processor **200** can include additional cores up to and including additional core **202N** represented by the dashed lined boxes. Each of processor cores **202A-202N** includes one or more internal cache units **204A-204N**. In some embodiments each processor core also has access to one or more shared cached units **206**. The internal cache units **204A-204N** and shared cache units **206** represent a cache memory hierarchy within the processor **200**. The cache memory hierarchy may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such

as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where the highest level of cache before external memory is classified as the LLC. In some embodiments, cache coherency logic maintains coherency between the various cache units **206** and **204A-204N**.

In some embodiments, processor **200** may also include a set of one or more bus controller units **216** and a system agent core **210**. The one or more bus controller units **216** manage a set of peripheral buses, such as one or more PCI or PCI express busses. System agent core **210** provides management functionality for the various processor components. In some embodiments, system agent core **210** includes one or more integrated memory controllers **214** to manage access to various external memory devices (not shown).

In some embodiments, one or more of the processor cores **202A-202N** include support for simultaneous multi-threading. In such embodiment, the system agent core **210** includes components for coordinating and operating cores **202A-202N** during multi-threaded processing. System agent core **210** may additionally include a power control unit (PCU), which includes logic and components to regulate the power state of processor cores **202A-202N** and graphics processor **208**.

In some embodiments, processor **200** additionally includes graphics processor **208** to execute graphics processing operations. In some embodiments, the graphics processor **208** couples with the set of shared cache units **206**, and the system agent core **210**, including the one or more integrated memory controllers **214**. In some embodiments, the system agent core **210** also includes a display controller **211** to drive graphics processor output to one or more coupled displays. In some embodiments, display controller **211** may also be a separate module coupled with the graphics processor via at least one interconnect, or may be integrated within the graphics processor **208**.

In some embodiments, a ring-based interconnect **212** is used to couple the internal components of the processor **200**. However, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques, including techniques well known in the art. In some embodiments, graphics processor **208** couples with the ring-based interconnect **212** via an I/O link **213**.

The exemplary I/O link **213** represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance embedded memory module **218**, such as an eDRAM module. In some embodiments, each of the processor cores **202A-202N** and graphics processor **208** can use embedded memory modules **218** as a shared Last Level Cache.

In some embodiments, processor cores **202A-202N** are homogenous cores executing the same instruction set architecture. In another embodiment, processor cores **202A-202N** are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor cores **202A-202N** execute a first instruction set, while at least one of the other cores executes a subset of the first instruction set or a different instruction set. In one embodiment, processor cores **202A-202N** are heterogeneous in terms of microarchitecture, where one or more cores having a relatively higher power consumption couple with one or more power cores having a lower power consumption. In one embodiment, processor cores **202A-202N** are heterogeneous in terms of computational capability. Additionally, processor **200** can be

implemented on one or more chips or as an SoC integrated circuit having the illustrated components, in addition to other components.

FIG. 2B is a block diagram of hardware logic of a graphics processor core **219**, according to some embodiments described herein. Elements of FIG. 2B having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such. The graphics processor core **219**, sometimes referred to as a core slice, can be one or multiple graphics cores within a modular graphics processor. The graphics processor core **219** is exemplary of one graphics core slice, and a graphics processor as described herein may include multiple graphics core slices based on target power and performance envelopes. Each graphics processor core **219** can include a fixed function block **230** coupled with multiple sub-cores **221A-221F**, also referred to as sub-slices, that include modular blocks of general-purpose and fixed function logic.

In some embodiments, the fixed function block **230** includes a geometry/fixed function pipeline **231** that can be shared by all sub-cores in the graphics processor core **219**, for example, in lower performance and/or lower power graphics processor implementations. In various embodiments, the geometry/fixed function pipeline **231** includes a 3D fixed function pipeline (e.g., 3D pipeline **312** as in FIG. 3A and FIG. 4, described below) a video front-end unit, a thread spawner and thread dispatcher, and a unified return buffer manager, which manages unified return buffers (e.g., unified return buffer (URB **418**) in FIG. 4, as described below).

In one embodiment the fixed function block **230** also includes a graphics SoC interface **232**, a graphics microcontroller **233**, and a media pipeline **234**. The graphics SoC interface **232** provides an interface between the graphics processor core **219** and other processor cores within a system on a chip integrated circuit. The graphics microcontroller **233** is a programmable sub-processor that is configurable to manage various functions of the graphics processor core **219**, including thread dispatch, scheduling, and pre-emption. The media pipeline **234** (e.g., media pipeline **316** of FIG. 3A and FIG. 4) includes logic to facilitate the decoding, encoding, pre-processing, and/or post-processing of multimedia data, including image and video data. The media pipeline **234** implement media operations via requests to compute or sampling logic within the sub-cores **221-221F**.

In one embodiment the SoC interface **232** enables the graphics processor core **219** to communicate with general-purpose application processor cores (e.g., CPUs) and/or other components within an SoC, including memory hierarchy elements such as a shared last level cache memory, the system RAM, and/or embedded on-chip or on-package DRAM. The SoC interface **232** can also enable communication with fixed function devices within the SoC, such as camera imaging pipelines, and enables the use of and/or implements global memory atomics that may be shared between the graphics processor core **219** and CPUs within the SoC. The SoC interface **232** can also implement power management controls for the graphics processor core **219** and enable an interface between a clock domain of the graphics processor core **219** and other clock domains within the SoC. In one embodiment the SoC interface **232** enables receipt of command buffers from a command streamer and global thread dispatcher that are configured to provide commands and instructions to each of one or more graphics cores within a graphics processor. The commands and

instructions can be dispatched to the media pipeline **234**, when media operations are to be performed, or a geometry and fixed function pipeline (e.g., geometry and fixed function pipeline **231**, geometry and fixed function pipeline **237**) when graphics processing operations are to be performed.

The graphics microcontroller **233** can be configured to perform various scheduling and management tasks for the graphics processor core **219**. In one embodiment the graphics microcontroller **233** can perform graphics and/or compute workload scheduling on the various graphics parallel engines within execution unit (EU) arrays **222A-222F**, **224A-224F** within the sub-cores **221A-221F**. In this scheduling model, host software executing on a CPU core of an SoC including the graphics processor core **219** can submit workloads one of multiple graphic processor doorbells, which invokes a scheduling operation on the appropriate graphics engine. Scheduling operations include determining which workload to run next, submitting a workload to a command streamer, pre-empting existing workloads running on an engine, monitoring progress of a workload, and notifying host software when a workload is complete. In one embodiment the graphics microcontroller **233** can also facilitate low-power or idle states for the graphics processor core **219**, providing the graphics processor core **219** with the ability to save and restore registers within the graphics processor core **219** across low-power state transitions independently from the operating system and/or graphics driver software on the system.

The graphics processor core **219** may have greater than or fewer than the illustrated sub-cores **221A-221F**, up to N modular sub-cores. For each set of N sub-cores, the graphics processor core **219** can also include shared function logic **235**, shared and/or cache memory **236**, a geometry/fixed function pipeline **237**, as well as additional fixed function logic **238** to accelerate various graphics and compute processing operations. The shared function logic **235** can include logic units associated with the shared function logic **420** of FIG. 4 (e.g., sampler, math, and/or inter-thread communication logic) that can be shared by each N sub-cores within the graphics processor core **219**. The shared and/or cache memory **236** can be a last-level cache for the set of N sub-cores **221A-221F** within the graphics processor core **219**, and can also serve as shared memory that is accessible by multiple sub-cores. The geometry/fixed function pipeline **237** can be included instead of the geometry/fixed function pipeline **231** within the fixed function block **230** and can include the same or similar logic units.

In one embodiment the graphics processor core **219** includes additional fixed function logic **238** that can include various fixed function acceleration logic for use by the graphics processor core **219**. In one embodiment the additional fixed function logic **238** includes an additional geometry pipeline for use in position only shading. In position-only shading, two geometry pipelines exist, the full geometry pipeline within the geometry/fixed function pipeline **231**, and a cull pipeline, which is an additional geometry pipeline which may be included within the additional fixed function logic **238**. In one embodiment the cull pipeline is a trimmed down version of the full geometry pipeline. The full pipeline and the cull pipeline can execute different instances of the same application, each instance having a separate context. Position only shading can hide long cull runs of discarded triangles, enabling shading to be completed earlier in some instances. For example and in one embodiment the cull pipeline logic within the additional fixed function logic **238** can execute position shaders in parallel with the main application and generally generates critical results faster

than the full pipeline, as the cull pipeline fetches and shades only the position attribute of the vertices, without performing rasterization and rendering of the pixels to the frame buffer. The cull pipeline can use the generated critical results to compute visibility information for all the triangles without regard to whether those triangles are culled. The full pipeline (which in this instance may be referred to as a replay pipeline) can consume the visibility information to skip the culled triangles to shade only the visible triangles that are finally passed to the rasterization phase.

In one embodiment the additional fixed function logic **238** can also include machine-learning acceleration logic, such as fixed function matrix multiplication logic, for implementations including optimizations for machine learning training or inferencing.

Within each graphics sub-core **221A-221F** includes a set of execution resources that may be used to perform graphics, media, and compute operations in response to requests by graphics pipeline, media pipeline, or shader programs. The graphics sub-cores **221A-221F** include multiple EU arrays **222A-222F**, **224A-224F**, thread dispatch and inter-thread communication (TD/IC) logic **223A-223F**, a 3D (e.g., texture) sampler **225A-225F**, a media sampler **226A-226F**, a shader processor **227A-227F**, and shared local memory (SLM) **228A-228F**. The EU arrays **222A-222F**, **224A-224F** each include multiple execution units, which are general-purpose graphics processing units capable of performing floating-point and integer/fixed-point logic operations in service of a graphics, media, or compute operation, including graphics, media, or compute shader/GPGPU programs. The TD/IC logic **223A-223F** performs local thread dispatch and thread control operations for the execution units within a sub-core and facilitate communication between threads executing on the execution units of the sub-core. The 3D sampler **225A-225F** can read texture or other 3D graphics related data into memory. The 3D sampler can read texture data differently based on a configured sample state and the texture format associated with a given texture. The media sampler **226A-226F** can perform similar read operations based on the type and format associated with media data. In one embodiment, each graphics sub-core **221A-221F** can alternately include a unified 3D and media sampler. Threads executing on the execution units within each of the sub-cores **221A-221F** can make use of shared local memory **228A-228F** within each sub-core, to enable threads executing within a thread group to execute using a common pool of on-chip memory.

FIG. 2C illustrates a graphics processing unit (GPU) **239** that includes dedicated sets of graphics processing resources arranged into multi-core groups **240A-240N**. The details of multi-core group **240A** are illustrated. Multi-core groups **240B-240N** may be equipped with the same or similar sets of graphics processing resources.

As illustrated, a multi-core group **240A** may include a set of graphics cores **243**, a set of tensor cores **244**, and a set of ray tracing cores **245**. A scheduler/dispatcher **241** schedules and dispatches the graphics threads for execution on the various cores **243**, **244**, **245**. In one embodiment the tensor cores **244** are sparse tensor cores with hardware to enable multiplication operations having a zero value input to be bypassed.

A set of register files **242** can store operand values used by the cores **243**, **244**, **245** when executing the graphics threads. These may include, for example, integer registers for storing integer values, floating point registers for storing floating point values, vector registers for storing packed data elements (integer and/or floating point data elements) and

tile registers for storing tensor/matrix values. In one embodiment, the tile registers are implemented as combined sets of vector registers.

One or more combined level 1 (L1) caches and shared memory units **247** store graphics data such as texture data, vertex data, pixel data, ray data, bounding volume data, etc., locally within each multi-core group **240A**. One or more texture units **247** can also be used to perform texturing operations, such as texture mapping and sampling. A Level 2 (L2) cache **253** shared by all or a subset of the multi-core groups **240A-240N** stores graphics data and/or instructions for multiple concurrent graphics threads. As illustrated, the L2 cache **253** may be shared across a plurality of multi-core groups **240A-240N**. One or more memory controllers **248** couple the GPU **239** to a memory **249** which may be a system memory (e.g., DRAM) and/or a dedicated graphics memory (e.g., GDDR6 memory).

Input/output (I/O) circuitry **250** couples the GPU **239** to one or more I/O devices **252** such as digital signal processors (DSPs), network controllers, or user input devices. An on-chip interconnect may be used to couple the I/O devices **252** to the GPU **239** and memory **249**. One or more I/O memory management units (IOMMUs) **251** of the I/O circuitry **250** couple the I/O devices **252** directly to the memory **249**. In one embodiment, the IOMMU **251** manages multiple sets of page tables to map virtual addresses to physical addresses in memory **249**. In this embodiment, the I/O devices **252**, CPU(s) **246**, and GPU **239** may share the same virtual address space.

In one implementation, the IOMMU **251** supports virtualization. In this case, it may manage a first set of page tables to map guest/graphics virtual addresses to guest/graphics physical addresses and a second set of page tables to map the guest/graphics physical addresses to system/host physical addresses (e.g., within memory **249**). The base addresses of each of the first and second sets of page tables may be stored in control registers and swapped out on a context switch (e.g., so that the new context is provided with access to the relevant set of page tables). While not illustrated in FIG. 2C, each of the cores **243**, **244**, **245** and/or multi-core groups **240A-240N** may include translation lookaside buffers (TLBs) to cache guest virtual to guest physical translations, guest physical to host physical translations, and guest virtual to host physical translations.

In one embodiment, the CPU(s) **246**, GPU **239**, and I/O devices **252** are integrated on a single semiconductor chip and/or chip package. The memory **249** may be integrated on the same chip or may be coupled to the memory controllers **248** via an off-chip interface. In one implementation, the memory **249** comprises GDDR6 memory which shares the same virtual address space as other physical system-level memories, although the underlying principles of the invention are not limited to this specific implementation.

In one embodiment, the tensor cores **244** include a plurality of execution units specifically designed to perform matrix operations, which are the fundamental compute operation used to perform deep learning operations. For example, simultaneous matrix multiplication operations may be used for neural network training and inferencing. The tensor cores **244** may perform matrix processing using a variety of operand precisions including single precision floating-point (e.g., 32 bits), half-precision floating point (e.g., 16 bits), integer words (16 bits), bytes (8 bits), and half-bytes (4 bits). In one embodiment, a neural network implementation extracts features of each rendered scene, potentially combining details from multiple frames, to construct a high-quality final image.

In deep learning implementations, parallel matrix multiplication work may be scheduled for execution on the tensor cores **244**. The training of neural networks, in particular, requires a significant number of matrix dot product operations. In order to process an inner-product formulation of an  $N \times N \times N$  matrix multiply, the tensor cores **244** may include at least  $N$  dot-product processing elements. Before the matrix multiply begins, one entire matrix is loaded into tile registers and at least one column of a second matrix is loaded each cycle for  $N$  cycles. Each cycle, there are  $N$  dot products that are processed.

Matrix elements may be stored at different precisions depending on the particular implementation, including 16-bit words, 8-bit bytes (e.g., INT8) and 4-bit half-bytes (e.g., INT4). Different precision modes may be specified for the tensor cores **244** to ensure that the most efficient precision is used for different workloads (e.g., such as inferencing workloads which can tolerate quantization to bytes and half-bytes).

In one embodiment, the ray tracing cores **245** accelerate ray tracing operations for both real-time ray tracing and non-real-time ray tracing implementations. In particular, the ray tracing cores **245** include ray traversal/intersection circuitry for performing ray traversal using bounding volume hierarchies (BVHs) and identifying intersections between rays and primitives enclosed within the BVH volumes. The ray tracing cores **245** may also include circuitry for performing depth testing and culling (e.g., using a Z buffer or similar arrangement). In one implementation, the ray tracing cores **245** perform traversal and intersection operations in concert with the image denoising techniques described herein, at least a portion of which may be executed on the tensor cores **244**. For example, in one embodiment, the tensor cores **244** implement a deep learning neural network to perform denoising of frames generated by the ray tracing cores **245**. However, the CPU(s) **246**, graphics cores **243**, and/or ray tracing cores **245** may also implement all or a portion of the denoising and/or deep learning algorithms.

In addition, as described above, a distributed approach to denoising may be employed in which the GPU **239** is in a computing device coupled to other computing devices over a network or high speed interconnect. In this embodiment, the interconnected computing devices share neural network learning/training data to improve the speed with which the overall system learns to perform denoising for different types of image frames and/or different graphics applications.

In one embodiment, the ray tracing cores **245** process all BVH traversal and ray-primitive intersections, saving the graphics cores **243** from being overloaded with thousands of instructions per ray. In one embodiment, each ray tracing core **245** includes a first set of specialized circuitry for performing bounding box tests (e.g., for traversal operations) and a second set of specialized circuitry for performing the ray-triangle intersection tests (e.g., intersecting rays which have been traversed). Thus, in one embodiment, the multi-core group **240A** can simply launch a ray probe, and the ray tracing cores **245** independently perform ray traversal and intersection and return hit data (e.g., a hit, no hit, multiple hits, etc.) to the thread context. The other cores **243**, **244** are freed to perform other graphics or compute work while the ray tracing cores **245** perform the traversal and intersection operations.

In one embodiment, each ray tracing core **245** includes a traversal unit to perform BVH testing operations and an intersection unit which performs ray-primitive intersection tests. The intersection unit generates a "hit", "no hit", or "multiple hit" response, which it provides to the appropriate

thread. During the traversal and intersection operations, the execution resources of the other cores (e.g., graphics cores **243** and tensor cores **244**) are freed to perform other forms of graphics work.

In one particular embodiment described below, a hybrid rasterization/ray tracing approach is used in which work is distributed between the graphics cores **243** and ray tracing cores **245**.

In one embodiment, the ray tracing cores **245** (and/or other cores **243**, **244**) include hardware support for a ray tracing instruction set such as Microsoft's DirectX Ray Tracing (DXR) which includes a DispatchRays command, as well as ray-generation, closest-hit, any-hit, and miss shaders, which enable the assignment of unique sets of shaders and textures for each object. Another ray tracing platform which may be supported by the ray tracing cores **245**, graphics cores **243** and tensor cores **244** is Vulkan 1.1.85. Note, however, that the underlying principles of the invention are not limited to any particular ray tracing ISA.

In general, the various cores **245**, **244**, **243** may support a ray tracing instruction set that includes instructions/functions for ray generation, closest hit, any hit, ray-primitive intersection, per-primitive and hierarchical bounding box construction, miss, visit, and exceptions. More specifically, one embodiment includes ray tracing instructions to perform the following functions:

**Ray Generation**—Ray generation instructions may be executed for each pixel, sample, or other user-defined work assignment.

**Closest Hit**—A closest hit instruction may be executed to locate the closest intersection point of a ray with primitives within a scene.

**Any Hit**—An any hit instruction identifies multiple intersections between a ray and primitives within a scene, potentially to identify a new closest intersection point.

**Intersection**—An intersection instruction performs a ray-primitive intersection test and outputs a result.

**Per-primitive Bounding box Construction**—This instruction builds a bounding box around a given primitive or group of primitives (e.g., when building a new BVH or other acceleration data structure).

**Miss**—Indicates that a ray misses all geometry within a scene, or specified region of a scene.

**Visit**—Indicates the children volumes a ray will traverse.

**Exceptions**—Includes various types of exception handlers (e.g., invoked for various error conditions).

In one embodiment the ray tracing cores **245** may be adapted to accelerate general-purpose compute operations that can be accelerated using computational techniques that are analogous to ray intersection tests. A compute framework can be provided that enables shader programs to be compiled into low level instructions and/or primitives that perform general-purpose compute operations via the ray tracing cores. Exemplary computational problems that can benefit from compute operations performed on the ray tracing cores **245** include computations involving beam, wave, ray, or particle propagation within a coordinate space. Interactions associated with that propagation can be computed relative to a geometry or mesh within the coordinate space. For example, computations associated with electromagnetic signal propagation through an environment can be accelerated via the use of instructions or primitives that are executed via the ray tracing cores. Diffraction and reflection of the signals by objects in the environment can be computed as direct ray-tracing analogies.

Ray tracing cores **245** can also be used to perform computations that are not directly analogous to ray tracing.

For example, mesh projection, mesh refinement, and volume sampling computations can be accelerated using the ray tracing cores **245**. Generic coordinate space calculations, such as nearest neighbor calculations can also be performed.

For example, the set of points near a given point can be discovered by defining a bounding box in the coordinate space around the point. BVH and ray probe logic within the ray tracing cores **245** can then be used to determine the set of point intersections within the bounding box. The intersections constitute the origin point and the nearest neighbors to that origin point. Computations that are performed using the ray tracing cores **245** can be performed in parallel with computations performed on the graphics cores **243** and tensor cores **244**. A shader compiler can be configured to compile a compute shader or other general-purpose graphics processing program into low level primitives that can be parallelized across the graphics cores **243**, tensor cores **244**, and ray tracing cores **245**.

FIG. 2D is a block diagram of general purpose graphics processing unit (GPGPU) **270** that can be configured as a graphics processor and/or compute accelerator, according to embodiments described herein. The GPGPU **270** can interconnect with host processors (e.g., one or more CPU(s) **246**) and memory **271**, **272** via one or more system and/or memory busses. In one embodiment the memory **271** is system memory that may be shared with the one or more CPU(s) **246**, while memory **272** is device memory that is dedicated to the GPGPU **270**. In one embodiment, components within the GPGPU **270** and memory **272** may be mapped into memory addresses that are accessible to the one or more CPU(s) **246**. Access to memory **271** and **272** may be facilitated via a memory controller **268**. In one embodiment the memory controller **268** includes an internal direct memory access (DMA) controller **269** or can include logic to perform operations that would otherwise be performed by a DMA controller.

The GPGPU **270** includes multiple cache memories, including an L2 cache **253**, L1 cache **254**, an instruction cache **255**, and shared memory **256**, at least a portion of which may also be partitioned as a cache memory. The GPGPU **270** also includes multiple compute units **260A-260N**. Each compute unit **260A-260N** includes a set of vector registers **261**, scalar registers **262**, vector logic units **263**, and scalar logic units **264**. The compute units **260A-260N** can also include local shared memory **265** and a program counter **266**. The compute units **260A-260N** can couple with a constant cache **267**, which can be used to store constant data, which is data that will not change during the run of kernel or shader program that executes on the GPGPU **270**. In one embodiment the constant cache **267** is a scalar data cache and cached data can be fetched directly into the scalar registers **262**.

During operation, the one or more CPU(s) **246** can write commands into registers or memory in the GPGPU **270** that has been mapped into an accessible address space. The command processors **257** can read the commands from registers or memory and determine how those commands will be processed within the GPGPU **270**. A thread dispatcher **258** can then be used to dispatch threads to the compute units **260A-260N** to perform those commands. Each compute unit **260A-260N** can execute threads independently of the other compute units. Additionally each compute unit **260A-260N** can be independently configured for conditional computation and can conditionally output the results of computation to memory. The command processors **257** can interrupt the one or more CPU(s) **246** when the submitted commands are complete.

15

FIGS. 3A-3C illustrate block diagrams of additional graphics processor and compute accelerator architectures provided by embodiments described herein. The elements of FIGS. 3A-3C having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such.

FIG. 3A is a block diagram of a graphics processor 300, which may be a discrete graphics processing unit, or may be a graphics processor integrated with a plurality of processing cores, or other semiconductor devices such as, but not limited to, memory devices or network interfaces. In some embodiments, the graphics processor communicates via a memory mapped I/O interface to registers on the graphics processor and with commands placed into the processor memory. In some embodiments, graphics processor 300 includes a memory interface 314 to access memory. Memory interface 314 can be an interface to local memory, one or more internal caches, one or more shared external caches, and/or to system memory.

In some embodiments, graphics processor 300 also includes a display controller 302 to drive display output data to a display device 318. Display controller 302 includes hardware for one or more overlay planes for the display and composition of multiple layers of video or user interface elements. The display device 318 can be an internal or external display device. In one embodiment the display device 318 is a head mounted display device, such as a virtual reality (VR) display device or an augmented reality (AR) display device. In some embodiments, graphics processor 300 includes a video codec engine 306 to encode, decode, or transcode media to, from, or between one or more media encoding formats, including, but not limited to Moving Picture Experts Group (MPEG) formats such as MPEG-2, Advanced Video Coding (AVC) formats such as H.264/MPEG-4 AVC, H.265/HEVC, Alliance for Open Media (AOMedia) VP8, VP9, as well as the Society of Motion Picture & Television Engineers (SMPTE) 421M/VC-1, and Joint Photographic Experts Group (JPEG) formats such as JPEG, and Motion JPEG (MJPEG) formats.

In some embodiments, graphics processor 300 includes a block image transfer (BLIT) engine 304 to perform two-dimensional (2D) rasterizer operations including, for example, bit-boundary block transfers. However, in one embodiment, 2D graphics operations are performed using one or more components of graphics processing engine (GPE 310). In some embodiments, GPE 310 is a compute engine for performing graphics operations, including three-dimensional (3D) graphics operations and media operations.

In some embodiments, GPE 310 includes a 3D pipeline 312 for performing 3D operations, such as rendering three-dimensional images and scenes using processing functions that act upon 3D primitive shapes (e.g., rectangle, triangle, etc.). The 3D pipeline 312 includes programmable and fixed function elements that perform various tasks within the element and/or spawn execution threads to a 3D/Media subsystem 315. While 3D pipeline 312 can be used to perform media operations, an embodiment of GPE 310 also includes a media pipeline 316 that is specifically used to perform media operations, such as video post-processing and image enhancement.

In some embodiments, media pipeline 316 includes fixed function or programmable logic units to perform one or more specialized media operations, such as video decode acceleration, video de-interlacing, and video encode acceleration in place of, or on behalf of video codec engine 306. In some embodiments, media pipeline 316 additionally

16

includes a thread spawning unit to spawn threads for execution on 3D/Media subsystem 315. The spawned threads perform computations for the media operations on one or more graphics execution units included in 3D/Media subsystem 315.

In some embodiments, 3D/Media subsystem 315 includes logic for executing threads spawned by 3D pipeline 312 and media pipeline 316. In one embodiment, the pipelines send thread execution requests to 3D/Media subsystem 315, which includes thread dispatch logic for arbitrating and dispatching the various requests to available thread execution resources. The execution resources include an array of graphics execution units to process the 3D and media threads. In some embodiments, 3D/Media subsystem 315 includes one or more internal caches for thread instructions and data. In some embodiments, the subsystem also includes shared memory, including registers and addressable memory, to share data between threads and to store output data.

FIG. 3B illustrates a graphics processor 320 having a tiled architecture, according to embodiments described herein. In one embodiment the graphics processor 320 includes a graphics processing engine cluster 322 having multiple instances of the GPE 310 of FIG. 3A within a graphics engine tile 310A-310D. Each graphics engine tile 310A-310D can be interconnected via a set of tile interconnects 323A-323F. Each graphics engine tile 310A-310D can also be connected to a memory module or memory device 326A-326D via memory interconnects 325A-325D. The memory devices 326A-326D can use any graphics memory technology. For example, the memory devices 326A-326D may be graphics double data rate (GDDR) memory. The memory devices 326A-326D, in one embodiment, are high-bandwidth memory (HBM) modules that can be on-die with their respective graphics engine tile 310A-310D. In one embodiment the memory devices 326A-326D are stacked memory devices that can be stacked on top of their respective graphics engine tile 310A-310D. In one embodiment, each graphics engine tile 310A-310D and associated memory 326A-326D reside on separate chiplets, which are bonded to a base die or base substrate, as described on further detail in FIGS. 11B-11D.

The graphics processor 320 may be configured with a non-uniform memory access (NUMA) system in which memory devices 326A-326D are coupled with associated graphics engine tiles 310A-310D. A given memory device may be accessed by graphics engine tiles other than the tile to which it is directly connected. However, access latency to the memory devices 326A-326D may be lowest when accessing a local tile. In one embodiment, a cache coherent NUMA (ccNUMA) system is enabled that uses the tile interconnects 323A-323F to enable communication between cache controllers within the graphics engine tiles 310A-310D to maintain a consistent memory image when more than one cache stores the same memory location.

The graphics processing engine cluster 322 can connect with a fabric interconnect 324, which may be an on-chip or on-package interconnect. In one embodiment the fabric interconnect 324 includes a network processor, network on a chip (NoC), or another switching processor to enable the fabric interconnect 324 to act as a packet switched fabric interconnect that switches data packets between components of the graphics processor 320. The fabric interconnect 324 can enable communication between graphics engine tiles 310A-310D and components such as the video codec engine 306 and one or more copy engines 304. The copy engines 304 can be used to move data out of, into, and between the



memory devices **326A-326D** and memory that is external to the graphics processor **320** (e.g., system memory). The fabric interconnect **324** can also couple with one or more of the tile interconnects **323A-323F** to facilitate or enhance the interconnection between the graphics engine tiles **310A-310D**. The fabric interconnect **324** is also configurable to interconnect multiple instances of the graphics processor **320** (e.g., via the host interface **328**), enabling tile-to-tile communication between graphics engine tiles **310A-310D** of multiple GPUs. In one embodiment, the graphics engine tiles **310A-310D** of multiple GPUs can be presented to a host system as a single logical device.

The graphics processor **320** may optionally include a display controller **302** to enable a connection with the display device **318**. The graphics processor may also be configured as a graphics or compute accelerator. In the accelerator configuration, the display controller **302** and display device **318** may be omitted.

The graphics processor **320** can connect to a host system via a host interface **328**. The host interface **328** can enable communication between the graphics processor **320**, system memory, and/or other system components. The host interface **328** can be, for example a PCI express bus or another type of host system interface. For example, the host interface **328** may be an NVLink or NVSwitch interface. The host interface **328** and fabric interconnect **324** can cooperate to enable multiple instances of the graphics processor **320** to act as single logical device. Cooperation between the host interface **328** and fabric interconnect **324** can also enable the individual graphics engine tiles **310A-310D** to be presented to the host system as distinct logical graphics devices.

FIG. 3C illustrates a compute accelerator **330**, according to embodiments described herein. The compute accelerator **330** can include architectural similarities with the graphics processor **320** of FIG. 3B and is optimized for compute acceleration. A compute engine cluster **332** can include a set of compute engine tiles **340A-340D** that include execution logic that is optimized for parallel or vector-based general-purpose compute operations. In some embodiments, the compute engine tiles **340A-340D** do not include fixed function graphics processing logic, although in one embodiment one or more of the compute engine tiles **340A-340D** can include logic to perform media acceleration. The compute engine tiles **340A-340D** can connect to memory **326A-326D** via memory interconnects **325A-325D**. The memory **326A-326D** and memory interconnects **325A-325D** may be similar technology as in graphics processor **320**, or can be different. The graphics compute engine tiles **340A-340D** can also be interconnected via a set of tile interconnects **323A-323F** and may be connected with and/or interconnected by a fabric interconnect **324**. Cross-tile communications can be facilitated via the fabric interconnect **324**. The fabric interconnect **324** (e.g., via the host interface **328**) can also facilitate communication between compute engine tiles **340A-340D** of multiple instances of the compute accelerator **330**. In one embodiment the compute accelerator **330** includes a large L3 cache **336** that can be configured as a device-wide cache. The compute accelerator **330** can also connect to a host processor and memory via a host interface **328** in a similar manner as the graphics processor **320** of FIG. 3B.

The compute accelerator **330** can also include a network interface **342**, which may be an integrated network interface. In one embodiment the network interface **342** includes a network processor and controller logic that enables the compute engine cluster **332** to communicate over a physical layer interconnect **344** without requiring data to traverse memory of a host system. In one embodiment, one of the

compute engine tiles **340A-340D** is replaced by network processor logic and data to be transmitted or received via the physical layer interconnect **344** may be transmitted directly to or from memory **326A-326D**. Multiple instances of the compute accelerator **330** may be joined via the physical layer interconnect **344** into a single logical device. Alternatively, the various compute engine tiles **340A-340D** may be presented as distinct network accessible compute accelerator devices.

#### Graphics Processing Engine

FIG. 4 is a block diagram of a graphics processing engine of a graphics processor in accordance with some embodiments. In one embodiment, the graphics processing engine (GPE **410**) is a version of the GPE **310** shown in FIG. 3A, and may also represent a graphics engine tile **310A-310D** of FIG. 3B. Elements of FIG. 4 having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such. For example, the 3D pipeline **312** and media pipeline **316** of FIG. 3A are illustrated. The media pipeline **316** is optional in some embodiments of the GPE **410** and may not be explicitly included within the GPE **410**. For example and in at least one embodiment, a separate media and/or image processor is coupled to the GPE **410**.

In some embodiments, GPE **410** couples with or includes a command streamer **403**, which provides a command stream to the 3D pipeline **312** and/or media pipelines **316**. Alternatively or additionally, the command streamer **403** may be directly coupled to a URB **418**. The URB **418** may be communicatively coupled to a graphics core array **414**. In some embodiments, command streamer **403** is coupled with memory, which can be system memory, or one or more of internal cache memory and shared cache memory. In some embodiments, command streamer **403** receives commands from the memory and sends the commands to 3D pipeline **312** and/or media pipeline **316**. The commands are directives fetched from a ring buffer, which stores commands for the 3D pipeline **312** and media pipeline **316**. In one embodiment, the ring buffer can additionally include batch command buffers storing batches of multiple commands. The commands for the 3D pipeline **312** can also include references to data stored in memory, such as but not limited to vertex and geometry data for the 3D pipeline **312** and/or image data and memory objects for the media pipeline **316**. The 3D pipeline **312** and media pipeline **316** process the commands and data by performing operations via logic within the respective pipelines or by dispatching one or more execution threads to a graphics core array **414**. In one embodiment the graphics core array **414** include one or more blocks of graphics cores (e.g., graphics core(s) **415A**, graphics core(s) **415B**), each block including one or more graphics cores. Each graphics core includes a set of graphics execution resources that includes general-purpose and graphics specific execution logic to perform graphics and compute operations, as well as fixed function texture processing and/or machine learning and artificial intelligence acceleration logic.

In various embodiments the 3D pipeline **312** can include fixed function and programmable logic to process one or more shader programs, such as vertex shaders, geometry shaders, pixel shaders, fragment shaders, compute shaders, or other shader and/or GPGPU programs, by processing the instructions and dispatching execution threads to the graphics core array **414**. The graphics core array **414** provides a unified block of execution resources for use in processing these shader programs. Multi-purpose execution logic (e.g.,

execution units) within the graphics core(s) **415A-414B** of the graphics core array **414** includes support for various 3D API shader languages and can execute multiple simultaneous execution threads associated with multiple shaders.

In some embodiments, the graphics core array **414** includes execution logic to perform media functions, such as video and/or image processing. In one embodiment, the execution units include general-purpose logic that is programmable to perform parallel general-purpose computational operations, in addition to graphics processing operations. The general-purpose logic can perform processing operations in parallel or in conjunction with general-purpose logic within the processor core(s) **107** of FIG. 1 or core **202A-202N** as in FIG. 2A.

Output data generated by threads executing on the graphics core array **414** can output data to memory in a URB **418**. The URB **418** can store data for multiple threads. In some embodiments the URB **418** may be used to send data between different threads executing on the graphics core array **414**. In some embodiments the URB **418** may additionally be used for synchronization between threads on the graphics core array and fixed function logic within the shared function logic **420**.

In some embodiments, graphics core array **414** is scalable, such that the array includes a variable number of graphics cores, each having a variable number of execution units based on the target power and performance level of GPE **410**. In one embodiment the execution resources are dynamically scalable, such that execution resources may be enabled or disabled as needed.

The graphics core array **414** couples with shared function logic **420** that includes multiple resources that are shared between the graphics cores in the graphics core array. The shared functions within the shared function logic **420** are hardware logic units that provide specialized supplemental functionality to the graphics core array **414**. In various embodiments, shared function logic **420** includes but is not limited to sampler logic **421**, math logic **422**, and inter-thread communication (ITC) **423** logic. Additionally, some embodiments implement one or more cache(s) **425** within the shared function logic **420**.

A shared function is implemented at least in a case where the demand for a given specialized function is insufficient for inclusion within the graphics core array **414**. Instead, a single instantiation of that specialized function is implemented as a stand-alone entity in the shared function logic **420** and shared among the execution resources within the graphics core array **414**. The precise set of functions that are shared between the graphics core array **414** and included within the graphics core array **414** varies across embodiments. In some embodiments, specific shared functions within the shared function logic **420** that are used extensively by the graphics core array **414** may be included within shared function logic **416** within the graphics core array **414**. In various embodiments, the shared function logic **416** within the graphics core array **414** can include some or all logic within the shared function logic **420**. In one embodiment, all logic elements within the shared function logic **420** may be duplicated within the shared function logic **416** of the graphics core array **414**. In one embodiment the shared function logic **420** is excluded in favor of the shared function logic **416** within the graphics core array **414**.

#### Execution Units

FIGS. 5A-5B illustrate thread execution logic **500** including an array of processing elements employed in a graphics processor core according to embodiments described herein. Elements of FIGS. 5A-5B having the same reference num-

bers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such. FIG. 5A-5B illustrates an overview of thread execution logic **500**, which may be representative of hardware logic illustrated with each sub-core **221A-221F** of FIG. 2B. FIG. 5A is representative of an execution unit within a general-purpose graphics processor, while FIG. 5B is representative of an execution unit that may be used within a compute accelerator.

As illustrated in FIG. 5A, in some embodiments thread execution logic **500** includes a shader processor **502**, a thread dispatcher **504**, instruction cache **506**, a scalable execution unit array including a plurality of graphics execution units **508A-508N**, a sampler **510**, shared local memory **511**, a data cache **512**, and a data port **514**. In one embodiment the scalable execution unit array can dynamically scale by enabling or disabling one or more execution units (e.g., any of graphics execution units **508A**, **508B**, **508C**, **508D**, through **508N-1** and **508N**) based on the computational requirements of a workload. In one embodiment the included components are interconnected via an interconnect fabric that links to each of the components. In some embodiments, thread execution logic **500** includes one or more connections to memory, such as system memory or cache memory, through one or more of instruction cache **506**, data port **514**, sampler **510**, and graphics execution units **508A-508N**. In some embodiments, each execution unit (e.g., graphics execution unit **508A**) is a stand-alone programmable general-purpose computational unit that is capable of executing multiple simultaneous hardware threads while processing multiple data elements in parallel for each thread. In various embodiments, the array of graphics execution units **508A-508N** is scalable to include any number individual execution units.

In some embodiments, the graphics execution units **508A-508N** are primarily used to execute shader programs. A shader processor **502** can process the various shader programs and dispatch execution threads associated with the shader programs via a thread dispatcher **504**. In one embodiment the thread dispatcher includes logic to arbitrate thread initiation requests from the graphics and media pipelines and instantiate the requested threads on one or more execution unit in the graphics execution units **508A-508N**. For example, a geometry pipeline can dispatch vertex, tessellation, or geometry shaders to the thread execution logic for processing. In some embodiments, thread dispatcher **504** can also process runtime thread spawning requests from the executing shader programs.

In some embodiments, the graphics execution units **508A-508N** support an instruction set that includes native support for many standard 3D graphics shader instructions, such that shader programs from graphics libraries (e.g., Direct 3D and OpenGL) are executed with a minimal translation. The execution units support vertex and geometry processing (e.g., vertex programs, geometry programs, vertex shaders), pixel processing (e.g., pixel shaders, fragment shaders) and general-purpose processing (e.g., compute and media shaders). Each of the graphics execution units **508A-508N** is capable of multi-issue single instruction multiple data (SIMD) execution and multi-threaded operation enables an efficient execution environment in the face of higher latency memory accesses. Each hardware thread within each execution unit has a dedicated high-bandwidth register file and associated independent thread-state. Execution is multi-issue per clock to pipelines capable of integer, single and double precision floating point operations, SIMD branch

capability, logical operations, transcendental operations, and other miscellaneous operations. While waiting for data from memory or one of the shared functions, dependency logic within the graphics execution units **508A-508N** causes a waiting thread to sleep until the requested data has been returned. While the waiting thread is sleeping, hardware resources may be devoted to processing other threads. For example, during a delay associated with a vertex shader operation, an execution unit can perform operations for a pixel shader, fragment shader, or another type of shader program, including a different vertex shader. Various embodiments can apply to use execution by use of Single Instruction Multiple Thread (SIMT) as an alternate to use of SIMD or in addition to use of SIMD. Reference to a SIMD core or operation can apply also to SIMT or apply to SIMD in combination with SIMT.

Each execution unit in graphics execution units **508A-508N** operates on arrays of data elements. The number of data elements is the “execution size,” or the number of channels for the instruction. An execution channel is a logical unit of execution for data element access, masking, and flow control within instructions. The number of channels may be independent of the number of physical Arithmetic Logic Units (ALUs), Floating Point Units (FPUs), or other logic units (e.g., tensor cores, ray tracing cores, etc.) for a particular graphics processor. In some embodiments, graphics execution units **508A-508N** support integer and floating-point data types.

The execution unit instruction set includes SIMD instructions. The various data elements can be stored as a packed data type in a register and the execution unit will process the various elements based on the data size of the elements. For example, when operating on a 256-bit wide vector, the 256 bits of the vector are stored in a register and the execution unit operates on the vector as four separate 54-bit packed data elements (Quad-Word (QW) size data elements), eight separate 32-bit packed data elements (Double Word (DW) size data elements), sixteen separate 16-bit packed data elements (Word (W) size data elements), or thirty-two separate 8-bit data elements (byte (B) size data elements). However, different vector widths and register sizes are possible.

In one embodiment one or more graphics execution units can be combined into a fused graphics execution unit **509A-509N** having thread control logic (**507A-507N**) that is common to the fused EUs. Multiple EUs can be fused into an EU group. Each EU in the fused EU group can be configured to execute a separate SIMD hardware thread. The number of EUs in a fused EU group can vary according to embodiments. Additionally, various SIMD widths can be performed per-EU, including but not limited to SIMD8, SIMD16, and SIMD32. Each fused graphics execution unit **509A-509N** includes at least two execution units. For example, fused graphics execution unit **509A** includes a first EU (graphics execution unit **508A**), second EU (graphics execution unit **508B**), and thread control logic **507A** that is common to the first EU and the second EU. The thread control logic **507A** controls threads executed on the fused graphics execution unit **509A**, allowing each EU within the fused graphics execution units **509A-509N** to execute using a common instruction pointer register.

One or more internal instruction caches (e.g., **506**) are included in the thread execution logic **500** to cache thread instructions for the execution units. In some embodiments, one or more data caches (e.g., **512**) are included to cache thread data during thread execution. Threads executing on the execution logic **500** can also store explicitly managed

data in the shared local memory **511**. In some embodiments, a sampler **510** is included to provide texture sampling for 3D operations and media sampling for media operations. In some embodiments, sampler **510** includes specialized texture or media sampling functionality to process texture or media data during the sampling process before providing the sampled data to an execution unit.

During execution, the graphics and media pipelines send thread initiation requests to thread execution logic **500** via thread spawning and dispatch logic. Once a group of geometric objects has been processed and rasterized into pixel data, pixel processor logic (e.g., pixel shader logic, fragment shader logic, etc.) within the shader processor **502** is invoked to further compute output information and cause results to be written to output surfaces (e.g., color buffers, depth buffers, stencil buffers, etc.). In some embodiments, a pixel shader or fragment shader calculates the values of the various vertex attributes that are to be interpolated across the rasterized object. In some embodiments, pixel processor logic within the shader processor **502** then executes an application programming interface (API)-supplied pixel or fragment shader program. To execute the shader program, the shader processor **502** dispatches threads to an execution unit (e.g., **508A**) via thread dispatcher **504**. In some embodiments, shader processor **502** uses texture sampling logic in the sampler **510** to access texture data in texture maps stored in memory. Arithmetic operations on the texture data and the input geometry data compute pixel color data for each geometric fragment, or discards one or more pixels from further processing.

In some embodiments, the data port **514** provides a memory access mechanism for the thread execution logic **500** to output processed data to memory for further processing on a graphics processor output pipeline. In some embodiments, the data port **514** includes or couples to one or more cache memories (e.g., data cache **512**) to cache data for memory access via the data port.

In one embodiment, the execution logic **500** can also include a ray tracer **505** that can provide ray tracing acceleration functionality. The ray tracer **505** can support a ray tracing instruction set that includes instructions/functions for ray generation. The ray tracing instruction set can be similar to or different from the ray-tracing instruction set supported by the ray tracing cores **245** in FIG. 2C.

FIG. 5B illustrates exemplary internal details of a graphics execution unit **508**, according to embodiments. The graphics execution unit **508** can include an instruction fetch unit **537**, a general register file array (GRF) **524**, an architectural register file array (ARF) **526**, a thread arbiter **522**, a send unit **530**, a branch unit **532**, a set of SIMD floating point units (FPUs) **534**, and in one embodiment a set of dedicated integer SIMD ALUs **535**. The GRF **524** and ARF **526** includes the set of general register files and architecture register files associated with each simultaneous hardware thread that may be active in the graphics execution unit **508**. In one embodiment, per thread architectural state is maintained in the ARF **526**, while data used during thread execution is stored in the GRF **524**. The execution state of each thread, including the instruction pointers for each thread, can be held in thread-specific registers in the ARF **526**.

In one embodiment the graphics execution unit **508** has an architecture that is a combination of Simultaneous Multi-Threading (SMT) and fine-grained Interleaved Multi-Threading (IMT). The architecture has a modular configuration that can be fine-tuned at design time based on a target number of simultaneous threads and number of registers per

execution unit, where execution unit resources are divided across logic used to execute multiple simultaneous threads. The number of logical threads that may be executed by the graphics execution unit **508** is not limited to the number of hardware threads, and multiple logical threads can be assigned to each hardware thread.

In one embodiment, the graphics execution unit **508** can co-issue multiple instructions, which may each be different instructions. The thread arbiter **522** of the graphics execution unit **508** can dispatch the instructions to one of the send unit **530**, branch unit **532**, or SIMD FPU(s) **534** for execution. Each execution thread can access **128** general-purpose registers within the GRF **524**, where each register can store 32 bytes, accessible as a SIMD 8-element vector of 32-bit data elements. In one embodiment, each execution unit thread has access to 4 Kbytes within the GRF **524**, although embodiments are not so limited, and greater or fewer register resources may be provided in other embodiments. In one embodiment the graphics execution unit **508** is partitioned into seven hardware threads that can independently perform computational operations, although the number of threads per execution unit can also vary according to embodiments. For example, in one embodiment up to 16 hardware threads are supported. In an embodiment in which seven threads may access 4 Kbytes, the GRF **524** can store a total of 28 Kbytes. Where 16 threads may access 4 Kbytes, the GRF **524** can store a total of 64 Kbytes. Flexible addressing modes can permit registers to be addressed together to build effectively wider registers or to represent strided rectangular block data structures.

In one embodiment, memory operations, sampler operations, and other longer-latency system communications are dispatched via “send” instructions that are executed by the message passing send unit **530**. In one embodiment, branch instructions are dispatched to a dedicated branch unit **532** to facilitate SIMD divergence and eventual convergence.

In one embodiment the graphics execution unit **508** includes one or more SIMD floating point units (FPU(s)) **534** to perform floating-point operations. In one embodiment, the FPU(s) **534** also support integer computation. In one embodiment the FPU(s) **534** can SIMD execute up to M number of 32-bit floating-point (or integer) operations, or SIMD execute up to 2M 16-bit integer or 16-bit floating-point operations. In one embodiment, at least one of the FPU(s) provides extended math capability to support high-throughput transcendental math functions and double precision 54-bit floating-point. In some embodiments, a set of 8-bit integer SIMD ALUs **535** are also present, and may be specifically optimized to perform operations associated with machine learning computations.

In one embodiment, arrays of multiple instances of the graphics execution unit **508** can be instantiated in a graphics sub-core grouping (e.g., a sub-slice). For scalability, product architects can choose the exact number of execution units per sub-core grouping. In one embodiment the graphics execution unit **508** can execute instructions across a plurality of execution channels. In a further embodiment, each thread executed on the graphics execution unit **508** is executed on a different channel.

FIG. 6 illustrates an additional execution unit **600**, according to an embodiment. The execution unit **600** may be a compute-optimized execution unit for use in, for example, a compute engine tile **340A-340D** as in FIG. 3C, but is not limited as such. Variants of the execution unit **600** may also be used in a graphics engine tile **310A-310D** as in FIG. 3B. In one embodiment, the execution unit **600** includes a thread control unit **601**, a thread state unit **602**, an instruction

fetch/prefetch unit **603**, and an instruction decode unit **604**. The execution unit **600** additionally includes a register file **606** that stores registers that can be assigned to hardware threads within the execution unit. The execution unit **600** additionally includes a send unit **607** and a branch unit **608**. In one embodiment, the send unit **607** and branch unit **608** can operate similarly as the send unit **530** and a branch unit **532** of the graphics execution unit **508** of FIG. 5B.

The execution unit **600** also includes a compute unit **610** that includes multiple different types of functional units. The compute unit **610** can include an ALU **611**, a systolic array **612**, and a math unit **613**. The ALU **611** includes an array of arithmetic logic units. The ALU **611** can be configured to perform 64-bit, 32-bit, and 16-bit integer and floating point operations across multiple processing lanes and data channels and for multiple hardware and/or software threads. The ALU **611** can perform integer and floating point operations simultaneously (e.g., within the same clock cycle).

The systolic array **612** includes a W wide and D deep network of data processing units that can be used to perform vector or other data-parallel operations in a systolic manner. In one embodiment the systolic array **612** can be configured to perform various matrix operations, including as dot product, outer product, and general matrix-matrix multiplication (GEMM) operations. In one embodiment the systolic array **612** supports 16-bit floating point operations, as well as 8-bit, 4-bit, 2-bit, and binary integer operations. The systolic array **612** can be configured to accelerate specific machine learning operations, in addition to matrix multiply operations. In such embodiments, the systolic array **612** can be configured with support for the bfloat (brain floating point) 16-bit floating point format or a tensor float 32-bit floating point format (TF32) that have different numbers of mantissa and exponent bits relative to Institute of Electrical and Electronics Engineers (IEEE) 754 formats.

The systolic array **612** includes hardware to accelerate sparse matrix operations. In one embodiment, multiplication operations for sparse regions of input data can be bypassed at the processing element level by skipping multiply operations that have a zero value operand. In one embodiment, sparsity within input matrices can be detected and operations having known output values can be bypassed before being submitted to the processing elements of the systolic array **612**. Additionally, the loading of zero value operands into the processing elements can be bypassed and the processing elements can be configured to perform multiplications on the non-zero value input elements. Output can be generated in a compressed (e.g., dense) format, with associated decompression or decoding metadata. The output can be cached in the compressed format. The output can be maintained in the compressed format when written to local memory or host system memory. The output may also be decompressed before being written to local memory or host system memory.

In one embodiment, the systolic array **612** includes hardware to enable operations on sparse data having a compressed representation. A compressed representation of a sparse matrix stores non-zero values and metadata that identifies the positions of the non-zero values within the matrix. Exemplary compressed representations include but are not limited to compressed tensor representations such as compressed sparse row (CSR), compressed sparse column (CSC), compressed sparse fiber (CSF) representations. Support for compressed representations enable operations to be performed on input in a compressed tensor format without requiring the compressed representation to be decompressed or decoded. In such embodiment, operations can be per-

25

formed only on non-zero input values and the resulting non-zero output values can be mapped into an output matrix. In some embodiments, hardware support is also provided for machine-specific lossless data compression formats that are used when transmitting data within hardware or across system busses. Such data may be retained in a compressed format for sparse input data and the systolic array **612** can use the compression metadata for the compressed data to enable operations to be performed on only non-zero values, or to enable blocks of zero data input to be bypassed for multiply operations.

In one embodiment, a math unit **613** can be included to perform a specific subset of mathematical operations in an efficient and lower-power manner than the ALU **611**. The math unit **613** can include a variant of math logic that may be found in shared function logic of a graphics processing engine provided by other embodiments (e.g., math logic **422** of the shared function logic **420** of FIG. 4). In one embodiment the math unit **613** can be configured to perform 32-bit and 64-bit floating point operations.

The thread control unit **601** includes logic to control the execution of threads within the execution unit. The thread control unit **601** can include thread arbitration logic to start, stop, and preempt execution of threads within the execution unit **600**. The thread state unit **602** can be used to store thread state for threads assigned to execute on the execution unit **600**. Storing the thread state within the execution unit **600** enables the rapid pre-emption of threads when those threads become blocked or idle. The instruction fetch/prefetch unit **603** can fetch instructions from an instruction cache of higher-level execution logic (e.g., instruction cache **506** as in FIG. 5A). The instruction fetch/prefetch unit **603** can also issue prefetch requests for instructions to be loaded into the instruction cache based on an analysis of currently executing threads. The instruction decode unit **604** can be used to decode instructions to be executed by the compute units. In one embodiment, the instruction decode unit **604** can be used as a secondary decoder to decode complex instructions into constituent micro-operations.

The execution unit **600** additionally includes a register file **606** that can be used by hardware threads executing on the execution unit **600**. Registers in the register file **606** can be divided across the logic used to execute multiple simultaneous threads within the compute unit **610** of the execution unit **600**. The number of logical threads that may be executed by the graphics execution unit **600** is not limited to the number of hardware threads, and multiple logical threads can be assigned to each hardware thread. The size of the register file **606** can vary across embodiments based on the number of supported hardware threads. In one embodiment, register renaming may be used to dynamically allocate registers to hardware threads.

FIG. 7 is a block diagram illustrating graphics processor instruction formats **700** according to some embodiments. In one or more embodiment, the graphics processor execution units support an instruction set having instructions in multiple formats. The solid lined boxes illustrate the components that are generally included in an execution unit instruction, while the dashed lines include components that are optional or that are only included in a sub-set of the instructions. In some embodiments, the graphics processor instruction format **700** described and illustrated are macro-instructions, in that they are instructions supplied to the execution unit, as opposed to micro-operations resulting from instruction decode once the instruction is processed. Thus, a single instructions may cause hardware to perform multiple micro-operations.

26

In some embodiments, the graphics processor execution units natively support instructions in a 128-bit instruction format **710**. A 64-bit compacted instruction format **730** is available for some instructions based on the selected instruction, instruction options, and number of operands. The native 128-bit instruction format **710** provides access to all instruction options, while some options and operations are restricted in the 64-bit compacted instruction format **730**. The native instructions available in the 64-bit compacted instruction format **730** vary by embodiment. In some embodiments, the instruction is compacted in part using a set of index values in an index field **713**. The execution unit hardware references a set of compaction tables based on the index values and uses the compaction table outputs to reconstruct a native instruction in the 128-bit instruction format **710**. Other sizes and formats of instruction can be used.

For each format, instruction opcode **712** defines the operation that the execution unit is to perform. The execution units execute each instruction in parallel across the multiple data elements of each operand. For example, in response to an add instruction the execution unit performs a simultaneous add operation across each color channel representing a texture element or picture element. By default, the execution unit performs each instruction across all data channels of the operands. In some embodiments, instruction control field **714** enables control over certain execution options, such as channels selection (e.g., predication) and data channel order (e.g., swizzle). For instructions in the 128-bit instruction format **710** an exec-size field **716** limits the number of data channels that will be executed in parallel. In some embodiments, exec-size field **716** is not available for use in the 64-bit compacted instruction format **730**.

Some execution unit instructions have up to three operands including two source operands, src0 **720**, src1 **722**, and one destination **718**. In some embodiments, the execution units support dual destination instructions, where one of the destinations is implied. Data manipulation instructions can have a third source operand (e.g., SRC2 **724**), where the instruction opcode **712** determines the number of source operands. An instruction's last source operand can be an immediate (e.g., hard-coded) value passed with the instruction.

In some embodiments, the 128-bit instruction format **710** includes an access/address mode field **726** specifying, for example, whether direct register addressing mode or indirect register addressing mode is used. When direct register addressing mode is used, the register address of one or more operands is directly provided by bits in the instruction.

In some embodiments, the 128-bit instruction format **710** includes an access/address mode field **726**, which specifies an address mode and/or an access mode for the instruction. In one embodiment the access mode is used to define a data access alignment for the instruction. Some embodiments support access modes including a 16-byte aligned access mode and a 1-byte aligned access mode, where the byte alignment of the access mode determines the access alignment of the instruction operands. For example, when in a first mode, the instruction may use byte-aligned addressing for source and destination operands and when in a second mode, the instruction may use 16-byte-aligned addressing for all source and destination operands.

In one embodiment, the address mode portion of the access/address mode field **726** determines whether the instruction is to use direct or indirect addressing. When direct register addressing mode is used bits in the instruction directly provide the register address of one or more oper-

ands. When indirect register addressing mode is used, the register address of one or more operands may be computed based on an address register value and an address immediate field in the instruction.

In some embodiments instructions are grouped based on instruction opcode **712** bit-fields to simplify Opcode decode **740**. For an 8-bit opcode, bits **4**, **5**, and **6** allow the execution unit to determine the type of opcode. The precise opcode grouping shown is merely an example. In some embodiments, a move and logic opcode group **742** includes data movement and logic instructions (e.g., move (mov), compare (cmp)). In some embodiments, move and logic group **742** shares the five most significant bits (MSB), where move (mov) instructions are in the form of 0000xxxxb and logic instructions are in the form of 0001xxxxb. A flow control instruction group **744** (e.g., call, jump (jmp)) includes instructions in the form of 0010xxxxb (e.g., 0x20). A miscellaneous instruction group **746** includes a mix of instructions, including synchronization instructions (e.g., wait, send) in the form of 0011xxxxb (e.g., 0x30). A parallel math instruction group **748** includes component-wise arithmetic instructions (e.g., add, multiply (mul)) in the form of 0100xxxxb (e.g., 0x40). The parallel math instruction group **748** performs the arithmetic operations in parallel across data channels. The vector math group **750** includes arithmetic instructions (e.g., dp4) in the form of 0101xxxxb (e.g., 0x50). The vector math group performs arithmetic such as dot product calculations on vector operands. The illustrated opcode decode **740**, in one embodiment, can be used to determine which portion of an execution unit will be used to execute a decoded instruction. For example, some instructions may be designated as systolic instructions that will be performed by a systolic array. Other instructions, such as ray-tracing instructions (not shown) can be routed to a ray-tracing core or ray-tracing logic within a slice or partition of execution logic.

#### Graphics Pipeline

FIG. **8** is a block diagram of another embodiment of a graphics processor **800**. Elements of FIG. **8** having the same reference numbers (or names) as the elements of any other figure herein can operate or function in any manner similar to that described elsewhere herein, but are not limited to such.

In some embodiments, graphics processor **800** includes a geometry pipeline **820**, a media pipeline **830**, a display engine **840**, thread execution logic **850**, and a render output pipeline **870**. In some embodiments, graphics processor **800** is a graphics processor within a multi-core processing system that includes one or more general-purpose processing cores. The graphics processor is controlled by register writes to one or more control registers (not shown) or via commands issued to graphics processor **800** via a ring interconnect **802**. In some embodiments, ring interconnect **802** couples graphics processor **800** to other processing components, such as other graphics processors or general-purpose processors. Commands from ring interconnect **802** are interpreted by a command streamer **803**, which supplies instructions to individual components of the geometry pipeline **820** or the media pipeline **830**.

In some embodiments, command streamer **803** directs the operation of a vertex fetcher **805** that reads vertex data from memory and executes vertex-processing commands provided by command streamer **803**. In some embodiments, vertex fetcher **805** provides vertex data to a vertex shader **807**, which performs coordinate space transformation and lighting operations to each vertex. In some embodiments, vertex fetcher **805** and vertex shader **807** execute vertex-

processing instructions by dispatching execution threads to execution units **852A-852B** via a thread dispatcher **831**.

In some embodiments, execution units **852A-852B** are an array of vector processors having an instruction set for performing graphics and media operations. In some embodiments, execution units **852A-852B** have an attached L1 cache **851** that is specific for each array or shared between the arrays. The cache can be configured as a data cache, an instruction cache, or a single cache that is partitioned to contain data and instructions in different partitions.

In some embodiments, geometry pipeline **820** includes tessellation components to perform hardware-accelerated tessellation of 3D objects. In some embodiments, a programmable hull shader **811** configures the tessellation operations. A programmable domain shader **817** provides back-end evaluation of tessellation output. A tessellator **813** operates at the direction of hull shader **811** and contains special purpose logic to generate a set of detailed geometric objects based on a coarse geometric model that is provided as input to geometry pipeline **820**. In some embodiments, if tessellation is not used, tessellation components (e.g., hull shader **811**, tessellator **813**, and domain shader **817**) can be bypassed. The tessellation components can operate based on data received from the vertex shader **807**.

In some embodiments, complete geometric objects can be processed by a geometry shader **819** via one or more threads dispatched to execution units **852A-852B**, or can proceed directly to the clipper **829**. In some embodiments, the geometry shader operates on entire geometric objects, rather than vertices or patches of vertices as in previous stages of the graphics pipeline. If the tessellation is disabled the geometry shader **819** receives input from the vertex shader **807**. In some embodiments, geometry shader **819** is programmable by a geometry shader program to perform geometry tessellation if the tessellation units are disabled.

Before rasterization, a clipper **829** processes vertex data. The clipper **829** may be a fixed function clipper or a programmable clipper having clipping and geometry shader functions. In some embodiments, a rasterizer and depth test component **873** in the render output pipeline **870** dispatches pixel shaders to convert the geometric objects into per pixel representations. In some embodiments, pixel shader logic is included in thread execution logic **850**. In some embodiments, an application can bypass the rasterizer and depth test component **873** and access un-rasterized vertex data via a stream out unit **823**.

The graphics processor **800** has an interconnect bus, interconnect fabric, or some other interconnect mechanism that allows data and message passing amongst the major components of the processor. In some embodiments, execution units **852A-852B** and associated logic units (e.g., L1 cache **851**, sampler **854**, texture cache **858**, etc.) interconnect via a data port **856** to perform memory access and communicate with render output pipeline components of the processor. In some embodiments, sampler **854**, caches **851**, **858** and execution units **852A-852B** each have separate memory access paths. In one embodiment the texture cache **858** can also be configured as a sampler cache.

In some embodiments, render output pipeline **870** contains a rasterizer and depth test component **873** that converts vertex-based objects into an associated pixel-based representation. In some embodiments, the rasterizer logic includes a windower/masker unit to perform fixed function triangle and line rasterization. An associated render cache **878** and depth cache **879** are also available in some embodiments. A pixel operations component **877** performs pixel-based operations on the data, though in some instances, pixel

operations associated with 2D operations (e.g. bit block image transfers with blending) are performed by the 2D engine **841**, or substituted at display time by the display controller **843** using overlay display planes. In some embodiments, a shared L3 cache **875** is available to all graphics components, allowing the sharing of data without the use of main system memory.

In some embodiments, media pipeline **830** includes a media engine **837** and a video front-end **834**. In some embodiments, video front-end **834** receives pipeline commands from the command streamer **803**. In some embodiments, media pipeline **830** includes a separate command streamer. In some embodiments, video front-end **834** processes media commands before sending the command to the media engine **837**. In some embodiments, media engine **837** includes thread spawning functionality to spawn threads for dispatch to thread execution logic **850** via thread dispatcher **831**.

In some embodiments, graphics processor **800** includes a display engine **840**. In some embodiments, display engine **840** is external to processor **800** and couples with the graphics processor via the ring interconnect **802**, or some other interconnect bus or fabric. In some embodiments, display engine **840** includes a 2D engine **841** and a display controller **843**. In some embodiments, display engine **840** contains special purpose logic capable of operating independently of the 3D pipeline. In some embodiments, display controller **843** couples with a display device (not shown), which may be a system integrated display device, as in a laptop computer, or an external display device attached via a display device connector.

In some embodiments, the geometry pipeline **820** and media pipeline **830** are configurable to perform operations based on multiple graphics and media programming interfaces and are not specific to any one application programming interface (API). In some embodiments, driver software for the graphics processor translates API calls that are specific to a particular graphics or media library into commands that can be processed by the graphics processor. In some embodiments, support is provided for the Open Graphics Library (OpenGL), Open Computing Language (OpenCL), and/or Vulkan graphics and compute API, all from the Khronos Group. In some embodiments, support may also be provided for the Direct3D library from the Microsoft Corporation. In some embodiments, a combination of these libraries may be supported. Support may also be provided for the Open Source Computer Vision Library (OpenCV). A future API with a compatible 3D pipeline would also be supported if a mapping can be made from the pipeline of the future API to the pipeline of the graphics processor.

#### Graphics Pipeline Programming

FIG. 9A is a block diagram illustrating a graphics processor command format **900** that may be used to program graphics processing pipelines according to some embodiments. FIG. 9B is a block diagram illustrating a graphics processor command sequence **910** according to an embodiment. The solid lined boxes in FIG. 9A illustrate the components that are generally included in a graphics command while the dashed lines include components that are optional or that are only included in a sub-set of the graphics commands. The exemplary graphics processor command format **900** of FIG. 9A includes data fields to identify a client **902**, a command operation code (opcode) **904**, and a data field **906** for the command. A sub-opcode **905** and a command size **908** are also included in some commands.

In some embodiments, client **902** specifies the client unit of the graphics device that processes the command data. In some embodiments, a graphics processor command parser examines the client field of each command to condition the further processing of the command and route the command data to the appropriate client unit. In some embodiments, the graphics processor client units include a memory interface unit, a render unit, a 2D unit, a 3D unit, and a media unit. Each client unit has a corresponding processing pipeline that processes the commands. Once the command is received by the client unit, the client unit reads the opcode **904** and, if present, sub-opcode **905** to determine the operation to perform. The client unit performs the command using information in data field **906**. For some commands an explicit command size **908** is expected to specify the size of the command. In some embodiments, the command parser automatically determines the size of at least some of the commands based on the command opcode. In some embodiments commands are aligned via multiples of a double word. Other command formats can be used.

The flow diagram in FIG. 9B illustrates an exemplary graphics processor command sequence **910**. In some embodiments, software or firmware of a data processing system that features an embodiment of a graphics processor uses a version of the command sequence shown to set up, execute, and terminate a set of graphics operations. A sample command sequence is shown and described for purposes of example only as embodiments are not limited to these specific commands or to this command sequence. Moreover, the commands may be issued as batch of commands in a command sequence, such that the graphics processor will process the sequence of commands in at least partially concurrence.

In some embodiments, the graphics processor command sequence **910** may begin with a pipeline flush command **912** to cause any active graphics pipeline to complete the currently pending commands for the pipeline. In some embodiments, the 3D pipeline **922** and the media pipeline **924** do not operate concurrently. The pipeline flush is performed to cause the active graphics pipeline to complete any pending commands. In response to a pipeline flush, the command parser for the graphics processor will pause command processing until the active drawing engines complete pending operations and the relevant read caches are invalidated. Optionally, any data in the render cache that is marked 'dirty' can be flushed to memory. In some embodiments, pipeline flush command **912** can be used for pipeline synchronization or before placing the graphics processor into a low power state.

In some embodiments, a pipeline select command **913** is used when a command sequence requires the graphics processor to explicitly switch between pipelines. In some embodiments, a pipeline select command **913** is required only once within an execution context before issuing pipeline commands unless the context is to issue commands for both pipelines. In some embodiments, a pipeline flush command **912** is required immediately before a pipeline switch via the pipeline select command **913**.

In some embodiments, a pipeline control command **914** configures a graphics pipeline for operation and is used to program the 3D pipeline **922** and the media pipeline **924**. In some embodiments, pipeline control command **914** configures the pipeline state for the active pipeline. In one embodiment, the pipeline control command **914** is used for pipeline synchronization and to clear data from one or more cache memories within the active pipeline before processing a batch of commands.



31

In some embodiments, commands related to the return buffer state **916** are used to configure a set of return buffers for the respective pipelines to write data. Some pipeline operations require the allocation, selection, or configuration of one or more return buffers into which the operations write intermediate data during processing. In some embodiments, the graphics processor also uses one or more return buffers to store output data and to perform cross thread communication. In some embodiments, the return buffer state **916** includes selecting the size and number of return buffers to use for a set of pipeline operations.

The remaining commands in the command sequence differ based on the active pipeline for operations. Based on a pipeline determination **920**, the command sequence is tailored to the 3D pipeline **922** beginning with the 3D pipeline state **930** or the media pipeline **924** beginning at the media pipeline state **940**.

The commands to configure the 3D pipeline state **930** include 3D state setting commands for vertex buffer state, vertex element state, constant color state, depth buffer state, and other state variables that are to be configured before 3D primitive commands are processed. The values of these commands are determined at least in part based on the particular 3D API library in use. In some embodiments, 3D pipeline state **930** commands are also able to selectively disable or bypass certain pipeline elements if those elements will not be used.

In some embodiments, 3D primitive **932** command is used to submit 3D primitives to be processed by the 3D pipeline. Commands and associated parameters that are passed to the graphics processor via the 3D primitive **932** command are forwarded to the vertex fetch function in the graphics pipeline. The vertex fetch function uses the 3D primitive **932** command data to generate vertex data structures. The vertex data structures are stored in one or more return buffers. In some embodiments, 3D primitive **932** command is used to perform vertex operations on 3D primitives via vertex shaders. To process vertex shaders, 3D pipeline **922** dispatches shader execution threads to graphics processor execution units.

In some embodiments, 3D pipeline **922** is triggered via an execute **934** command or event. In some embodiments, a register write triggers command execution. In some embodiments execution is triggered via a 'go' or 'kick' command in the command sequence. In one embodiment, command execution is triggered using a pipeline synchronization command to flush the command sequence through the graphics pipeline. The 3D pipeline will perform geometry processing for the 3D primitives. Once operations are complete, the resulting geometric objects are rasterized and the pixel engine colors the resulting pixels. Additional commands to control pixel shading and pixel back end operations may also be included for those operations.

In some embodiments, the graphics processor command sequence **910** follows the media pipeline **924** path when performing media operations. In general, the specific use and manner of programming for the media pipeline **924** depends on the media or compute operations to be performed. Specific media decode operations may be offloaded to the media pipeline during media decode. In some embodiments, the media pipeline can also be bypassed and media decode can be performed in whole or in part using resources provided by one or more general-purpose processing cores. In one embodiment, the media pipeline also includes elements for general-purpose graphics processor unit (GPGPU) operations, where the graphics processor is used to perform

32

SIMD vector operations using computational shader programs that are not explicitly related to the rendering of graphics primitives.

In some embodiments, media pipeline **924** is configured in a similar manner as the 3D pipeline **922**. A set of commands to configure the media pipeline state **940** are dispatched or placed into a command queue before the media object commands **942**. In some embodiments, commands for the media pipeline state **940** include data to configure the media pipeline elements that will be used to process the media objects. This includes data to configure the video decode and video encode logic within the media pipeline, such as encode or decode format. In some embodiments, commands for the media pipeline state **940** also support the use of one or more pointers to "indirect" state elements that contain a batch of state settings.

In some embodiments, media object commands **942** supply pointers to media objects for processing by the media pipeline. The media objects include memory buffers containing video data to be processed. In some embodiments, all media pipeline states must be valid before issuing a media object command **942**. Once the pipeline state is configured and media object commands **942** are queued, the media pipeline **924** is triggered via an execute command **944** or an equivalent execute event (e.g., register write). Output from media pipeline **924** may then be post processed by operations provided by the 3D pipeline **922** or the media pipeline **924**. In some embodiments, GPGPU operations are configured and executed in a similar manner as media operations. Graphics Software Architecture

FIG. **10** illustrates an exemplary graphics software architecture for a data processing system **1000** according to some embodiments. In some embodiments, software architecture includes a 3D graphics application **1010**, an operating system **1020**, and at least one processor **1030**. In some embodiments, processor **1030** includes a graphics processor **1032** and one or more general-purpose processor core(s) **1034**. The graphics application **1010** and operating system **1020** each execute in the system memory **1050** of the data processing system.

In some embodiments, 3D graphics application **1010** contains one or more shader programs including shader instructions **1012**. The shader language instructions may be in a high-level shader language, such as the High-Level Shader Language (HLSL) of DirectX3D, the OpenGL Shader Language (GLSL), and so forth. The application also includes executable instructions **1014** in a machine language suitable for execution by the general-purpose processor core **1034**. The application also includes graphics objects **1016** defined by vertex data.

In some embodiments, operating system **1020** is a Microsoft® Windows® operating system from the Microsoft Corporation, a proprietary UNIX-like operating system, or an open source UNIX-like operating system using a variant of the Linux kernel. The operating system **1020** can support a graphics API **1022** such as the DirectX3D API, the OpenGL API, or the Vulkan API. When the DirectX3D API is in use, the operating system **1020** uses a front-end shader compiler **1024** to compile any shader instructions **1012** in HLSL into a lower-level shader language. The compilation may be a just-in-time (JIT) compilation or the application can perform shader pre-compilation. In some embodiments, high-level shaders are compiled into low-level shaders during the compilation of the 3D graphics application **1010**. In some embodiments, the shader instructions **1012** are provided in an intermediate form, such



as a version of the Standard Portable Intermediate Representation (SPIR) used by the Vulkan API.

In some embodiments, user mode graphics driver **1026** contains a back-end shader compiler **1027** to convert the shader instructions **1012** into a hardware specific representation. When the OpenGL API is in use, shader instructions **1012** in the GLSL high-level language are passed to a user mode graphics driver **1026** for compilation. In some embodiments, user mode graphics driver **1026** uses operating system kernel mode functions **1028** to communicate with a kernel mode graphics driver **1029**. In some embodiments, kernel mode graphics driver **1029** communicates with graphics processor **1032** to dispatch commands and instructions.

#### IP Core Implementations

One or more aspects of at least one embodiment may be implemented by representative code stored on a machine-readable medium which represents and/or defines logic within an integrated circuit such as a processor. For example, the machine-readable medium may include instructions which represent various logic within the processor. When read by a machine, the instructions may cause the machine to fabricate the logic to perform the techniques described herein. Such representations, known as “IP cores,” are reusable units of logic for an integrated circuit that may be stored on a tangible, machine-readable medium as a hardware model that describes the structure of the integrated circuit. The hardware model may be supplied to various customers or manufacturing facilities, which load the hardware model on fabrication machines that manufacture the integrated circuit. The integrated circuit may be fabricated such that the circuit performs operations described in association with any of the embodiments described herein.

FIG. **11A** is a block diagram illustrating an IP core development system **1100** that may be used to manufacture an integrated circuit to perform operations according to an embodiment. The IP core development system **1100** may be used to generate modular, re-usable designs that can be incorporated into a larger design or used to construct an entire integrated circuit (e.g., an SOC integrated circuit). A design facility **1130** can generate a software simulation **1110** of an IP core design in a high-level programming language (e.g., C/C++). The software simulation **1110** can be used to design, test, and verify the behavior of the IP core using a simulation model **1112**. The simulation model **1112** may include functional, behavioral, and/or timing simulations. A register transfer level (RTL) design **1115** can then be created or synthesized from the simulation model **1112**. The RTL design **1115** is an abstraction of the behavior of the integrated circuit that models the flow of digital signals between hardware registers, including the associated logic performed using the modeled digital signals. In addition to an RTL design **1115**, lower-level designs at the logic level or transistor level may also be created, designed, or synthesized. Thus, the particular details of the initial design and simulation may vary.

The RTL design **1115** or equivalent may be further synthesized by the design facility into a hardware model **1120**, which may be in a hardware description language (HDL), or some other representation of physical design data. The HDL may be further simulated or tested to verify the IP core design. The IP core design can be stored for delivery to a 3rd party fabrication facility **1165** using non-volatile memory **1140** (e.g., hard disk, flash memory, or any non-volatile storage medium). Alternatively, the IP core design may be transmitted (e.g., via the Internet) over a wired connection **1150** or wireless connection **1160**. The fabrica-

tion facility **1165** may then fabricate an integrated circuit that is based at least in part on the IP core design. The fabricated integrated circuit can be configured to perform operations in accordance with at least one embodiment described herein.

FIG. **11B** illustrates a cross-section side view of an integrated circuit package assembly **1170**, according to some embodiments described herein. The integrated circuit package assembly **1170** illustrates an implementation of one or more processor or accelerator devices as described herein. The package assembly **1170** includes multiple units of logic **1172** and logic or I/O **1174** connected to a substrate **1180**. The logic **1172** and logic or I/O **1174** may be implemented at least partly in configurable logic or fixed-functionality logic hardware, and can include one or more portions of any of the processor core(s), graphics processor(s), or other accelerator devices described herein. Each unit of logic **1172** and logic or I/O **1174** can be implemented within a semiconductor die and coupled with the substrate **1180** via an interconnect structure **1173**. The interconnect structure **1173** may be configured to route electrical signals between the logic **1172** and logic or I/O **1174** and the substrate **1180**, and can include interconnects such as, but not limited to bumps or pillars. In some embodiments, the interconnect structure **1173** may be configured to route electrical signals such as, for example, input/output (I/O) signals and/or power or ground signals associated with the operation of the logic **1172** and logic or I/O **1174**. In some embodiments, the substrate **1180** is an epoxy-based laminate substrate. The substrate **1180** may include other suitable types of substrates in other embodiments. The package assembly **1170** can be connected to other electrical devices via a package interconnect **1183**. The package interconnect **1183** may be coupled to a surface of the substrate **1180** to route electrical signals to other electrical devices, such as a motherboard, other chipset, or multi-chip module.

In some embodiments, the units of logic **1172** and logic or I/O **1174** are electrically coupled with a bridge **1182** that is configured to route electrical signals between the logic **1172** and logic or I/O **1174**. The bridge **1182** may be a dense interconnect structure that provides a route for electrical signals. The bridge **1182** may include a bridge substrate composed of glass or a suitable semiconductor material. Electrical routing features can be formed on the bridge substrate to provide a chip-to-chip connection between the logic **1172** and logic or I/O **1174**.

Although two units of logic **1172** and logic or I/O **1174** and a bridge **1182** are illustrated, embodiments described herein may include more or fewer logic units on one or more dies. The one or more dies may be connected by zero or more bridges, as the bridge **1182** may be excluded when the logic is included on a single die. Alternatively, multiple dies or units of logic can be connected by one or more bridges. Additionally, multiple logic units, dies, and bridges can be connected together in other possible configurations, including three-dimensional configurations.

FIG. **11C** illustrates a package assembly **1190** that includes multiple units of hardware logic chiplets connected to a substrate **1180**. A graphics processing unit, parallel processor, and/or compute accelerator as described herein can be composed from diverse silicon chiplets that are separately manufactured. In this context, a chiplet is an at least partially packaged integrated circuit that includes distinct units of logic that can be assembled with other chiplets into a larger package. A diverse set of chiplets with different IP core logic can be assembled into a single device. Additionally the chiplets can be integrated into a base die or base

chiplet using active interposer technology. The concepts described herein enable the interconnection and communication between the different forms of IP within the GPU. IP cores can be manufactured using different process technologies and composed during manufacturing, which avoids the complexity of converging multiple IPs, especially on a large SoC with several flavors IPs, to the same manufacturing process. Enabling the use of multiple process technologies improves the time to market and provides a cost-effective way to create multiple product SKUs. Additionally, the disaggregated IPs are more amenable to being power gated independently, components that are not in use on a given workload can be powered off, reducing overall power consumption.

In various embodiments a package assembly 1190 can include components and chiplets that are interconnected by a fabric 1185 and/or one or more bridges (bridge 1187). The chiplets within the package assembly 1190 may have a 2.5D arrangement using Chip-on-Wafer-on-Substrate stacking in which multiple dies are stacked side-by-side on a silicon interposer 1189 that couples the chiplets with the substrate 1180. The substrate 1180 includes electrical connections to the package interconnect 1183. In one embodiment the silicon interposer 1189 is a passive interposer that includes through-silicon vias (TSVs) to electrically couple chiplets within the package assembly 1190 to the substrate 1180. In one embodiment, silicon interposer 1189 is an active interposer that includes embedded logic in addition to TSVs. In such embodiment, the chiplets within the package assembly 1190 are arranged using 3D face to face die stacking on top of the active interposer 1189. The active interposer 1189 can include hardware logic for I/O 1191, cache memory 1192, and other hardware logic 1193, in addition to interconnect fabric 1185 and the bridge 1187. The fabric 1185 enables communication between the various logic chiplets (logic 1172, logic or I/O 1174) and the logic (I/O 1191, logic 1193) within the active interposer 1189. The fabric 1185 may be an NoC interconnect or another form of packet switched fabric that switches data packets between components of the package assembly. For complex assemblies, the fabric 1185 may be a dedicated chiplet enables communication between the various hardware logic of the package assembly 1190. The bridge 1187 within the active interposer 1189 may be used to facilitate a point to point interconnect between, for example, chiplets including logic or I/O 1174 and memory chiplets 1175. In some implementations, the bridge 1187 may also be embedded within the substrate 1180. The hardware logic chiplets can include special purpose hardware logic chiplets (logic 1172), logic or I/O 1174, and/or memory chiplets 1175. The chiplets for the logic 1172 and logic or I/O 1174 may be implemented at least partly in configurable logic or fixed-functionality logic hardware and can include one or more portions of any of the processor core(s), graphics processor(s), parallel processors, or other accelerator devices described herein. The memory chiplets 1175 can be DRAM (e.g., GDDR, HBM) memory or cache (SRAM) memory. Cache memory 1192 within the active interposer 1189 (or substrate 1180) can act as a global cache for the package assembly 1190, part of a distributed global cache, or as a dedicated cache for the fabric 1185.

Each chiplet can be fabricated as separate semiconductor die and coupled with a base die that is embedded within or coupled with the substrate 1180. The coupling with the substrate 1180 can be performed via an interconnect structure 1173. The interconnect structure 1173 may be configured to route electrical signals between the various chiplets and logic within the substrate 1180. The interconnect struc-

ture 1173 can include interconnects such as, but not limited to bumps or pillars. In some embodiments, the interconnect structure 1173 may be configured to route electrical signals such as, for example, input/output (I/O) signals and/or power or ground signals associated with the operation of the logic, I/O and memory chiplets. In one embodiment, an additional interconnect structure couples the active interposer 1189 with the substrate 1180.

In some embodiments, the substrate 1180 is an epoxy-based laminate substrate. The substrate 1180 may include other suitable types of substrates in other embodiments. The package assembly 1190 can be connected to other electrical devices via a package interconnect 1183. The package interconnect 1183 may be coupled to a surface of the substrate 1180 to route electrical signals to other electrical devices, such as a motherboard, other chipset, or multi-chip module.

In some embodiments, a chiplet including logic or I/O 1174 and a memory chiplet 1175 can be electrically coupled via a bridge 1187 that is configured to route electrical signals between the chiplet including logic or I/O 1174 and a memory chiplet 1175. The bridge 1187 may be a dense interconnect structure that provides a route for electrical signals. The bridge 1187 may include a bridge substrate composed of glass or a suitable semiconductor material. Electrical routing features can be formed on the bridge substrate to provide a chip-to-chip connection between the logic or I/O 1174 and a memory chiplet 1175. The bridge 1187 may also be referred to as a silicon bridge or an interconnect bridge. For example, the bridge 1187, in some embodiments, is an Embedded Multi-die Interconnect Bridge (EMIB). In some embodiments, the bridge 1187 may simply be a direct connection from one chiplet to another chiplet.

FIG. 11D illustrates a package assembly 1194 including interchangeable chiplets 1195, according to an embodiment. The interchangeable chiplets 1195 can be assembled into standardized slots on one or more base chiplets 1196, 1198. The base chiplets 1196, 1198 can be coupled via a bridge interconnect 1197, which can be similar to the other bridge interconnects described herein and may be, for example, an EMIB. Memory chiplets can also be connected to logic or I/O chiplets via a bridge interconnect. I/O and logic chiplets can communicate via an interconnect fabric. The base chiplets can each support one or more slots in a standardized format for one of logic or I/O or memory/cache.

In one embodiment, SRAM and power delivery circuits can be fabricated into one or more of the base chiplets 1196, 1198, which can be fabricated using a different process technology relative to the interchangeable chiplets 1195 that are stacked on top of the base chiplets. For example, the base chiplets 1196, 1198 can be fabricated using a larger process technology, while the interchangeable chiplets can be manufactured using a smaller process technology. One or more of the interchangeable chiplets 1195 may be memory (e.g., DRAM) chiplets. Different memory densities can be selected for the package assembly 1194 based on the power, and/or performance targeted for the product that uses the package assembly 1194. Additionally, logic chiplets with a different number of type of functional units can be selected at time of assembly based on the power, and/or performance targeted for the product. Additionally, chiplets containing IP logic cores of differing types can be inserted into the interchangeable chiplet slots, enabling hybrid processor designs that can mix and match different technology IP blocks.

Exemplary System on a Chip Integrated Circuit

FIGS. 12-13B illustrate exemplary integrated circuits and associated graphics processors that may be fabricated using one or more IP cores, according to various embodiments described herein. In addition to what is illustrated, other logic and circuits may be included, including additional graphics processors/cores, peripheral interface controllers, or general-purpose processor cores.

FIG. 12 is a block diagram illustrating an exemplary system on a chip integrated circuit 1200 that may be fabricated using one or more IP cores, according to an embodiment. Exemplary integrated circuit 1200 includes one or more application processor(s) 1205 (e.g., CPUs), at least one graphics processor 1210, and may additionally include an image processor 1215 and/or a video processor 1220, any of which may be a modular IP core from the same or multiple different design facilities. Integrated circuit 1200 includes peripheral or bus logic including a USB controller 1225, UART controller 1230, an SPI/SDIO controller 1235, and an I<sup>2</sup>S/I<sup>2</sup>C controller 1240. Additionally, the integrated circuit can include a display device 1245 coupled to one or more of a high-definition multimedia interface (HDMI) controller 1250 and a mobile industry processor interface (MIPI) display interface 1255. Storage may be provided by a flash memory subsystem 1260 including flash memory and a flash memory controller. Memory interface may be provided via a memory controller 1265 for access to SDRAM or SRAM memory devices. Some integrated circuits additionally include an embedded security engine 1270.

FIGS. 13A-13B are block diagrams illustrating exemplary graphics processors for use within an SoC, according to embodiments described herein. FIG. 13A illustrates an exemplary graphics processor 1310 of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to an embodiment. FIG. 13B illustrates an additional exemplary graphics processor 1340 of a system on a chip integrated circuit that may be fabricated using one or more IP cores, according to an embodiment. Graphics processor 1310 of FIG. 13A is an example of a low power graphics processor core. Graphics processor 1340 of FIG. 13B is an example of a higher performance graphics processor core. Each of graphics processor 1310 and graphics processor 1340 can be variants of the graphics processor 1210 of FIG. 12.

As shown in FIG. 13A, graphics processor 1310 includes a vertex processor 1305 and one or more fragment processor(s) 1315A-1315N (e.g., 1315A, 1315B, 1315C, 1315D, through 1315N-1, and 1315N). Graphics processor 1310 can execute different shader programs via separate logic, such that the vertex processor 1305 is optimized to execute operations for vertex shader programs, while the one or more fragment processor(s) 1315A-1315N execute fragment (e.g., pixel) shading operations for fragment or pixel shader programs. The vertex processor 1305 performs the vertex processing stage of the 3D graphics pipeline and generates primitives and vertex data. The fragment processor(s) 1315A-1315N use the primitive and vertex data generated by the vertex processor 1305 to produce a framebuffer that is displayed on a display device. In one embodiment, the fragment processor(s) 1315A-1315N are optimized to execute fragment shader programs as provided for in the OpenGL API, which may be used to perform similar operations as a pixel shader program as provided for in the Direct 3D API.

Graphics processor 1310 additionally includes one or more memory management units (MMUs) 1320A-1320B, cache(s) 1325A-1325B, and circuit interconnect(s) 1330A-

1330B. The one or more MMU(s) 1320A-1320B provide for virtual to physical address mapping for the graphics processor 1310, including for the vertex processor 1305 and/or fragment processor(s) 1315A-1315N, which may reference vertex or image/texture data stored in memory, in addition to vertex or image/texture data stored in the one or more cache(s) 1325A-1325B. In one embodiment the one or more MMU(s) 1320A-1320B may be synchronized with other MMUs within the system, including one or more MMUs associated with the one or more application processor(s) 1205, image processor 1215, and/or video processor 1220 of FIG. 12, such that each processor 1205-1220 can participate in a shared or unified virtual memory system. The one or more circuit interconnect(s) 1330A-1330B enable graphics processor 1310 to interface with other IP cores within the SoC, either via an internal bus of the SoC or via a direct connection, according to embodiments.

As shown FIG. 13B, graphics processor 1340 includes the one or more MMU(s) 1320A-1320B, cache(s) 1325A-1325B, and circuit interconnect(s) 1330A-1330B of the graphics processor 1310 of FIG. 13A. Graphics processor 1340 includes one or more shader core(s) 1355A-1355N (e.g., 1455A, 1355B, 1355C, 1355D, 1355E, 1355F, through 1355N-1, and 1355N), which provides for a unified shader core architecture in which a single core or type or core can execute all types of programmable shader code, including shader program code to implement vertex shaders, fragment shaders, and/or compute shaders. The unified shader core architecture is also configurable to execute direct compiled high-level GPGPU programs (e.g., CUDA). The exact number of shader cores present can vary among embodiments and implementations. Additionally, graphics processor 1340 includes an inter-core task manager 1345, which acts as a thread dispatcher to dispatch execution threads to one or more shader cores 1355A-1355N and a tiling unit 1358 to accelerate tiling operations for tile-based rendering, in which rendering operations for a scene are subdivided in image space, for example to exploit local spatial coherence within a scene or to optimize use of internal caches. Exemplary Data Processing System Including a Display Engine

FIG. 14 is a block diagram of a data processing system 1400, according to an embodiment. The data processing system 1400 is a heterogeneous processing system having a processor 1402, unified memory 1410, and a GPGPU 1420 including machine learning acceleration logic. The processor 1402 and the GPGPU 1420 can be any of the processors and GPGPU/parallel processors as described herein. For example, with additional reference to FIG. 1, processor 1402 can be a variant of and/or share an architecture with a processor of the illustrated one or more processor(s) 102 and the GPGPU 1420 can be a variant of and/or share an architecture with graphics processor(s) 108.

The processor 1402 can execute instructions for a compiler 1415 stored in system memory 1412. The compiler 1415 executes on the processor 1402 to compile source code 1414A into compiled code 1414B. The compiled code 1414B can include instructions that may be executed by the processor 1402 and/or instructions that may be executed by the GPGPU 1420. Compilation of instructions to be executed by the GPGPU can be facilitated using shader or compute program compilers, such as shader compiler 1027 and/or shader compiler 1024 as in FIG. 10. During compilation, the compiler 1415 can perform operations to insert metadata, including hints as to the level of data parallelism present in the compiled code 1414B and/or hints regarding the data locality associated with threads to be dispatched

based on the compiled code **1414B**. Hints can also be provided as to which processing resources of the GPGPU **1420** (or application processor **1402**) should be used to execute a given set of instructions within the compiled code **1414B**. In one embodiment, API hints can be provided as to a throughput, latency, or power target for instructions within the compiled code **1414B**. In one embodiment, specific instructions will be directed for execution by specific processing resources. The compiler **1415** can include the information necessary to perform such operations or the operations can be performed with the assistance of a runtime library **1416**. The runtime library **1416** can also assist the compiler **1415** in the compilation of the source code **1414A** and can also include instructions that are linked at runtime with the compiled code **1414B** to facilitate execution of the compiled instructions on the GPGPU **1420**. The compiler **1415** can also facilitate register allocation for variables via a register allocator (RA) and generate load and store instructions to move data for variables between memory and the register assigned for the variable.

The unified memory **1410** represents a unified address space that may be accessed by the processor **1402** and the GPGPU **1420**. The unified memory can include system memory **1412** as well as GPGPU memory **1418**. The GPGPU memory **1418** is memory within an address space of the GPGPU **1420** and can include some or all of system memory **1412** and the local memory **1434** of the GPGPU **1420**. In one embodiment the GPGPU memory **1418** can also include at least a portion of any memory accessible by the GPGPU **1420**, such memory in other devices that are accessible to the GPGPU **1420**. In one embodiment, the application processor **1402** can map the compiled code **1414B** stored in system memory **1412** into GPGPU memory **1418** for access by the GPGPU **1420**. In one embodiment, accesses to the unified memory **1410** are coherent accesses, where coherency is maintained via a coherent interconnect such as compute express link (CXL).

The GPGPU **1420** includes multiple compute blocks **1424A-1424N**, which can include one or more of a variety of processing resources described herein. The processing resources can be or include a variety of different computational resources such as, for example, execution units, compute units, streaming multiprocessors, graphics multiprocessors, or multi-core groups, for example, as shown in the various graphics processor architectures described herein. In one embodiment the GPGPU **1420** additionally includes a tensor accelerator **1423** (e.g., matrix accelerator), which can include one or more special function compute units that are designed to accelerate a subset of matrix operations (e.g., dot product, etc.). The tensor accelerator **1423** may also be referred to as a tensor accelerator or tensor core. In one embodiment, logic components within the tensor accelerator **1423** may be distributed across the processing resources of the multiple compute blocks **1424A-1424N**.

The GPGPU **1420** can also include a set of resources that can be shared by the compute blocks **1424A-1424N** and the tensor accelerator **1423**, including but not limited to a power and performance module **1426**, and a cache **1427**. The power and performance module **1426** can be configured to adjust power delivery and clock frequencies for the compute blocks **1424A-1424N** to power gate idle components within the compute blocks **1424A-1424N**. In various embodiments the cache **1427** can include an instruction cache and/or a lower-level data cache.

The GPGPU **1420** can additionally include an L3 data cache **1430**, which can be used to cache data accessed from

the unified memory **1410** by the tensor accelerator **1423** and/or the compute elements within the compute blocks **1424A-1424N**. In one embodiment the L3 data cache **1430** includes shared local memory **1432** that can be shared by the compute elements within the compute blocks **1424A-1424N** and the tensor accelerator **1423**. The GPGPU **1420** can also include a local memory **1434** that is local device memory of the GPGPU **1420**.

In one embodiment the GPGPU **1420** includes instruction handling logic, such as a fetch and decode unit **1421** and a scheduler controller **1422**. The fetch and decode unit **1421** includes a fetch unit and decode unit to fetch and decode instructions for execution by one or more of the compute blocks **1424A-1424N** or the tensor accelerator **1423**. The instructions can be scheduled to the appropriate functional unit within the compute block **1424A-1424N** or the tensor accelerator via the scheduler controller **1422**. In one embodiment the scheduler controller **1422** is an ASIC configurable to perform advanced scheduling operations. In one embodiment the scheduler controller **1422** is a micro-controller or a low energy-per-instruction processing core capable of executing scheduler instructions loaded from a firmware module.

In one embodiment some functions to be performed by the compute blocks **1424A-1424N** can be directly scheduled to or offloaded to the tensor accelerator **1423**. In various embodiments the tensor accelerator **1423** includes processing element logic configured to efficiently perform matrix compute operations, such as multiply and add operations and dot product operations used by 3D graphics or compute shader programs. In one embodiment the tensor accelerator **1423** can be configured to accelerate operations used by machine learning frameworks. In one embodiment the tensor accelerator **1423** is an application specific integrated circuit explicitly configured to perform a specific set of parallel matrix multiplication and/or addition operations. In one embodiment the tensor accelerator **1423** is a field programmable gate array (FPGA) that provides hardware logic that can be updated between workloads. In one embodiment, the set of compute operations that can be performed by the tensor accelerator **1423** may be limited relative to the operations that can be performed by the compute block **1424A-1424N**. However, the tensor accelerator **1423** can perform parallel tensor operations at a significantly higher throughput relative to the compute block **1424A-1424N**.

The GPGPU **1420** also includes a display subsystem that includes a display engine **1425**. In one embodiment, the display engine **1425** is configured to display an output buffer to one or more attached display devices, such as display device **111** of FIG. 1 or display device **318** of FIG. 3A-3B. The output buffer can be a framebuffer that includes pixel data that is rendered by the GPGPU **1420** based on commands provided by an operating system that is executed by the application processor **1402**. The display engine **1425** can receive an interrupt in response to detection of a hotplugged display device to a display interface (e.g., HDMI, Display-Port) of the GPGPU **1420**. A hotplugged display is a display device that is attached while the computing system that includes the GPGPU **1420** is operational. In response to the detected hotplugged display, the display engine **1425** will enable the attached display and configure the attached display to receive framebuffer output.

Display Engine Operation in a Graphics Subsystem

FIG. 15 illustrates display engine operation in a graphics subsystem **1500**. The graphics subsystem **1500** includes a render engine **1506**, a render cache **1504**, video memory **1502**, a display buffer **1508**, and the display engine **1425**.

41

The display engine 1425 couples with physical display interfaces that enable the output of pixel data to various display devices (e.g., display device 1512, display device 1514, display device 1518A-1518B), one or more of which may be connected via a docking station 1516 or indirectly connected via a daisy chain with another display device. The render engine 1506 includes, in one embodiment a render output pipeline, such as the render output pipeline 870 of FIG. 8. The render cache 1504, which may be the render cache 878 of FIG. 8, is a cache memory to which the render engine 1506 outputs pixel data before the pixel data is evicted to the video memory 1502. The video memory 1502, in various embodiments, can be system memory (e.g., system memory 1412) or local memory (e.g., local memory 1434). During operation, the render engine 1506 renders pixel data for a framebuffer to video memory 1502 via a render cache 1504. The display engine 1425 can then isochronously transfer framebuffer data from the video memory 1502 to the display buffer 1508.

The display engine 1425 can include or couple with a display buffer 1508. In one embodiment, the display buffer 1508 is a cache memory within the display engine 1425. The display buffer 1508 may also be referred to as a line buffer and stores scan lines to be output to the display devices. The display engine 1425 can issue fill requests to the video memory 1502 to fill the display buffer 1508. The display engine 1425 then outputs scan lines stored in the display buffer 1508 to the various display devices. The fill rate (e.g., fetch rate) from video memory 1502 to the display buffer 1508 is higher than display engine 1425 scanout (e.g., consumption rate) rate from the display buffer 1508.

As the fill rate is higher than the scanout rate, in one embodiment the display engine 1425 fetches frame buffer data from video memory 1502 into the display buffer 1508 in a race-to-halt fashion. A large amount of frame buffer data can be quickly fetched from video memory 1502 and the video memory 1502 can then be transitioned into a low power state. The display engine 1425 can then output scan lines from the display buffer 1508 to the display device. The rate at which the display buffer 1508 is drained depends on the size (e.g., resolution) of the framebuffer and the refresh rate of the display. Higher resolutions and refresh rates drain the display buffer 1508 more rapidly. In one embodiment, the minimum level of data for the display buffer 1508 and the associated memory request behavior can be configured using a display watermark system. Watermarks are values calculated based on the resolution and refresh rate for the active displays. When a low watermark value is reached, the display engine sends a memory request to the video memory 1502 to replenish the display buffer 1508. If for any reason, such as memory bandwidth constraints, system load, or a delay in bringing the video memory 1502 out of the low power state, the fill rate into the display buffer 1508 may be reduced. If this reduction reduces the fill rate below the consumption rate, an underrun may occur, which can cause visible flashing or blinking to be observed on one or more of the various display devices.

When a new monitor or display device is attached while other display devices are active, the graphics driver and display engine 1425 modify display subsystem resources (e.g., existing display buffer 1508 allocation, memory bandwidth allocation, and watermark programming) to accommodate resources for the newly attached display device. In graphics devices that are configured to immediately enable newly attached display devices before the display subsystem resource reallocation is complete, an underrun may occur, which can cause flickering and/or other display instability.

42

This issue is exacerbated significantly in the event multiple monitors are attached at once, for example, if a docking station 1516 is attached that couples with multiple display devices 1518A-1518B, or if a daisy chained set of display devices is attached, for example, if display device 1514 is attached and display device 1514 is also coupled with display device 1518B (e.g., via USB-C or DisplayPort).

In graphics devices that are configured to delay the enabling of a newly attached display, it may take some time to fully reconfigure the display subsystem, which, in this configuration, can delay the presentation of the framebuffer to the newly attached display. Reallocation and splitting of resources should be done such that there is no impact to existing display operation. For example, ideally the display engine 1425 should not immediately redistribute or reallocate display subsystem resources and should wait until an end of frame is reached and resources are allocated for the newly attached display(s). Otherwise, tearing, flickering, or other display instability will be observed. Additionally, depending on the resolution and refresh rate of the newly attached display devices, the display engine 1425 may be required to increase its internal display clock to meet the updated demands imposed on the display subsystem. This delay may introduce doubt into the user as to whether the monitor hotplug was successful, which may cause the user to detach and re-attached the displays or docking station 1516, which further delays the activation of the newly attached displays.

In one embodiment, when a new display is attached, the display buffer 1508 is re-partitioned to support the existing display as well as the newly attached display. This re-partitioning can increase the amount of time required to reconfigure the display subsystem for an updated display configuration and is not done properly, can cause flickering or instability on currently active displays.

FIG. 16 illustrates a display subsystem 1600 with updated hotplug handling for display devices. The display subsystem 1600 for the graphics subsystem 1500 of FIG. 15 can be configured to activate newly attached displays upon hotplug of the displays without disrupting output that is provided to currently active displays and while providing an indication to a user that the newly attached display or displays have been detected and also providing the display subsystem 1600 time to reallocate resources (e.g., display buffer 1508 allocation, memory bandwidth allocation, and watermark programming) to support the updated display configuration.

In one embodiment, upon detection of a hotplug event, the display subsystem 1600, via display engine 1425 logic or graphics driver logic, can program (1603) a background color 1604, such that even if there is no other data is output to the display, a background color can be output. This feature enables the display to be enabled after the appropriate display timings are configured, even if no data is ready for output to the display, allowing the newly attached display devices to be rapidly enabled without disrupting active displays, as the newly attached displays are able to be enabled before those displays are configured to receive framebuffer data. In this embodiment, an indication is provided to the user that the display has been detected. However, the lack of any information being presented on the newly attached display may lead some users to believe that the display has not been successfully enabled.

In one embodiment, the display subsystem 1600 can program or populate a message (1601) into a reserved portion 1602 the display buffer 1508. This message can be in the form of pixel data that presents a message (e.g., "Coming Up", "Please Wait", "Configuring Display"), an

image (e.g., an hourglass image, a product logo, a vendor logo) or another indicator that shows that the hotplugged display or displays have been detected. The pixel data in the reserved portion **1602** of the display buffer **1508** can be blended with the background color via a display blender **1608**, that will be output to the newly attached display device (e.g., display device **1514**).

Depending on the display mode (e.g., resolution, refresh, depth etc.) of the newly attached display device, the pixel data in the reserved portion **1602** of the display buffer **1508** can be output to a display scaler **1606** to upscale the pixel data. In one embodiment, the pixel data can be provided to the display scaler **1606** before the pixel data is provided to the display blender **1608**. In one embodiment, the display scaler can be positioned after the display blender **1608**. The pixel data in the reserved portion **1602** of the display buffer **1508** may bypass the display scaler **1606** for a smaller display or a display configured for a relatively smaller mode (e.g., resolution, refresh, depth etc.), while the pixel data may be provided to the display scaler **1606** for a larger display or a display configured for a larger mode (e.g., resolution, refresh, etc.).

In various embodiments, the reserved portion **1602** of the display buffer **1508** can be a persistently reserved portion or a portion that is reserved upon detection of a hotplugged display. In one embodiment, a small amount of additional or otherwise unused memory within the display controller can be adapted to store pixel data to be output to newly attached display. In one embodiment, hardware of the display engine **1425** can be configured to programmatically drive a pattern or image to newly attached display devices while the display subsystem **1600** is reconfigured. For embodiments that create the reserved portion **1602** in the display buffer **1508** upon detection of a hotplug event, the relatively small size of the reserved portion **1602** relative to the size of the display buffer **1508** can allow the creation of the portion to be performed without disruption to active display devices, even in embodiments in which the entire display buffer **1508** is in use due to over-allocation. For example, in one embodiment in which the entire display buffer **1508** is allocated, upon detection of a hotplug event, the high-watermark of the display buffer can be reduced and upon the next fill event from video memory **1502**, less than the entire display buffer will be filled, leaving space for the reserved portion **1602** to be generated. This operation can occur without perceptible display to the user. When reconfiguration is complete, the reserved portion can be removed, enabling the entire display buffer **1508** to be used to drive the updated set of displays.

FIG. 17 illustrates a method **1700** of enabling and providing a notification to a user for hotplugged displays without disrupting output of currently attached displays. The method **1700** can be performed by control logic of the display subsystem of a graphics processor, such as the display subsystem **1600** of FIG. 16. The control logic of the display subsystem includes graphics driver logic, graphics processor firmware, and hardware logic within the display engine (e.g., display engine **1425**) of the graphics processor. Each of the graphics driver logic, firmware, or display engine, alone, or in combination, can be configured to perform the method **1700**.

In one embodiment, method **1700** includes operations to detect a connection of a second or subsequent display device to the graphics subsystem (e.g., via the display subsystem of the graphics subsystem) while presenting framebuffer data from the operating system to a first display device of the graphics subsystem (**1702**). The second display device can represent a single newly attached display device or multiple

newly attached display devices. For example, a docking station or daisy-chained set of display devices can be attached. The one or more second display devices can be detected based on a hotplug interrupt that is raised by the display engine upon attachment of a display to a physical display interface. Where a second display device is described, those operations can also be performed for any subsequent display devices that are attached.

In one embodiment, the graphics subsystem can then reserve a portion of memory associated with a graphics subsystem to store pixel data and then write pre-determined pixel data to the reserved portion of memory (**1704**). In other embodiments, the reserved portion of memory may be reserved prior to the hotplug event or may be persistently reserved memory. The reserved portion of memory, in one embodiment, is a portion of a display buffer (e.g., display buffer **1508**), which may be a cache memory within the display engine. The pre-determined pixel data can include a text message or an image that will be presented on the one or more newly attached second display device to indicate that those displays are in the process of being enabled. In one embodiment, the reserved portion of the memory with pre-determined pixel data can be utilized for multiple displays at the same time.

The graphics subsystem can then configure timings for the second or subsequent display device while maintaining resources allocated to the first display device (**1706**). The timings for the second display device can be configured according to one or more of the native resolution of the display device or a resolution selected from one of multiple supported resolutions of the display device. In one embodiment, the timings can be configured for the lowest supported resolution of the display device, or the resolution that is enabled after the power on self-test (POST) for the system at initial startup.

The graphics subsystem can then display the pixel data from the reserved portion of the memory on the second or subsequent display device while reallocating resources of the graphics subsystem to enable output the framebuffer data to the second or subsequent display device (**1708**). In one embodiment, to display the pixel data on the second or subsequent display device includes programing a background color for display, outputting the pixel data in the reserved portion of the memory to a display blender, blending the background color with the pixel data via the display blender, and providing output from the display blender to the newly attached display device. If the pixel data should be upscaled, the pixel data in the reserved portion of memory can be provided to a display scaler and the output of the display scaler can be provided to the display blender.

The reallocation can include re-allocating display subsystem memory, such as partitioning the display buffer to support multiple displays. The reallocation can also include adjusting a clock speed of the display engine based on the resolution and refresh rate of the second or subsequent display device. The operating system will also be notified of the newly attached display. In some embodiments, the graphic subsystem can wait until the end of a frame currently displayed on the first display device before reallocating resources of the graphics subsystem to enable output the framebuffer data to the second display device. In one embodiment, the graphics subsystem can wait until a framebuffer update is received from the operating system before reallocating resources of the graphics subsystem.

The graphics subsystem can then transition the second or subsequent display device from displaying the pixel data in the reserved portion of the memory to displaying the frame-

buffer data after resources of the graphics subsystem are reallocated (1710). After the transition, the second display device will be configured to receive framebuffer data, such as a desktop of an operating system or application that is presented via the graphics subsystem. In some embodiments, the graphics subsystem can then release the reserved portion of memory (1712). The release of the reserved portion of memory can occur in concert with the transition of the second display device to displaying framebuffer data. For example, the reconfigured display buffer can be configured such that, after reconfiguration of the display buffer, fetched data from the video memory will overwrite the reserved portion.

FIG. 18A-18B illustrate a disaggregated 3D-structured SoC architecture of a graphics processor SoC 1800, according to various embodiments. FIG. 18A illustrates a components of a graphics processor SoC 1800, according to embodiments described herein. FIG. 18B illustrates an implementation of the graphics processor SoC 1800 in which the low power compute engine described herein is included in a system die 1806.

As shown in FIG. 18A, the graphics processor SoC 1800 includes a package substrate 1802 having an active base die 1804 that includes a level 4 (L4) cache memory. The active base die 1804 interfaces with a compute die 1807 and a system die 1806 that are positioned on top of the active base die 1804. The graphics processor SoC 1800 includes memory interconnects 1808A-1808B that couple local device memory to the active base die 1804 and the system die 1806. The local device memory can be low power double data rate (LPDDR) or graphics DDR (GDDR) memory. In some embodiments, the local device memory can also be high bandwidth memory (HBM).

FIG. 18B shows additional architectural details for the active base die 1804, system die 1806, and compute die 1807, according to one embodiment. In one embodiment the active base die 1804 includes a set of die interconnects 1811A-1811D that couple circuitry within the active base die 1804 to the system die 1806 and the compute die 1807. The active base die 1804 also includes an L4 cache having a set of L4 cache blocks 1812A-1812F and an L4 cache controller 1813. The L4 cache controller 1813 caches data associated with memory accesses to the local device memory within the L4 cache blocks 412A-1812F. The number of L4 cache blocks 1812A-1812F can vary based on the size of the L4 cache and L4 cache can be sized proportionally to the size of the local device memory. In one embodiment local memory accesses performed by the compute engine 1814 and the media engine 1836 are serviced via the L4 cache, with the L4 cache controller 1813 accessing the local device memory in the event of a cache miss. The L4 cache controller 1813 accesses the local device memory via a memory interface 1830 that connects with the local device memory via memory interconnects 1808A-1808B.

The compute die 1807 includes a compute engine 1814, L4 interface 1815, and multiple compute express link (CXL) channels 1816A-1816B. The compute engine 1814 includes general-purpose graphics processing elements in the form of one or more instances of the compute blocks 1424A-1424N of FIG. 14. The compute engine 1814 is disaggregated from other components of the graphics processor SoC 1800, which enables a modular architecture in which the processing capability of the graphics processor SoC 1800 can be easily adjusted via the use of different implementations of the compute die 1807. Additionally, different process technologies and/or different manufacturers can be used to manufacture different implementations of the compute die

1807, without requiring significant adjustments to the active base die 1804 or system die 1806. The L4 interface 1815 facilitates access by the compute engine 1814 to the L4 cache. Cached memory accesses performed by the compute engine 1814 to local device memory can be serviced via the L4 interface 1815. The CXL channels 1816A-1816B enable coherent access to a common memory space that includes both local device memory and system memory.

The system die 1806 includes multiple CXL channels for various CXL protocols, including CXL.io, CXL.cache and/or CXL.memory, and include the CXL channels 1816A-1816B. The system die 1806 also includes a system interface 1820, display engine 1821, and a system fabric 1825. The system fabric 1825 includes primary switch fabrics for the graphics processor SoC 1800. The system die 1806 also includes the media engine 1836, which includes functional units to perform media encode and decode operations. The display engine 1821 facilitates presentation of framebuffer memory and enables control of display devices that are coupled over various physical display interfaces.

Additional Exemplary Computing Device

FIG. 19 is a block diagram of a computing device 1900 including a graphics processor 1904, according to an embodiment. Versions of the computing device 1900 may be or be included within a communication device such as a set-top box (e.g., Internet-based cable television set-top boxes, etc.), global positioning system (GPS)-based devices, etc. The computing device 1900 may also be or be included within mobile computing devices such as cellular phones, smartphones, personal digital assistants (PDAs), tablet computers, laptop computers, e-readers, smart televisions, television platforms, wearable devices (e.g., glasses, watches, bracelets, smartcards, jewelry, clothing items, etc.), media players, etc. For example, in one embodiment, the computing device 1900 includes a mobile computing device employing an integrated circuit, such as system on a chip integrated circuit that integrates various hardware and/or firmware components of computing device 1900 on a single chip. The computing device 1900 can be a computing device such as the processing system 100 as in of FIG. 1.

The computing device 1900 includes a graphics processor 1904. The graphics processor 1904 represents any graphics processor described herein. In one embodiment, the graphics processor 1904 includes a cache 1914, which can be a single cache or divided into multiple segments of cache memory, including but not limited to any number of L1, L2, L3, or L4 caches, render caches, depth caches, sampler caches, and/or shader unit caches. In one embodiment the cache 1914 may be a last level cache that is shared with the application processor 1906.

In one embodiment the graphics processor 1904 includes a graphics microcontroller that implements control and scheduling logic for the graphics processor. The control and scheduling logic can be firmware executed by the graphics microcontroller 1915. The firmware may be loaded at boot by the graphics driver logic 1922. The firmware may also be programmed to an electronically erasable programmable read only memory or loaded from a flash memory device within the graphics microcontroller 1915. The firmware may enable a GPU OS 1916 that includes device management logic 1917, device driver logic 1918, and a scheduler 1919. The GPU OS 1916 may also include a graphics memory manager 1920 that can supplement or replace the graphics memory manager 1921 within the graphics driver logic 1922.

The graphics processor 1904 also includes a GPGPU engine 1944 that includes one or more graphics engine(s),



graphics processor cores, and other graphics execution resources as described herein. Such graphics execution resources can be presented in the forms including but not limited to execution units, shader engines, fragment processors, vertex processors, streaming multiprocessors, graphics processor clusters, or any collection of computing resources suitable for the processing of graphics resources or image resources or performing general purpose computational operations in a heterogeneous processor. The processing resources of the GPGPU engine **1944** can be included within multiple tiles of hardware logic connected to a substrate, as illustrated in FIG. **11B-11D**. The GPGPU engine **1944** can include GPU tiles **1945** that include graphics processing and execution resources, caches, samplers, etc. The GPU tiles **1945** may also include local volatile memory or can be coupled with one or more memory tiles, for example, as shown in FIG. **3B-3C**.

The GPGPU engine **1944** can also include one or more special tiles **1946** that include, for example, a non-volatile memory tile **1956**, a network processor tile **1957**, and/or a general-purpose compute tile **1958**. The GPGPU engine **1944** can also include a matrix multiply accelerator **1960**. The non-volatile memory tile **1956** can include non-volatile memory cells and controller logic. The controller logic of the non-volatile memory tile **1956** may be managed by one of device management logic **1917** or device driver logic **1918**. The network processor tile **1957** can include network processing resources that are coupled to a physical interface within the input/output (I/O) sources **1910** of the computing device **1900**. The network processor tile **1957** may be managed by one or more of device management logic **1917** or device driver logic **1918**.

The matrix multiply accelerator **1960** is a modular scalable sparse matrix multiply accelerator as described herein. The matrix multiply accelerator **1960** can include multiple processing paths, with each processing path including multiple pipeline stages. Each processing path can execute a separate instruction. In various embodiments, the matrix multiply accelerator **1960** can have architectural features of any one of more of the matrix multiply accelerators described herein. For example, in one embodiment, the matrix multiply accelerator **1960** is a four-deep systolic array with a feedback loop that is configurable to operate with a multiple of four number of logical stages (e.g., four, eight, twelve, sixteen, etc.). In one embodiment the matrix multiply accelerator **1960** includes one or more instances of a two-path matrix multiply accelerator with a four stage pipeline or a four-path matrix multiply accelerator with a two stage pipeline. In one embodiment the matrix multiply accelerator **1960** includes processing elements configured as a scalable sparse matrix multiply accelerator that is configured to operate only on non-zero values of at least one input matrix. Operations on entire columns or submatrices can be bypassed where block sparsity is present. The matrix multiply accelerator **1960** can also include any logic based on any combination of these embodiments. The matrix multiply accelerator **1960** is also configurable to perform matrix computations for the machine learning models described herein, including models configured to perform instance segmentation and style transfer. In various embodiments, the matrix multiply accelerator **1960** can accelerate matrix operations performed during training and inference.

As illustrated, in one embodiment, and in addition to the graphics processor **1904**, the computing device **1900** may further include any number and type of hardware components and/or software components, including, but not limited to an application processor **1906**, memory **1908**, and

input/output (I/O) sources **1910**. The application processor **1906** can interact with a hardware graphics pipeline, as illustrated with reference to FIG. **3A**, to share graphics pipeline functionality. Processed data is stored in a buffer in the hardware graphics pipeline and state information is stored in memory **1908**. The resulting data can be transferred to a display controller for output via a display device, such as the display device **318** of FIG. **3A**. The display device may be of various types, such as Cathode Ray Tube (CRT), Thin Film Transistor (TFT), Liquid Crystal Display (LCD), Organic Light Emitting Diode (OLED) array, etc., and may be configured to display information to a user via a graphical user interface.

The application processor **1906** can include one or processors, such as processor(s) **102** of FIG. **1** and may be the central processing unit (CPU) that is used at least in part to execute an operating system (OS) **1902** for the computing device **1900**. The OS **1902** can serve as an interface between hardware and/or physical resources of the computing device **1900** and one or more users. The OS **1902** can include driver logic for various hardware devices in the computing device **1900**. The driver logic can include graphics driver logic **1922**, which can include the user mode graphics driver **1026** and/or kernel mode graphics driver **1029** of FIG. **10**. The graphics driver logic can include a graphics memory manager **1921** to manage a virtual memory address space for the graphics processor **1904**.

It is contemplated that in some embodiments the graphics processor **1904** may exist as part of the application processor **1906** (such as part of a physical CPU package) in which case, at least a portion of the memory **1908** may be shared by the application processor **1906** and graphics processor **1904**, although at least a portion of the memory **1908** may be exclusive to the graphics processor **1904**, or the graphics processor **1904** may have a separate store of memory. The memory **1908** may comprise a pre-allocated region of a buffer (e.g., framebuffer); however, it should be understood by one of ordinary skill in the art that the embodiments are not so limited, and that any memory accessible to the lower graphics pipeline may be used. The memory **1908** may include various forms of random-access memory (RAM) (e.g., SDRAM, SRAM, etc.) comprising an application that makes use of the graphics processor **1904** to render a desktop or 3D graphics scene. A memory controller hub, such as memory controller **116** of FIG. **1**, may access data in the memory **1908** and forward it to graphics processor **1904** for graphics pipeline processing. The memory **1908** may be made available to other components within the computing device **1900**. For example, any data (e.g., input graphics data) received from various I/O sources **1910** of the computing device **1900** can be temporarily queued into memory **1908** prior to their being operated upon by one or more processor(s) (e.g., application processor **1906**) in the implementation of a software program or application. Similarly, data that a software program determines should be sent from the computing device **1900** to an outside entity through one of the computing system interfaces, or stored into an internal storage element, is often temporarily queued in memory **1908** prior to its being transmitted or stored.

The I/O sources can include devices such as touchscreens, touch panels, touch pads, virtual or regular keyboards, virtual or regular mice, ports, connectors, network devices, or the like, and can attach via a platform controller hub **130** as referenced in FIG. **1**. Additionally, the I/O sources **1910** may include one or more I/O devices that are implemented for transferring data to and/or from the computing device **1900** (e.g., a networking adapter); or, for a large-scale



non-volatile storage within the computing device 1900 (e.g., SSD/HDD). User input devices, including alphanumeric and other keys, may be used to communicate information and command selections to graphics processor 1904. Another type of user input device is cursor control, such as a mouse, a trackball, a touchscreen, a touchpad, or cursor direction keys to communicate direction information and command selections to GPU and to control cursor movement on the display device. Camera and microphone arrays of the computing device 1900 may be employed to observe gestures, record audio and video and to receive and transmit visual and audio commands.

The I/O sources 1910 can include one or more network interfaces. The network interfaces may include associated network processing logic and/or be coupled with the network processor tile 1957. The one or more network interface can provide access to a LAN, a wide area network (WAN), a metropolitan area network (MAN), a personal area network (PAN), Bluetooth, a cloud network, a cellular or mobile network (e.g., 3<sup>rd</sup> Generation (3G), 4<sup>th</sup> Generation (4G), 5<sup>th</sup> Generation (5G), etc.), an intranet, the Internet, etc. Network interface(s) may include, for example, a wireless network interface having one or more antenna (e). Network interface(s) may also include, for example, a wired network interface to communicate with remote devices via network cable, which may be, for example, an Ethernet cable, a coaxial cable, a fiber optic cable, a serial cable, or a parallel cable.

Network interface(s) may provide access to a LAN, for example, by conforming to IEEE 802.11 standards, and/or the wireless network interface may provide access to a personal area network, for example, by conforming to Bluetooth standards. Other wireless network interfaces and/or protocols, including previous and subsequent versions of the standards, may also be supported. In addition to, or instead of, communication via the wireless LAN standards, network interface(s) may provide wireless communication using, for example, Time Division, Multiple Access (TDMA) protocols, Global Systems for Mobile Communications (GSM) protocols, Code Division, Multiple Access (CDMA) protocols, and/or any other type of wireless communications protocols.

It is to be appreciated that a lesser or more equipped system than the example described above may be preferred for certain implementations. Therefore, the configuration of the computing devices described herein may vary from implementation to implementation depending upon numerous factors, such as price constraints, performance requirements, technological improvements, or other circumstances. Examples include (without limitation) a mobile device, a personal digital assistant, a mobile computing device, a smartphone, a cellular telephone, a handset, a one-way pager, a two-way pager, a messaging device, a computer, a personal computer (PC), a desktop computer, a laptop computer, a notebook computer, a handheld computer, a tablet computer, a server, a server array or server farm, a web server, a network server, an Internet server, a work station, a mini-computer, a main frame computer, a supercomputer, a network appliance, a web appliance, a distributed computing system, multiprocessor systems, processor-based systems, consumer electronics, programmable consumer electronics, television, digital television, set top box, wireless access point, base station, subscriber station, mobile subscriber center, radio network controller, router, hub, gateway, bridge, switch, machine, or combinations thereof.

It is to be noted that terms like “node”, “computing node”, “server”, “server device”, “cloud computer”, “cloud server”,

“cloud server computer”, “machine”, “host machine”, “device”, “computing device”, “computer”, “computing system”, and the like, may be used interchangeably throughout this document. It is to be further noted that terms like “application”, “software application”, “program”, “software program”, “package”, “software package”, and the like, may be used interchangeably throughout this document. Also, terms like “job”, “input”, “request”, “message”, and the like, may be used interchangeably throughout this document.

Embodiments may be provided, for example, as a computer program product which may include one or more machine-readable media having stored thereon machine-executable instructions that, when executed by one or more machines such as a computer, network of computers, or other electronic devices, may result in the one or more machines carrying out operations in accordance with embodiments described herein. A machine-readable medium may include, but is not limited to, floppy diskettes, optical disks, CD-ROMs (Compact Disc-Read Only Memories), and magneto-optical disks, ROMs, RAMs, EPROMs (Erasable Programmable Read Only Memories), EEPROMs (Electrically Erasable Programmable Read Only Memories), magnetic or optical cards, flash memory, or other type of media/machine-readable medium suitable for storing machine-executable instructions.

Moreover, embodiments may be downloaded as a computer program product, wherein the program may be transferred from a remote computer (e.g., a server) to a requesting computer (e.g., a client) by way of one or more data signals embodied in and/or modulated by a carrier wave or other propagation medium via a communication link (e.g., a modem and/or network connection).

Throughout the document, term “user” may be interchangeably referred to as “viewer”, “observer”, “person”, “individual”, “end-user”, and/or the like. It is to be noted that throughout this document, terms like “graphics domain” may be referenced interchangeably with “graphics processing unit”, “graphics processor”, or simply “GPU” and similarly, “CPU domain” or “host domain” may be referenced interchangeably with “computer processing unit”, “application processor”, or simply “CPU”.

It is to be noted that terms like “node”, “computing node”, “server”, “server device”, “cloud computer”, “cloud server”, “cloud server computer”, “machine”, “host machine”, “device”, “computing device”, “computer”, “computing system”, and the like, may be used interchangeably throughout this document. It is to be further noted that terms like “application”, “software application”, “program”, “software program”, “package”, “software package”, and the like, may be used interchangeably throughout this document. Also, terms like “job”, “input”, “request”, “message”, and the like, may be used interchangeably throughout this document.

References herein to “one embodiment,” “an embodiment,” “an example embodiment,” etc., indicate that the embodiment described may include a particular feature, structure, or characteristic, but every embodiment may not necessarily include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it is submitted that it is within the knowledge of one skilled in the art to affect such feature, structure, or characteristic in connection with other embodiments whether explicitly described.

In the various embodiments described above, unless specifically noted otherwise, disjunctive language such as the phrase “at least one of A, B, or C” is intended to be

## 51

understood to mean either A, B, or C, or any combination thereof (e.g., A, B, and/or C). As such, disjunctive language is not intended to, nor should it be understood to, imply that a given embodiment requires at least one of A, at least one of B, or at least one of C to each be present.

One embodiment provides a graphics processor device that includes circuitry configured to detect a connection of a second display device to a display subsystem of the graphics processor while a first display device of the graphics subsystem is active, write pre-determined pixel data to a reserved portion of memory associated with the display subsystem, configure timings for the second display device while resources allocated to the first display device remain available to the first display device, display the pre-determined pixel data from the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem to enable output the framebuffer data to the second display device, and transition the second display device from the display of the pixel data in the reserved portion of the memory to the display of the framebuffer data after resources of the display subsystem are reallocated. In one embodiment, the graphics processor is included in an apparatus, such as a graphics processor add-in card or a general-purpose processor that includes an integrated graphics device.

One embodiment provides a method comprising detecting a connection of a second display device to a display subsystem of a graphics processor while a first display device of the graphics subsystem is active; writing pre-determined pixel data to a reserved portion of memory associated with the display subsystem; configuring timings for the second display device while resources allocated to the first display device remain available to the first display device; displaying the pre-determined pixel data from the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem to enable output the framebuffer data to the second display device; and transitioning the second display device from the display of the pixel data in the reserved portion of the memory to the display of the framebuffer data after resources of the display subsystem are reallocated.

One embodiment provides a data processing system including means to perform a method as described herein.

The specification and drawings are, accordingly, to be regarded in an illustrative rather than a restrictive sense. Those skilled in the art will appreciate that the broad techniques of the embodiments described herein can be implemented in a variety of forms. Therefore, while the embodiments have been described in connection with examples thereof, the true scope of the embodiments should not be so limited since other modifications will become apparent to the skilled practitioner upon a study of the drawings, specification, and following claims.

What is claimed is:

1. An apparatus comprising:

a graphics processor device including circuitry configured to:

detect a connection of a second display device to a display subsystem of the graphics processor while a first display device of the graphics subsystem is active;

## 52

write pre-determined pixel data to a reserved portion of memory associated with the display subsystem; configure timings for the second display device while resources allocated to the first display device remain available to the first display device;

display the pre-determined pixel data from the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem to enable output framebuffer data to the second display device; and

transition the second display device from the display of the pixel data in the reserved portion of the memory to the display of the framebuffer data after resources of the display subsystem are reallocated.

2. The apparatus as in claim 1, wherein the circuitry is additionally configured to release the reserved portion of the memory.

3. The apparatus as in claim 1, wherein the memory associated with the display subsystem is memory of a display buffer associated with a display engine of the display subsystem.

4. The apparatus as in claim 3, wherein the display buffer is a cache memory included within the display engine.

5. The apparatus as in claim 3, the circuitry configured to adjust a clock speed of the display engine based on a resolution and refresh rate of the second display device.

6. The apparatus as in claim 1, wherein the pixel data written to the reserved portion of the memory includes a text message to indicate that the second display device has been connected to the display subsystem.

7. The apparatus as in claim 1, wherein the pixel data written to the reserved portion of the memory includes an image to indicate that the second display device has been connected to the display subsystem.

8. The apparatus as in claim 1, wherein the circuitry is configured to wait until the end of a frame currently displayed on the first display device before reallocation of resources of the display subsystem to enable output the framebuffer data to the second display device.

9. The apparatus as in claim 1, wherein to display the pixel data in the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem, the circuitry is configured to:

program a background color to display on the second display device;

blend the background color with pixel data from the reserved portion of memory via a display blender; and provide output from the display blender to the second display device.

10. The apparatus as in claim 9, wherein the circuitry is configured to scale the pixel data from the reserved portion of memory via a display scaler.

11. A method comprising:

detecting a connection of a second display device to a display subsystem of a graphics processor while a first display device of the graphics subsystem is active;

writing pre-determined pixel data to a reserved portion of memory associated with the display subsystem;

configuring timings for the second display device while resources allocated to the first display device remain available to the first display device;

displaying the pre-determined pixel data from the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem to enable output framebuffer data to the second display device; and

53

transitioning the second display device from the display of the pixel data in the reserved portion of the memory to the display of the framebuffer data after resources of the display subsystem are reallocated.

12. The method as in claim 11, further comprising releasing the reserved portion of the memory. 5

13. The method as in claim 11, wherein the memory associated with the display subsystem is memory of a display buffer associated with a display engine of the display subsystem and the display buffer is a cache memory included within the display engine. 10

14. The method as in claim 13, further comprising adjusting a clock speed of the display engine based on a resolution and refresh rate of the second display device.

15. The method as in claim 11, wherein the pixel data written to the reserved portion of the memory includes a text message or an image and the text message or image indicate that the second display device has been connected to the display subsystem. 15

16. The method as in claim 11, further comprising waiting until the end of a frame currently displayed on the first display device before reallocation of resources of the display subsystem to enable output the framebuffer data to the second display device. 20

17. The method as in claim 11, wherein displaying the pixel data in the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem includes: 25

programming a background color to display on the second display device;

blending the background color with pixel data from the reserved portion of memory via a display blender; and

providing output from the display blender to the second display device. 30

18. The method as in claim 17, further comprising scaling the pixel data from the reserved portion of memory via a display scaler. 35

54

19. A data processing system comprising:  
a memory device; and

a graphics processor including circuitry configured to:  
detect a connection of a second display device to a display subsystem of the graphics processor while a first display device of the graphics subsystem is active;

write pre-determined pixel data to a reserved portion of memory associated with the display subsystem;

configure timings for the second display device while resources allocated to the first display device remain available to the first display device;

display the pre-determined pixel data from the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem to enable output framebuffer data to the second display device; and

transition the second display device from the display of the pixel data in the reserved portion of the memory to the display of the framebuffer data after resources of the display subsystem are reallocated.

20. The data processing system as in claim 19, wherein the memory associated with the display subsystem is memory of a display buffer associated with a display engine of the display subsystem, the display buffer is a cache memory included within the display engine, and to display the pixel data in the reserved portion of the memory on the second display device during reallocation of the resources of the display subsystem includes to: 25

program a background color to display on the second display device;

blend the background color with pixel data from the reserved portion of memory via a display blender; and provide output from the display blender to the second display device. 30

\* \* \* \* \*